# Kotlin

**var/val** - мутабельная и иммутабельная переменная соответственно.
**val** можно изменять только через состояние объекта.

**const val/companion object** - статические переменные, доступ к которым можно получить не имея объект класса.
**const val** - compile-time константа (аналог final static в JVM).
**val и companion object** - runtime константа.

**data class** - это обертка над обычным классом, которая упрощает работу с некоторыми из них, которые предназначены для хранения данных. Data class имеют свои переопределенные методы - *toString, equals, hashcode, copy, componentN.*
Если переопределить хотя бы один из *toString, equals, hashcode, copy* - остальные не сгенерируются автоматически.
Эти методы генерируются только по свойствам из основного (primary) конструктора.
Объявить **var** в **primary constructor data class** можно, но плохо: equals()/hashCode() зависят от изменяемого поля → объект может "потеряться" в Set/Map после мутации.

**Деструктуризация** - это фича Kotlin, которая позволяет «распаковать» объект сразу в несколько переменных. Самый частый и понятный пример — с data class (component1(), component2() и т.д.).

```
// Деструктуризация:
val (name, age, email) = user
```

Обычные классы тоже могут использовать деструктуризацию, если переопределить все *operator fun component1() = x;* в конструкторе напрямую.

Inline usecases
Модификатор **inline** заставляет компилятор Kotlin (не JVM) встраивать тело функции прямо в место вызова — то есть вместо вызова функции подставляется её код.
Это происходит на этапе компиляции Kotlin → байт-код, а не на уровне JVM.
Зачем это нужно?
Главное преимущество — устранение оверхеда при работе с лямбдами.
Без inline лямбда превращается в объект (создаётся анонимный класс), что дорого по памяти и времени.
С inline — лямбда встраивается как обычный код, и объекта не создаётся.

Когда **inline** используют почти всегда:

- let, apply, also, run, with — все они inline
- Любая функция, принимающая лямбду как параметр и вызывающая её напрямую
- Особенно важно для high-order functions

Когда **inline** НЕЛЬЗЯ использовать:

- Если функция содержит return из внешней функции (обычно запрещено)
- Если функция большая — инлайнинг раздувает код

Бонус: noinline и crossinline

- **crossinline** — если внутри инлайн-функции лямбда передаётся дальше (например, в другую функцию), и нельзя делать non-local return
- **noinline** — если одну из лямбд не хочешь инлайнить

Когда использовать **crossinline**:

- Лямбда передаётся в другую функцию (в т.ч. в run, launch, async, Thread, setOnClickListener и т.д.)
- Ты хочешь запретить return из лямбды (чтобы не сломать логику внешней функции)

Когда использовать **noinline**:
У inline-функции все лямбды по умолчанию инлайнятся.
Но иногда одну из лямбд нельзя или не хочется инлайнить.
Случай 1: Лямбда сохраняется как объект (например, в свойство)
Случай 2: Лямбда слишком большая — не хочется раздувать код

Оператор **!!** - форсирует компилятор воспринимать переменную как nonnullable.
**!!** - бросает KotlinNullPointerException, если значение null.

**==** в Kotlin — это структурное сравнение (вызывает .equals())
**===** это сравнение ссылок (то же, что == в Java).

**Extension function** — это функция, которая добавляет поведение к существующему классу без наследования и изменения его кода.
Паттерн проектирования, который реализуют extension functions — Decorator (в открытой форме) или чаще говорят Visitor, но в сообществе Kotlin чаще всего называют это реализацией паттерна Extension / Open/Closed Principle (открыт для расширения, закрыт для модификации).

Главное преимущество **sealed class** — исчерпывающая проверка в when (exhaustive when).
Компилятор обязывает обработать все возможные подклассы, иначе — ошибка компиляции.

Enum тоже это делает, но:

- enum — только фиксированные экземпляры (один объект на значение)
- sealed class — каждый подтип может иметь разные свойства и состояние (как обычные классы)

Когда обязательно использовать sealed class вместо обычного:
Когда ты хочешь моделировать ограниченное множество типов (как Result, ViewState, UiEvent и т.д.) и нужна исчерпывающая обработка в when + каждое состояние может нести свои данные.
Пример:

```
sealed class Result {
    data class Success(val data: String) : Result()
    data class Error(val exception: Throwable) : Result()
    object Loading : Result()
}
```

**Any** — корень всей иерархии типов Kotlin (как Object в Java). Все классы неявно наследуются от Any. Не-nullable по умолчанию.
**Unit** — тип, обозначающий «функция ничего полезного не возвращает». Имеет единственное значение — Unit (объект-одиночка). Аналог void в Java, но является настоящим типом. Пример: fun printHello(): Unit { println("Hello") } (можно опустить : Unit)
**Nothing** — тип, у которого нет значений. Означает «функция никогда не завершится нормально». Используется для:

- функций, которые всегда бросают исключение: fun fail(): Nothing = throw RuntimeException()
- бесконечных циклов
- мест, где компилятор понимает, что код недостижим

**Nothing?** — nullable версия Nothing. Единственное возможное значение — null. Полезен в generics для обозначения «список, который всегда пуст»: List<Nothing?>

Только **const val** — настоящая compile-time константа, которую можно использовать в аннотациях.
**@JvmStatic** val в companion — даёт статическое поле + статический геттер, удобно вызывать из Java как MyClass.VALUE.
**@JvmField** val — убирает геттер полностью, поле становится как обычное public поле Java. Часто используют в data class-ах, когда нужно совместимость с фреймворками (Gson, Jackson и т.д.).

По умолчанию в **Kotlin**:

- Все классы — final (нельзя наследоваться).
- Все функции и свойства — final (нельзя переопределять).

**open class / open fun**:

- open class — разрешает наследование от этого класса.
- open fun / open val — разрешает переопределять эту функцию или свойство в дочерних классах.
- Используется редко — только когда ты явно хочешь разрешить переопределение.

**abstract class / abstract fun**:

- abstract class — нельзя создать объект напрямую, обязательно нужно унаследовать и реализовать абстрактные члены.
- abstract fun / abstract val — не имеют тела/значения, обязательно переопределять в наследнике.
- Может содержать и обычные функции с реализацией, и состояние (поля).
- Используется, когда есть общее поведение + нужно заставить наследников реализовать что-то своё.

**interface**:

- По умолчанию все функции — public open (можно переопределять, но не обязательно).
- С Kotlin 1.2+ можно писать функции с телом (реализация по умолчанию).
- До Kotlin 1.4 нельзя было иметь поля с бэкингом (только геттеры/сеттеры), сейчас можно только private val.
- Класс может реализовывать сколько угодно интерфейсов.
- Используется для описания поведения (контракта), особенно когда нужна множественная реализация.

Краткое правило выбора:

- Нужна множественная реализация → interface
- Есть общее состояние или защищённые члены → abstract class
- Хочешь запретить наследование по умолчанию → просто class (final)
- Редко: хочешь контролируемо разрешить наследование → open class + open fun

**public**

- Везде и всегда видно: из любого файла, пакета, модуля.
- Это модификатор по умолчанию (если ничего не написать).

**internal**

- Видно только внутри одного модуля (в Gradle — один проект = один модуль).
- Не зависит от пакета: даже если классы в разных пакетах, но в одном модуле — видно друг другу.
- Если вынесешь в другой модуль — станет невидимым.
- Главное отличие от Java package-private.

**protected**

- Нельзя использовать на top-level (вне класса).
- Внутри класса: видно в этом классе и во всех его подклассах, даже если подкласс в другом пакете или модуле.
- Не видно из других классов, даже в том же пакете.

**private**

- Top-level (вне класса): видно только внутри одного .kt файла.
- Внутри класса: видно только внутри этого класса, подклассы не видят.
- Самый строгий уровень.

В обычных generic-функциях тип стирается на этапе выполнения (type erasure) — в JVM в рантайме нет информации, какой именно T был передан.

```
fun <T> createList(): List<T> = ArrayList<T>()  // в рантайме T = Unknown
```

**reified** + inline позволяет сохранить реальный тип в рантайме, но только внутри inline-функций.

```
inline fun <reified T> Any.isInstanceOf(): Boolean = this is T
```

**reified** — это суперсила Kotlin, которая позволяет писать type-safe, чистый и удобный API для работы с дженериками в рантайме.
Без inline fun <reified T> таких вещей, как filterIsInstance(), Gson.fromJson<T>(), Room @Query, Moshi.adapter<T>() — просто бы не существовало в удобной форме.
Использовать этот модификатор нужно именно для рефлексии и всего, что с ней связано. В рантайме (в JVM) обычные дженерики стираются, поэтому без reified ты не знаешь, какой именно T был передан.

- Проверка типа (is, as)
- Получение Class<T> или KClass<T>
- Работа с аннотациями на типе
- Сериализация/десериализация (Gson, Moshi, Jackson-Kotlin, Room)
- Room @Query с generic DAO:
- Стандартные функции (filterIsInstance()<T>, intent.getSerializableExtra<T>())

**Bytecode** — это файл/последовательность инструкций (в формате .class), которые получаются после компиляции Java/Kotlin-кода. Это статический набор команд для виртуальной машины Java.
**JVM (runtime)** — это программа-исполнитель, которая в момент запуска читает и выполняет этот bytecode. Это динамическая среда выполнения: загрузчик классов, куча, сборщик мусора, JIT-компилятор, стеки потоков и т.д.

**Generics** в Kotlin (и Java) нужны для трёх главных вещей:

1. Type-safety на этапе компиляции

```
val list: List<String> = listOf("hello", "world")
val s: String = list[1]  // OK, компилятор знает, что там String
// list.add(123) — ошибка компиляции!
```

2. Избавление от лишних приведений типов (casts)

```
// Java-style (до generics)
String s = (String) list.get(0);
// Kotlin с generics — каст не нужен
val s: String = list[0]  // автоматически
```

3. Повторное использование кода с разными типами. Один и тот же класс/функция работает с любым типом:

```
fun <T> identity(value: T): T = value
identity("hello")   // T = String
identity(42)        // T = Int
identity<User>(user)
```

Инструкция (**statement**) — делает что-то, но не возвращает значение
Выражение (**expression**) — всегда возвращает значение и имеет тип
В Kotlin почти всё — выражение:

- if — выражение (обязательно должен быть else, если используется как выражение)
- when — выражение
- try-catch — выражение
- Даже elvis ?: throw — выражение

В Java if, for, while — только инструкции → нельзя присвоить результат.

1. **Int** — примитивный тип в Kotlin (на уровне JVM — int). Не может быть null, не является объектом, не наследуется от Any?.
2. **Int?** — nullable примитив, в JVM при использовании — автоматически бокситься в java.lang.Integer.
3. **kotlin.Int** — это алиас (то же самое, что просто Int). Пишется полностью только в редких случаях (например, при рефлексии).
4. **java.lang.Integer** — обёртка из Java. Всегда объект, может быть null, наследуется от Any?.

Главный вопрос: почему Int НЕ наследуется от Any??
Потому что Int — примитив (int), а примитивы в JVM не участвуют в иерархии объектов. Any в Kotlin соответствует java.lang.Object → только ссылочные типы наследуются от него.

**vararg** — синтаксический сахар для передачи нефиксированного количества аргументов одного типа в функцию. Под капотом превращается в Array<T>.
**List<T>** — интерфейс коллекции (immutable или mutable). Нельзя использовать как параметр функции вместо vararg.
**Array<T>** — примитивный массив (как int[] в Java). Это не List, у него другая производительность и поведение.

Почему for (i in list) работает?
Потому что у List<T> есть расширение-оператор **iterator**():

```
operator fun <T> Iterable<T>.iterator(): Iterator<T>
```

Под капотом:

- 0..9 → IntRange → наследует IntProgression
- Все эти .., until, downTo, step — возвращают объекты типа IntProgression

**Диапазоны** в Kotlin — это не просто синтаксический сахар, а настоящие объекты.
Оператор **..** создаёт включительный диапазон: от начального значения до конечного, включая оба края.
Функция until создаёт исключающий верхнюю границу диапазон: конечное значение в цикл не попадает.
downTo создаёт диапазон, идущий вниз, и конечная граница включается.
step позволяет задать шаг больше 1, работает и в прямом, и в обратном направлении.
until спасает от классической off-by-one ошибки: когда нужно пройти по индексам списка или массива, правильнее писать от нуля и до size с until, а не с .., потому что последний индекс всегда size - 1.
Диапазоны работают не только с числами: можно перебирать символы от 'a' до 'z' — это тоже полноценный диапазон.
Цикл for в Kotlin на самом деле работает не только с диапазонами, а с любым объектом, у которого определён оператор iterator. У всех стандартных коллекций (List, Set, Map.keys, Map.values и т.д.) этот оператор есть по умолчанию, поэтому по любой коллекции можно писать for (item in collection) — это обычный foreach.
Под капотом диапазоны .., until, downTo, step реализуются через класс IntProgression (или LongProgression, CharProgression). Это лёгкие объекты с тремя полями: start, end и step. Они не создают промежуточных списков — перебор идёт эффективно, почти как обычный цикл while.

Все виды **object** в Kotlin:

1. **Object declaration** (именованный объект, **синглтон**) Это настоящий синглтон — в приложении существует ровно один экземпляр этого объекта. Создаётся лениво при первом обращении. Используется вместо статических полей/методов из Java, для логгеров, конфигов, кэшей и т.д. Пример из жизни: Json, Clock.System, Dispatchers.Main.
2. **Companion object** Это тот же синглтон, но живёт внутри класса и привязан к нему. Используется как место для фабричных методов, констант и всего, что в Java было бы static. У каждого класса может быть максимум один companion object (с именем или без). В стандартной библиотеке — у всех коллекций: List, Map, Set и т.д.
3. **Object expression** (анонимный объект) Это не синглтон! Каждый раз создаётся новый экземпляр. Это прямой аналог анонимного класса в Java. Используется там, где нужно быстро реализовать интерфейс или унаследовать класс в одном месте: клик-листенеры, коллбэки, обработчики событий и т.д.
4. Вложенные **object внутри object** Можно внутри одного синглтона объявлять другие объекты. Используется для красивой группировки констант и подмодулей: например, Retrofit, Endpoints, Headers, Permissions и т.д.

**Primary constructor** — это часть заголовка класса: class User(val name: String). Он выполняется до любого init-блока.
**Secondary constructor** — это обычный constructor(...) внутри тела класса, вызывается через this(...) и обязателен только если нет primary или нужно несколько вариантов создания.

Да, **можно расширять любой класс** (даже final и даже из Java) через extension functions/properties. Это одна из главных фишек Kotlin.

**by lazy** — ленивая инициализация при первом обращении, значение кэшируется, работает с val.
**lateinit var** — инициализация вручную позже, только для var и только для классов (не примитивов), иначе краш.

**@JvmStatic** в companion object делает метод/свойство настоящим static в байткоде. Без него из Java вызываешь MyClass.Companion.method(), с ним — MyClass.method().

**Delegates.observable**(initialValue) { prop, old, new -> … } — создаёт свойство, которое вызывает лямбду при каждом изменении значения.

by lazy по умолчанию использует **LazyThreadSafetyMode.SYNCHRONIZED** — потокобезопасно с блокировкой.
by lazy(**LazyThreadSafetyMode.NONE**) { … } — без синхронизации, быстрее, но только если уверен, что доступ из одного потока.

**typealias** — псевдоним типа.
Очень полезен для длинных функциональных типов:
typealias ClickListener = (View) -> Unit

**value class** (с Kotlin 1.5+, раньше inline class) — обёртка над одним значением (примитивом), которая не создаёт объект в рантайме при использовании.
Пример: @JvmInline value class UserId(val value: Long) — в JVM будет просто long.

**expect / actual** — механизм multiplatform.
expect class Platform в commonMain, actual class Platform в androidMain/iosMain/jvmMain и т.д.
expect = «я обещаю, что где-то будет реализация»
actual = «вот реализация именно для этой платформы»

**let** — возвращает результат лямбды, it-референс, часто для цепочек и null-check.
**apply** — возвращает сам объект, this-референс, для настройки объекта.
**run** — возвращает результат лямбды, this-референс, для выполнения блока и получения результата.

**TODO()** — функция, всегда бросает NotImplementedError, тип Nothing.

**tailrec** — оптимизирует tail-recursive функцию в цикл, предотвращая StackOverflow.

**by** — делегирование свойства в Kotlin.
Свойство не хранит значение само, а передаёт геттер/сеттер другому объекту (делегату), реализующему интерфейс ReadOnlyProperty или ReadWriteProperty.
Как работает:

1. Компилятор генерирует скрытое поле для хранения делегата.
2. При обращении к свойству вызывает getValue() / setValue() у делегата.
3. Делегат может вычислять значение, кэшировать, логировать и т.д.

**Invariance**: Default in generics. List<String> не подтип List<Any>.
Обеспечивает максимальную безопасность, но ограничивает гибкость.
List<String> нельзя присвоить переменной List<Any> и наоборот.
Коллекции, которые можно изменять (например, MutableList<T>), по умолчанию инвариантны.
**Covariance** (out T): Producer-вариант. List<out String> подтип List<out Any>.
Позволяет безопасно использовать более "специфичные" типы в контексте "более общих". Идеально для только для чтения (read-only) коллекций или интерфейсов, которые возвращают параметр T.
List<String> это List<Any> (безопасно, так как String - это Any, но нельзя добавить в List<Any> что-то кроме Any).
**Contravariance** (in T): Consumer-вариант. Comparator<in Any> подтип Comparator<in String>.
Позволяет использовать более "общие" типы там, где ожидаются "специфичные", если интерфейс только принимает параметр T (например, функция, которая принимает T).
Comparator<Any> может сравнивать Int и Double, потому что Any "шире", чем Int или Double

**List<*>** — unknown generic (star-projection): тип неизвестен, можно читать как Any?, нельзя писать ничего.
**List<Any?>** — конкретный тип: элементы Any? (nullable Any), можно писать null или любой объект, читать как Any?.
List<*> безопаснее для чтения из неизвестных списков, без кастов.

**inner class** — вложенный класс, который имеет ссылку на внешний класс (non-static в Java).

- Доступен this@Outer из inner.
- Можно обращаться к свойствам/методам внешнего класса напрямую.
- Нельзя создать inner без экземпляра outer.

Без inner — просто nested class (static, без ссылки на outer).

**Композиция** — "has-a" отношение: класс содержит экземпляр другого класса как свойство.

```
class Engine(val power: Int)
class Car {
    private val engine = Engine(200)  // композиция: Car имеет Engine
    fun start() = println("Engine $engine.power started")
}
```

Car использует Engine, а не наследуется от него.
Основной принцип OOP: предпочитай композицию наследованию (гибче, меньше связанности).

# ОП, ООП

**Value type vs reference type**:
Value type хранит значение напрямую в стеке (примитивы как int, boolean).
Reference type хранит ссылку на объект в куче.
Копирование value type копирует значение, reference — только ссылку.

**Функция vs метод**:
Функция — самостоятельный блок кода, не привязан к классу.
Метод — функция внутри класса, может работать с его полями и вызываться на объекте.
В Kotlin всё обычно методы, но top-level функции — как функции.

**Рекурсия**:
Функция вызывает сама себя для решения задачи на подзадачах.
Полезна для деревьев, факториалов, но рискует переполнить стек.
Требует базового случая для остановки.

**ООП**:
Парадигма, где программа строится из объектов, сочетающих данные и поведение.
Ключевые принципы: инкапсуляция, наследование, полиморфизм, абстракция.
Позволяет моделировать реальный мир.

**Инкапсуляция**:
Инкапсуляция в объектно-ориентированном программировании представляет собой принцип сокрытия внутренних деталей реализации класса от внешнего мира, обеспечивая доступ к данным только через контролируемые методы (геттеры и сеттеры). Это повышает безопасность и упрощает поддержку кода. В Android-разработке примером служит ViewModel, где поля состояния объявляются private, а доступ к ним предоставляется через public-методы или observable свойства (LiveData/StateFlow), предотвращая прямую модификацию данных из Activity или Fragment и обеспечивая контролируемые обновления UI.

**Наследование**:
Наследование в ООП — это механизм, позволяющий одному классу (подклассу) получать свойства и методы другого класса (суперкласса), способствуя переиспользованию кода и созданию иерархий. В Kotlin классы и методы по умолчанию final, поэтому для наследования требуется модификатор open. Это позволяет расширять функциональность базового класса без дублирования кода.

**Полиморфизм**:
Полиморфизм в ООП подразумевает способность объектов разных классов обрабатываться через общий интерфейс или суперкласс, проявляясь в overriding (динамический полиморфизм) или overloading. Пример в Android: разные реализации Repository (RoomRepository, NetworkRepository) могут реализовывать один интерфейс Repository, позволяя ViewModel работать с любой реализацией без изменений, что упрощает переключение источников данных. Включает runtime (overriding) и compile-time (overloading).

**Overloading**:
Несколько методов с одним именем, но разными параметрами.
Разрешается на этапе компиляции.
Удобно для разных способов вызова.

**Overriding**:
Переопределение метода родителя в потомке.
Работает в runtime (dynamic dispatch).
Требует open в родителе.

**Абстракция**:
Абстракция в ООП — принцип скрытия сложных деталей реализации и предоставления только необходимого интерфейса для взаимодействия. Это достигается через абстрактные классы или интерфейсы, фокусируя внимание на "что" делает объект, а не "как". Абстракция упрощает проектирование систем и повышает читаемость кода. Разница между абстрактным и обычным классом заключается в том, что абстрактный класс не может быть инстанциирован напрямую и может содержать абстрактные методы (без реализации), которые должны быть переопределены в подклассах, в то время как обычный класс полностью реализован и готов к созданию объектов.

**Абстрактный класс vs интерфейс**:
Абстрактный класс — частичная реализация + состояние.
Интерфейс — контракт, с Kotlin 1.2+ реализация по умолчанию, без состояния (кроме private).
Класс наследует один абстрактный, реализует много интерфейсов.

**Интерфейс**
Интерфейс в ООП — это контракт, определяющий набор методов, которые класс должен реализовать, без хранения состояния (кроме private полей в Kotlin). Разница с абстрактным классом: интерфейс поддерживает множественную реализацию, может иметь default-реализации, но не наследует состояние; абстрактный класс — единственное наследование с возможным состоянием и частичной реализацией.
В Android интерфейсы предпочитают для callbacks, поскольку они позволяют множественную реализацию, упрощают тестирование (легко мокать) и избегают проблем наследования, обеспечивая слабую связанность между компонентами (например, OnClickListener).

**Композиция** предпочтительнее наследования в Android, поскольку она обеспечивает большую гибкость, снижает связанность и избегает проблем "хрупкой базовой иерархии". Пример: ViewModel содержит экземпляр Repository как поле (has-a отношение), позволяя легко заменить реализацию (например, на mock для тестов) без изменения иерархии классов.

**ООП** применяется в Android компонентах через наследование (Activity наследует AppCompatActivity), инкапсуляцию (private поля в Fragment), полиморфизм (разные View в RecyclerView) и абстракцию (интерфейсы для callbacks), что позволяет создавать расширяемые и тестируемые компоненты с чётким разделением ответственности.

**Проблемы множественного наследования** в Android избегают использованием интерфейсов вместо классов для контрактов, а также композицией и делегированием, что предотвращает конфликты методов и упрощает архитектуру.

**SOLID**:
Принципы для чистого, поддерживаемого кода.

**Single Responsibility Principle**:
Класс имеет одну причину для изменения (одна ответственность).
Упрощает тестирование и поддержку.
**Пример SRP в Android**:
ViewModel только управляет данными, Repository — только доступ к БД.

**Open/Closed Principle**:
Открыт для расширения, закрыт для модификации.
**Пример OCP в Android**:
Расширяем RecyclerView.Adapter новыми ViewHolder без изменения базового.

**Liskov Substitution Principle**:
Подтипы заменяемы на супер-типы без нарушения поведения.
**Пример LSP в Android**:
CustomView наследует View и работает везде, где ожидается View.

**Interface Segregation Principle**:
Много маленьких интерфейсов лучше одного большого.
**Пример ISP в Android**:
Отдельные интерфейсы для click/listener вместо одного огромного.

**Dependency Inversion Principle**:
Зависеть от абстракций, не от конкретных реализаций.
**Пример DIP в Android**:
ViewModel зависит от Repository интерфейса, а не конкретного RoomRepo; инжект через Hilt.

**Factory pattern**:
Создаёт объекты без указания конкретного класса.
**Пример Factory в Android**:
ViewModelProvider.Factory для создания ViewModel с параметрами.

**Builder pattern**:
Пошаговое создание сложного объекта.
**Пример Builder в Android**:
AlertDialog.Builder для настройки диалога.

**Observer pattern**:
Объекты уведомляют подписчиков об изменениях.
**Пример Observer в Android**:
LiveData/StateFlow — View наблюдает за данными в ViewModel.
**Adapter pattern**:
Преобразует интерфейс одного класса в другой.
**Пример Adapter в Android**:
RecyclerView.Adapter адаптирует данные для ViewHolder.

**Facade pattern**:
Упрощённый интерфейс к сложной подсистеме.
**Пример Facade в Android**:
MediaPlayer или Retrofit скрывают сложность.

**Proxy pattern**:
Контролирует доступ к объекту (lazy, cache).
**Пример Proxy в Android**:
Glide/Picasso для ленивой загрузки изображений.

**Command pattern**:
Инкапсулирует запрос как объект.
**Пример Command в Android**:
Undo/Redo операции в редакторах
.
**Strategy pattern**:
Сменяемая стратегия поведения.
**Пример Strategy в Android**:
Разные сортировки в RecyclerView DiffUtil.

**State pattern**:
Поведение меняется в зависимости от состояния.
**Пример State in Android UI**:
Loading/Error/Success экраны в ViewModel.

**MVC**:
Model-View-Controller: Model данные, View UI, Controller логика.
**Как MVC применяется в Android**:
Activity/Fragment как Controller+View, Model отдельно.

**MVP**:
Model-View-Presenter: Presenter посредник.
**Как MVP применяется в Android**:
Presenter держит логику, View — интерфейс с методами.

**MVVM**:
Model-View-ViewModel: ViewModel экспонирует данные.
**Как MVVM применяется в Android**:
ViewModel + LiveData/StateFlow + DataBinding.

**Разница MVC/MVP/MVVM**:
Эти три архитектуры предназначены для разделения ответственности в приложении, чтобы код был более тестируемым, поддерживаемым и масштабируемым. В Android они применяются для организации взаимодействия между UI, логикой и данными. Ниже — детальное описание каждой и сравнение.

MVC (Model-View-Controller)

- **Model**: отвечает за данные и бизнес-логику (репозитории, базы данных, сетевые запросы).
- **View**: отвечает за отображение (Activity/Fragment, XML-лейауты).
- **Controller**: посредник — обрабатывает пользовательский ввод, обновляет Model и View.
- **Как работает в Android**: Activity/Fragment часто выступает одновременно Controller и View (Controller получает события, обновляет Model, вручную обновляет UI).
- **Плюсы**: простота, подходит для маленьких приложений.
- **Минусы**: Activity становится "жирной" (God object), сложно тестировать, сильная связанность View и Controller.

MVP (Model-View-Presenter)

- **Model**: те же данные и бизнес-логика.
- **View**: пассивный интерфейс — только отображает данные и передаёт события (интерфейс с методами showProgress(), showData()).
- **Presenter**: вся логика приложения — получает события от View, работает с Model, обновляет View через интерфейс.
- **Как работает в Android**: Activity/Fragment реализует View-интерфейс, Presenter держит ссылку на View. Нет прямой зависимости от Android-фреймворка в Presenter.
- **Плюсы**: Presenter легко unit-тестировать (без Android), чёткое разделение, View пассивен.
- **Минусы**: много boilerplate-кода (интерфейсы, ручное обновление View), Presenter может стать большим, проблемы с lifecycle (нужно отписываться вручную).

MVVM (Model-View-ViewModel)

- **Model**: данные и репозитории.
- **View**: UI (Activity/Fragment/Compose), наблюдает за данными.
- **ViewModel**: хранит и управляет UI-состоянием, экспонирует observable данные (LiveData/StateFlow), работает с Model.
- **Как работает в Android**: ViewModel survives configuration changes, View наблюдает за LiveData/StateFlow — обновления автоматические. Data Binding или Compose упрощают привязку.
- **Плюсы**: автоматические обновления UI, lifecycle-aware (LiveData/Flow), ViewModel легко тестировать, минимум boilerplate.
- **Минусы**: сложнее отлаживать реактивные потоки, возможен over-use observable.

**Strong/weak/soft/phantom reference**
В Java (и соответственно в Kotlin) существуют четыре основных типа ссылок на объекты в куче: strong, weak, soft и phantom. Они определяют, как Garbage Collector (GC) взаимодействует с объектом и когда он может его собрать.
По умолчанию все обычные ссылки — strong.
Специальные типы ссылок находятся в пакете java.lang.ref и используются для тонкого контроля над памятью (кэши, слушатели, cleanup).

**Strong Reference (обычная сильная ссылка)**
Это стандартная ссылка, которую мы используем повседневно: Object obj = new Object();.
Пока существует хотя бы одна strong-ссылка на объект, GC **никогда** не соберёт его, даже если памяти мало.
Объект становится доступным для сборки только после того, как все strong-ссылки на него исчезнут (становятся null или переприсваиваются).
Это основной механизм, обеспечивающий нормальную работу программы.
Проблема: может привести к memory leak, если забыть очистить ссылку (например, в статических коллекциях).

**Weak Reference (слабая ссылка)**
Создаётся через WeakReference<T> ref = new WeakReference<>(object);.
GC **может собрать** объект сразу же, как только на него не останется strong-ссылок, независимо от объёма памяти.
Получить объект: T obj = ref.get(); — может вернуть null, если уже собран.
Используется для кэшей, где данные можно пересоздать (например, WeakHashMap), и для избежания утечек (слушатели событий).
В Android: часто для Context в нестатических внутренних классах или в библиотеках вроде LeakCanary.

**Soft Reference (мягкая ссылка)**
Создаётся через SoftReference<T> ref = new SoftReference<>(object);.
GC собирает объект **только при нехватке памяти** (перед OutOfMemoryError).
Идеально для кэшей, где данные дорого пересоздавать (изображения, большие вычисления).
Объект живёт дольше weak, но всё равно может быть собран.
В Android: раньше использовали для bitmap-кэша, сейчас чаще LruCache.

**Phantom Reference (фантомная ссылка)**

Создаётся через PhantomReference<T> ref = new PhantomReference<>(object, referenceQueue);.
Это самая слабая ссылка: ref.get() всегда возвращает null — нельзя получить сам объект.
Объект считается достижимым, пока не очищен, но служит только для уведомления о сборке.
После сборки объекта ссылка помещается в ReferenceQueue, откуда можно получить уведомление.
Используется для выполнения cleanup-действий после финализации (например, освобождение нативных ресурсов).

**Как lifecycle влияет на ресурсы**

Очистка ресурсов в Android обычно происходит в методах lifecycle, таких как onDestroy() в Activity или Fragment, где закрываются соединения (например, базы данных, подписки на LiveData, сетевые клиенты) или освобождаются тяжёлые объекты (Bitmap, Cursor). Это предотвращает memory leaks и обеспечивает эффективное использование памяти при пересоздании компонентов.

**REST**

REST — это архитектурный стиль для проектирования сетевых приложений, прежде всего веб-API. Он был предложен Роем Филдингом в 2000 году в его диссертации и стал де-факто стандартом для современных веб-сервисов.
Основные принципы REST (6 ограничений):
*Client-Server*
Клиент и сервер разделены: клиент отвечает за UI, сервер — за логику и данные. Это позволяет независимо развивать обе части.
*Stateless*
Каждый запрос от клиента должен содержать всю необходимую информацию для его обработки. Сервер не хранит состояние сессии между запросами. Это упрощает масштабирование и повышает надёжность.
*Cacheable*
Ответы сервера должны явно указывать, можно ли их кэшировать (через заголовки Cache-Control, ETag и т.д.). Это снижает нагрузку на сервер и ускоряет работу клиента.
*Uniform Interface (единообразный интерфейс)*
Ключевой принцип, включающий:
Идентификация ресурсов через URI (например, /users/123).
Манипуляция ресурсами через стандартные HTTP-методы.
Самоописательные сообщения (заголовки + тело).

*Layered System*
Архитектура может состоять из слоёв (прокси, балансировщики, кэши), клиент не знает, с каким именно слоем общается.
*Code on Demand (опционально)*
Сервер может отправлять исполняемый код клиенту (например, JavaScript), расширяя функциональность клиента.

**Как работает REST на практике**
Ресурсы: всё моделируется как ресурсы (users, orders, products).
HTTP-методы:
GET — чтение (безопасный, идемпотентный).
POST — создание.
PUT/PATCH — обновление (PUT идемпотентный).
DELETE — удаление.
Статусы HTTP: 200 OK, 201 Created, 404 Not Found, 500 Internal Server Error и т.д.
Форматы данных: чаще JSON, иногда XML.
Преимущества REST
- Простота и понятность.
- Масштабируемость (stateless).
- Кэшируемость.
- Совместимость с веб-инфраструктурой (браузеры, прокси).

Недостатки
- Не всегда подходит для сложных операций (нужны несколько запросов).
- Over-fetching/under-fetching (получаешь лишние или недостающие данные).
- Альтернативы: GraphQL, gRPC.

В Android-разработке REST — основной способ общения с бэкендом (Retrofit, OkHttp).
Большинство публичных API (Google, GitHub, Twitter) — RESTful.

**Integration test**
Тест взаимодействия компонентов (API + DB + UI), проверяет интеграцию, использует mocks или реальные сервисы.

**TDD, BDD**
TDD: тесты перед кодом, цикл red-green-refactor.
BDD: фокус на поведении, сценарии Given-When-Then, инструменты Cucumber.

**Pure function**
Функция без side effects, одинаковый ввод — одинаковый вывод, не меняет состояние.

**Side effect**
Изменение внешнего состояния: мутация, IO, логи.

**ООП помогает в тестировании** Android-приложений, предоставляя чёткие границы (инкапсуляция), интерфейсы для моков (например, мок Repository в ViewModel-тестах) и полиморфизм для инъекции зависимостей.

# Алгоритмы и Структуры Данных

## Что такое временная и пространственная сложность алгоритма?

Временная сложность алгоритма описывает, как количество операций растёт с увеличением размера входных данных, обычно выражается в терминах количества элементарных шагов. Пространственная сложность оценивает объём дополнительной памяти, необходимой алгоритму (не включая входные данные), включая стек рекурсии или вспомогательные структуры. Эти метрики помогают прогнозировать производительность и масштабируемость кода, особенно в мобильных приложениях, где ресурсы ограничены.

## Как обозначается Big O, Omega и Theta нотация?

Big O (O(f(n))) обозначает верхнюю границу роста функции — худший случай или асимптотическую верхнюю оценку. Omega (Ω(f(n))) — нижнюю границу, лучший случай или асимптотическую нижнюю оценку. Theta (Θ(f(n))) — точную границу, когда верхняя и нижняя оценки совпадают, то есть алгоритм ведёт себя именно как f(n) в асимптотике.

## Примеры алгоритмов с сложностью O(1), O(log n), O(n), O(n log n), O($n^2$)?

O(1): доступ по индексу в массиве или HashMap.get (constant time). O(log n): бинарный поиск в отсортированном массиве. O(n): линейный поиск или проход по списку. O(n log n): эффективные сортировки вроде QuickSort или MergeSort в среднем случае. O($n^2$): вложенные циклы, как Bubble Sort или наивный поиск подстроки.

## Что такое массив и его преимущества/недостатки? Как работает доступ по индексу в массиве.

Массив — непрерывный блок памяти фиксированного размера для хранения элементов одного типа. Преимущества: быстрый доступ по индексу O(1), эффективное использование кэша. Недостатки: фиксированный размер, дорогая вставка/удаление (сдвиг элементов). Доступ по индексу работает через вычисление адреса: base_address + index * element_size, что позволяет мгновенный переход.

### Что такое динамический массив (ArrayList в Java/Kotlin)? Как ArrayList расширяется при добавлении элементов.

ArrayList — динамический массив, реализующий интерфейс List, с автоматическим изменением размера. Под капотом использует обычный массив, который при переполнении (load factor) копируется в новый больший (обычно в 1.5 раза). Это обеспечивает амортизированную O(1) вставку в конец, но иногда O(n) при resiz'e.

### Пример использования SparseArray/SparseIntArray в Android и почему они эффективнее HashMap для разреженных данных?

SparseArray/SparseIntArray — специализированные структуры Android для хранения пар key-value, где key — int. Пример: хранение View по ID в Activity. Они эффективнее HashMap для разреженных данных (много пропусков в ключах), потому что используют два массива (keys и values) с бинарным поиском вместо хэширования и боксинга Integer, экономя память и время.

### Основные операции со строками и их сложность. Как работает String Pool в Java?

Основные операции: конкатенация O(n), substring O(1) в Java 7+, length O(1), charAt O(1). String immutable, поэтому конкатенация создаёт новые объекты. String Pool — область heap для хранения строковых литералов; одинаковые литералы ссылаются на один объект, экономя память (intern() добавляет в пул).

### Что такое односвязный и двусвязный список? Преимущества LinkedList над ArrayList.

Односвязный список — каждый узел содержит данные и ссылку на следующий. Двусвязный — плюс ссылка на предыдущий. Преимущества LinkedList: O(1) вставка/удаление в известной позиции (без сдвига), динамический размер. Недостатки: O(n) доступ по индексу, больше памяти на ссылки.

### Когда в Android лучше использовать LinkedList? Основные операции (add, remove, get) и их сложность в связанном списке.

LinkedList лучше в Android для частых вставок/удалений в середине (например, очередь задач). Операции: add в конец O(1), remove известного узла O(1), get по индексу O(n) (проход по ссылкам).

**Что такое стек (Stack) и очередь (Queue)? Примеры использования очереди в Android (Handler, MessageQueue). Что такое Deque и PriorityQueue?**

Стек — LIFO структура (push/pop). Очередь — FIFO (enqueue/dequeue). В Android очередь используется в Handler/MessageQueue для обработки сообщений в главном потоке. Deque — двусторонняя очередь (добавление/удаление с обоих концов). PriorityQueue — очередь с приоритетом (heap-based).

## Что такое хэш-таблица (HashMap/HashSet)? Как работает хэширование и разрешение коллизий (chaining, open addressing)? Сложность операций в HashMap (average и worst case).

HashMap — структура key-value на основе хэша ключа для быстрого доступа. Хэширование: hashCode() → индекс в массиве. Коллизии: chaining (списки в бакетах) или open addressing (пробинг). Сложность average O(1), worst O(n) при всех коллизиях.

## Что такое load factor и rehashing?

Load factor — отношение заполненных бакетов к размеру массива (обычно 0.75). Rehashing — при превышении load factor массив увеличивается, все элементы перехэшируются в новый массив для сохранения производительности.

## Разница HashMap и LinkedHashMap?

LinkedHashMap сохраняет порядок вставки (или доступа) через дополнительный двусвязный список, в отличие от обычного HashMap с произвольным порядком.

## Когда использовать HashMap в Android, а когда SparseArray?

HashMap для любых ключей (объекты), SparseArray для int-ключей — экономит память (без боксинга Integer) и быстрее для разреженных данных.

## Что такое бинарное дерево? Пример дерева в Android (например, View hierarchy).

Бинарное дерево — структура, где каждый узел имеет до двух детей. В Android View hierarchy — дерево ViewGroup с дочерними View, используемое для отрисовки и событий.

### Что такое граф? Ориентированный и неориентированный? Способы представления графа (матрица смежности, список смежности).

Граф — набор вершин и рёбер. Неориентированный — рёбра без направления, ориентированный — с направлением. Представление: матрица смежности (2D массив) для плотных графов, список смежности (массив списков) для разреженных — экономит память.

### Что такое DFS и BFS? Разница DFS и BFS по памяти и применению. Когда использовать BFS в Android (например, поиск кратчайшего пути в навигации)?

DFS (Depth-First Search) — углубление по ветви, использует стек. BFS (Breadth-First Search) — по уровням, использует очередь. DFS экономит память (O(h)), BFS — O(w). BFS для кратчайшего пути в невзвешенном графе, в Android — навигация или поиск соседних элементов.

### Что такое цикл в графе и как его обнаружить?

Цикл — путь, возвращающийся в начальную вершину. Обнаружение: DFS с visited и parent, или Union-Find для неориентированных.

### Сортировки:

Bubble Sort — многократный проход по массиву с обменом соседних элементов, если они в неправильном порядке, сложность $O(n^2)$.
Insertion Sort — вставка элементов в отсортированную часть массива по одному, эффективен для маленьких данных.
Selection Sort — поиск минимума в неотсортированной части и обмен с началом, $O(n^2)$.
Merge Sort — divide-and-conquer: делит массив пополам, сортирует рекурсивно и сливает, стабильная O(n log n).
Quick Sort — divide-and-conquer: выбор pivot, разделение массива и рекурсия, средняя O(n log n), быстрее Merge на практике из-за кэш-локальности.
Стабильные сортировки сохраняют порядок равных элементов (MergeSort), нестабильные — нет (QuickSort).
Collections.sort в Android использует TimSort (гибрид Merge + Insertion), стабильный O(n log n).

**Кэш-локальность** (cache locality) в контексте QuickSort относится к эффективному использованию процессорного кэша благодаря особенностям доступа к памяти во время выполнения алгоритма.

QuickSort работает «in-place» — сортирует массив без создания дополнительных больших структур данных, выполняя обмены элементов непосредственно в исходном массиве. Когда алгоритм выбирает pivot и разделяет массив на части (partitioning), он последовательно проходит по соседним элементам, сравнивая и меняя их местами. Такой последовательный доступ к памяти (sequential memory access) хорошо соответствует организации кэша процессора: при загрузке одного элемента в кэш-линию автоматически подгружаются соседние элементы, что минимизирует дорогостоящие обращения к оперативной памяти.

В результате QuickSort демонстрирует высокую кэш-локальность, особенно по сравнению с MergeSort, который требует дополнительного массива для слияния и выполняет множество разрозненных обращений к памяти при копировании данных. Это одна из главных причин, почему QuickSort на практике часто оказывается быстрее MergeSort при одинаковой асимптотической сложности O(n log n) — лучшее использование кэша снижает реальное время выполнения на современных процессорах.

## Поиски:

Линейный поиск — последовательный перебор элементов, сложность O(n).
Бинарный поиск — деление отсортированного массива пополам, требует отсортированных данных, сложность O(log n).
Пример бинарного поиска в Android — поиск в отсортированном списке или Arrays.binarySearch.

## Когда рекурсия предпочтительнее итерации?

Рекурсия предпочтительнее для задач с естественной рекуррентной структурой (деревья, графы, divide-and-conquer), когда код проще и читабельнее, но итерация лучше для производительности и избежания stack overflow в глубоких случаях.

## Что такое динамическое программирование?

Динамическое программирование — метод оптимизации, разбивающий задачу на подзадачи, сохраняющий результаты для повторного использования, чтобы избежать экспоненциальной сложности.

### Разница memoization и tabulation.

Memoization — top-down рекурсия с кэшем результатов подзадач. Tabulation — bottom-up итеративное заполнение таблицы от простых к сложным подзадачам.

### Пример DP: вычисление Фибоначчи. Пример DP в Android (например, оптимизация RecyclerView DiffUtil).

Фибоначчи DP: храним уже вычисленные значения в массиве или map, снижая сложность с экспоненциальной до O(n). В Android DiffUtil использует DP (алгоритм Майерса) для эффективного вычисления различий между списками в RecyclerView.

**Алгоритм Майерса** (Myers' algorithm) — это эффективный алгоритм вычисления минимальной разницы (difference) между двумя последовательностями, разработанный Юджином Майерсом в 1986 году. Он используется для нахождения кратчайшей последовательности редактирования (shortest edit script), то есть минимального набора операций вставки, удаления и замены, необходимых для преобразования одной последовательности в другую.

Основная идея алгоритма заключается в поиске самого длинного общего подпоследовательности (LCS — Longest Common Subsequence) с помощью динамического программирования, но с оптимизацией по памяти и времени: сложность O(ND), где N — сумма длин последовательностей, D — количество различий (в худшем случае $O(N^2)$, но на практике значительно лучше).

В Android-разработке алгоритм Майерса лежит в основе **DiffUtil** (из Jetpack), который эффективно вычисляет различия между старым и новым списками данных в RecyclerView. Это позволяет обновлять только изменённые элементы, а не перерисовывать весь список, что существенно повышает производительность и плавность анимаций при обновлении UI. Без такого алгоритма простое сравнение списков было бы слишком медленным для больших коллекций.

### Что такое LruCache и как работает? Пример использования LruCache для кэша изображений.

LruCache — кэш с фиксированным размером, удаляющий least recently used элементы при переполнении (LinkedHashMap с access order). В Android используется для кэша изображений (Bitmap), экономя память и ускоряя загрузку.

### Что такое BitSet и когда использовать?

BitSet — структура для хранения битов, эффективная для флагов или множеств больших индексов. Использовать для разреженных булевых массивов или фильтров.

### Что такое Bloom Filter (концепция)?

Bloom Filter — вероятностная структура для проверки принадлежности элемента множеству с возможными ложными срабатываниями, но без ложных отрицаний, экономит память за счёт хэшей.

### Как выбрать структуру данных для частых поисков по ID в Android?

Для частых поисков по int-ID — SparseArray (быстрее и меньше памяти). Для объектных ключей — HashMap. Для сортированных — TreeMap.

### Как алгоритмы влияют на ANR в Android?

Плохие алгоритмы ($O(n^2)$ на большом n) блокируют главный поток, вызывая ANR (Application Not Responding) при задержке >5 секунд.

### Почему $O(n^2)$ алгоритмы опасны в onDraw или RecyclerView?

В onDraw или bindViewHolder $O(n^2)$ на большом списке приводит к дропам фреймов, лагам UI или ANR, так как операции в главном потоке.

### Как оптимизировать поиск в большом списке в Android?

Использовать HashMap для O(1) по ключу, бинарный поиск для отсортированного списка или индексы в БД.

### Пример, где плохой выбор структуры данных приводит к OOM.

HashMap<Integer, Bitmap> для тысяч изображений — боксинг Integer + overhead → OOM; лучше SparseArray или LruCache.

### Как алгоритмы помогают в обработке больших JSON в Android?

Gson/Moshi с streaming (JsonReader) позволяют парсить большие JSON поэлементно O(n), избегая загрузки всего в память и OOM.

# Java Collections

### Основные интерфейсы в Collections: Collection, List, Set, Map, Queue.

Java Collections Framework включает ключевые интерфейсы для работы с группами объектов. Collection — корневой интерфейс для всех коллекций (кроме Map). List — упорядоченная коллекция с дубликатами и доступом по индексу. Set — коллекция уникальных элементов без дубликатов. Map — отображение ключ-значение, где ключи уникальны. Queue — коллекция для хранения элементов в порядке обработки (FIFO), с поддержкой приоритетов.

### Иерархия Collection: какие интерфейсы от чего наследуются?

Иерархия начинается с Iterable (для for-each). Collection наследует Iterable. List, Set и Queue наследуют Collection. SortedSet наследует Set, NavigableSet — SortedSet. Map отдельный, не наследует Collection. Deque наследует Queue. Это обеспечивает общий API для итерации и операций.

### Разница между Collection и Collections?

Collection — интерфейс для коллекций (List, Set и т.д.). Collections — утилитарный класс с статическими методами (sort, synchronizedList, unmodifiableList и т.д.). Collection определяет поведение, Collections предоставляет вспомогательные функции.

### Что такое Iterable и Iterator?

Iterable — интерфейс, позволяющий объекту быть целью for-each, с методом iterator(). Iterator — объект для последовательного прохода по коллекции (hasNext, next, remove). Это базовый механизм итерации в Collections.

### Что такое fail-fast и fail-safe итераторы? Пример fail-fast поведения в Android.

Fail-fast итератор бросает ConcurrentModificationException при структурной модификации коллекции во время итерации. Fail-safe работает с копией или позволяет модификации (CopyOnWriteArrayList). В Android fail-fast в ArrayList: изменение списка в цикле for-each вызывает исключение.

### Что такое List? Основные характеристики. Когда использовать ArrayList, а когда LinkedList?

List — упорядоченная коллекция с дубликатами и доступом по индексу. ArrayList — динамический массив, быстрый доступ O(1), медленная вставка в середину. LinkedList — двусвязный список, быстрая вставка/удаление O(1), медленный доступ O(n). ArrayList для чтения/поиска, LinkedList для частых вставок в середину.

## Что такое Vector и почему он устарел?

Vector — синхронизированная версия ArrayList из старых версий Java. Устарел, потому что синхронизация на каждый метод снижает производительность, лучше использовать Collections.synchronizedList или Concurrent коллекции.

## Что такое CopyOnWriteArrayList и когда использовать?

CopyOnWriteArrayList — потокобезопасный List, создающий копию массива при модификации. Итераторы не бросают исключение. Использовать для "читай много, пиши редко" (например, слушатели событий).

### Что такое Set? Основные характеристики. Разница между HashSet, LinkedHashSet и TreeSet? Как HashSet обеспечивает уникальность элементов?

Set — коллекция уникальных элементов без дубликатов. HashSet — неупорядоченный, быстрый O(1). LinkedHashSet — порядок вставки. TreeSet — сортированный по Comparable/Comparator. Уникальность в HashSet через hashCode и equals.

## Как работает hashCode и equals в HashSet?

HashSet использует hashCode для бакета, equals для проверки коллизий. Правильная реализация обеспечивает корректную уникальность.

## Почему LinkedHashSet сохраняет порядок вставки?

LinkedHashSet использует HashMap + двусвязный список для сохранения порядка вставки при итерации.

## Когда использовать TreeSet вместо HashSet?

TreeSet для сортированных данных (natural order или Comparator), когда нужен порядок или навигация (floor, ceiling).

## Что такое EnumSet и BitSet?

EnumSet — специализированный Set для enum, эффективный (битовый). BitSet — массив битов для флагов или больших индексов.

**Что такое Map? Основные характеристики. Разница между HashMap, LinkedHashMap и TreeMap?**
Map — это интерфейс, представляющий коллекцию пар «ключ—значение», где каждый ключ уникален и сопоставлен с одним значением. Основные характеристики: ключи не могут повторяться, значения могут; доступ, добавление и удаление по ключу. HashMap — неупорядоченная реализация, обеспечивает среднюю сложность O(1) для основных операций, но порядок элементов не гарантируется. LinkedHashMap сохраняет порядок вставки (или порядок доступа, если включён режим access-order), что полезно, когда нужно итерировать элементы в том порядке, в котором они были добавлены. TreeMap сортирует ключи по естественному порядку (Comparable) или заданному компаратору, сложность операций O(log n), но позволяет выполнять навигацию (floor, ceiling, subMap).

**Что изменилось в HashMap в Java 8 (деревья в бакетах)?**
В Java 8 в HashMap была добавлена оптимизация для случаев с большим количеством коллизий: если в одном бакете (ячейке массива) накапливается более 8 элементов, связанный список преобразуется в красно-чёрное дерево. Это снижает worst-case сложность поиска, вставки и удаления с O(n) до O(log n) при большом числе коллизий (например, при плохом hashCode). При уменьшении количества элементов дерево может обратно превратиться в список. Это улучшило производительность в экстремальных сценариях, не затрагивая средний случай O(1).

**Что такое IdentityHashMap?**
IdentityHashMap — это специальная реализация Map, которая использует для сравнения ключей оператор == (проверку на идентичность объектов), а не метод equals(). Это означает, что два разных объекта, даже если equals() возвращает true, будут считаться разными ключами. Используется в ситуациях, когда нужно различать объекты по ссылке, например, при работе с экземплярами классов, где equals() переопределён, но требуется именно уникальность по памяти.

**Что такое WeakHashMap и когда использовать?**
WeakHashMap — это Map, в котором ключи хранятся через WeakReference. Это означает, что если на ключ больше нет сильных ссылок, он (и соответствующая пара ключ-значение) может быть удалён сборщиком мусора в любой момент. WeakHashMap полезен для реализации кэшей, где ключ — это объект, который может больше не использоваться в программе (например, кэш метаданных объектов), и мы хотим, чтобы записи автоматически очищались при GC, не удерживая объекты в памяти.

**Что такое ConcurrentHashMap и его преимущества? Разница HashMap и ConcurrentHashMap?**
ConcurrentHashMap — потокобезопасная реализация Map, позволяющая множественным потокам одновременно читать и записывать данные без блокировки всего объекта. Преимущества: высокая конкурентность, хорошая масштабируемость при большом числе потоков, lock-free чтение. HashMap не потокобезопасен — при одновременной модификации может привести к повреждению структуры. ConcurrentHashMap работает быстрее в многопоточной среде, чем Collections.synchronizedMap(HashMap), который блокирует весь объект.

**Как ConcurrentHashMap обеспечивает потокобезопасность?**
ConcurrentHashMap делит внутренний массив на сегменты (в Java 8+ — до 16 сегментов, в новых версиях — динамически). При записи поток блокирует только нужный сегмент, а не всю карту. Чтение выполняется без блокировок благодаря volatile-полям и CAS-операциям (Compare-And-Swap). В Java 8+ при коллизиях используются деревья, как в обычном HashMap, но с дополнительной синхронизацией на уровне узлов.

**Что такое Queue и Deque? Разница между Queue и Deque?**
Queue — интерфейс для очереди FIFO (First In — First Out), предоставляет методы для добавления в конец (offer/add) и извлечения из начала (poll/remove). Deque (Double-ended queue) расширяет Queue и позволяет добавлять/удалять элементы с обоих концов (addFirst/addLast, pollFirst/pollLast). Таким образом, Deque — более универсальная структура, которая может использоваться и как очередь, и как стек.

**Что такое PriorityQueue и как работает?**
PriorityQueue — очередь с приоритетом, которая извлекает элементы в порядке их естественного сравнения (Comparable) или по заданному Comparator. Под капотом реализована на основе двоичной куче (binary heap), что обеспечивает O(log n) для вставки и извлечения, O(1) для получения минимального/максимального элемента. Не гарантирует стабильность порядка равных элементов.

**Что такое ArrayDeque и когда использовать?**
ArrayDeque — эффективная реализация Deque на основе циклического массива. Поддерживает O(1) операции с обоих концов, не имеет overhead’a на узлы, как LinkedList. Используется как очередь или стек, когда нужна максимальная производительность и нет необходимости в случайном доступе по индексу. Рекомендуется как предпочтительная реализация Deque.

**Разница PriorityQueue и PriorityBlockingQueue?**
PriorityQueue — обычная очередь с приоритетом, не потокобезопасна. PriorityBlockingQueue — потокобезопасная версия, реализует BlockingQueue, блокирует поток при попытке извлечь из пустой очереди или добавить в полную (с заданной ёмкостью). Подходит для сценариев producer-consumer в многопоточной среде.

**Что делает Collections.unmodifiableList/Set/Map? Что делает Collections.synchronizedList/Set/Map?**
Collections.unmodifiableList/Set/Map создаёт неизменяемую (read-only) обёртку над коллекцией: все методы модификации (add, remove и т.д.) бросают UnsupportedOperationException. Collections.synchronizedList/Set/Map создаёт потокобезопасную обёртку, где каждый метод коллекции обёрнут в synchronized-блок, что делает её безопасной для использования несколькими потоками.

**Разница между synchronized и concurrent коллекциями?**
Synchronized-коллекции (Collections.synchronized*) блокируют весь объект при каждой операции, что безопасно, но сильно снижает производительность при высокой конкуренции. Concurrent-коллекции (ConcurrentHashMap, CopyOnWriteArrayList и т.д.) используют сегментную блокировку, CAS или копирование, обеспечивая лучшую конкурентность и масштабируемость в многопоточных приложениях.

**Что такое unmodifiable vs immutable коллекции (Java 9+)?**
Unmodifiable-коллекции (Collections.unmodifiableList и т.д.) — это view: исходная коллекция остаётся изменяемой, но через unmodifiable-обёртку модификация невозможна. Immutable-коллекции (List.of(), Set.of(), Map.of() с Java 9+) — это настоящие неизменяемые копии: сами данные защищены, никто не может их изменить даже через оригинал.

**Как получить synchronized Map/List/Set?**
Используйте статические методы Collections: synchronizedMap(new HashMap<>()), synchronizedList(new ArrayList<>()), synchronizedSet(new HashSet<>()).

**Что такое Iterator и ListIterator? Разница Iterator и ListIterator?**
Iterator — базовый итератор для всех Collection: методы hasNext(), next(), remove(). ListIterator — расширение Iterator, специфичное для List: позволяет двигаться в обе стороны (hasPrevious(), previous()), добавлять элементы (add()), заменять текущий (set()).

**Что такое Spliterator (Java 8)? Как Spliterator используется в parallel streams?**
Spliterator — итератор, способный разделяться на части (trySplit()), что позволяет эффективно обрабатывать коллекции параллельно. В parallel streams Spliterator делит данные на подмножества, каждое из которых обрабатывается в отдельном потоке, что ускоряет операции map, filter, reduce.

**Что такое ConcurrentSkipListMap/Set?**
ConcurrentSkipListMap/Set — потокобезопасные реализации Map/Set с сортировкой (на основе skip list). Поддерживают O(log n) операции и параллельный доступ, подходят для случаев, когда нужна одновременно сортировка и потокобезопасность.

**Почему mutable объекты опасны как ключи в HashMap?**
Если ключ изменяется после добавления в HashMap, его hashCode() может измениться, и он окажется в неправильном бакете. В результате get() не найдёт элемент, хотя он физически присутствует в коллекции, что приводит к логическим ошибкам.

**Как правильно переопределять hashCode и equals?**
Метод equals() должен проверять логическое равенство объектов. hashCode() должен возвращать одинаковые значения для равных объектов (контракт: если a.equals(b), то a.hashCode() == b.hashCode()). Используйте Objects.equals() и Objects.hash() для удобного и безопасного переопределения.

**Что происходит, если equals изменён для ключа в HashMap?**
Если после вставки изменить ключ так, что equals() начнёт возвращать другое значение, объект останется в старом бакете. При поиске по новому значению HashMap не найдёт ключ, хотя он физически присутствует.

**Как избежать ConcurrentModificationException?**
Не модифицировать коллекцию во время итерации for-each или Iterator. Использовать итератор явно и вызывать remove() только через него. Для многопоточной модификации применять concurrent-коллекции (ConcurrentHashMap, CopyOnWriteArrayList) или блокировки.

**Как настроить HashMap для лучшей производительности?**
Заранее задавать initialCapacity и loadFactor (по умолчанию 0.75). initialCapacity = (ожидаемое количество элементов / loadFactor) + 1. Использовать immutable ключи с хорошим hashCode, чтобы минимизировать коллизии.

**Почему String/Integer хороши как ключи в Map?**
String и Integer — immutable, их hashCode() и equals() надёжны и хорошо распределены. Не меняются после создания, поэтому ключ не «потеряется» в HashMap. String кэширует литералы, Integer кэширует значения -128..127.

**Какие коллекции чаще используются в Android-разработке?**
ArrayList — для списков данных, HashMap — для словарей (ID → объект), HashSet — для уникальных элементов, SparseArray/SparseIntArray — для int-ключей, LruCache — для кэша, ConcurrentHashMap — в многопоточных сценариях.

**Почему в Android есть SparseArray/SparseIntArray/SparseBooleanArray?**
Эти классы оптимизированы для хранения данных с int-ключами. Они избегают боксинга Integer и используют два массива (ключи и значения) с бинарным поиском, что экономит память и снижает overhead по сравнению с HashMap<Integer, V>.

## Разница SparseArray и HashMap в Android? Когда использовать SparseArray вместо HashMap?
SparseArray использует два массива + бинарный поиск, не боксит ключи, потребляет меньше памяти. HashMap — хэш-таблица с боксингом Integer. SparseArray предпочтительнее при int-ключах и разреженных данных (например, View по ID).

**Что такое LruCache и как работает? Пример LruCache для кэша изображений.**
LruCache — кэш с политикой Least Recently Used: хранит ограниченное число элементов, при превышении размера удаляет наименее недавно использованный. Реализован на базе LinkedHashMap с access-order. Пример: LruCache<String, Bitmap> для кэширования изображений по URL — Bitmap загружается один раз, затем берётся из кэша.

## Как ArrayMap отличается от HashMap в Android? Когда использовать ArrayMap вместо HashMap?
ArrayMap — компактная реализация Map на двух массивах (ключи и значения) + хэш. Работает медленнее HashMap при большом числе элементов, но потребляет меньше памяти для маленьких коллекций (< ~1000 элементов). Использовать ArrayMap в Android для небольших словарей (например, в Bundle, View-параметры).

**Deque** — это интерфейс (Double Ended Queue), расширяющий Queue и позволяющий добавлять/удалять элементы с обоих концов (голова и хвост).

**ArrayDeque** — эффективная реализация Deque на основе циклического массива, обеспечивающая O(1) для операций на концах, без overhead на узлы и с лучшей производительностью по сравнению с LinkedList как Deque.

**Как коллекции влияют на память в Android?**
Коллекции создают множество объектов, особенно при боксинге примитивов (Integer вместо int). HashMap имеет overhead на бакеты и записи. Большие коллекции → больше GC-пауз и риск OOM. Разреженные данные или примитивы → лучше SparseArray/ArrayMap.

**Как избежать memory leak с коллекциями в Android?**
Не храните Context, Activity, Fragment, View в статических коллекциях или долгоживущих объектах. Используйте WeakReference для ключей/значений, если нужно. Очищайте коллекции в onDestroy/onStop. Применяйте LruCache вместо обычных Map для кэша.

# GIT

**Что такое Git и чем он отличается от GitHub?**
Git — это распределённая система контроля версий, разработанная для отслеживания изменений в исходном коде проектов, позволяющая нескольким разработчикам работать над одним кодом одновременно без потери данных. Она работает локально на компьютере пользователя, храня всю историю изменений в репозитории, и поддерживает операции такие как создание веток, слияние изменений и восстановление версий. GitHub, в свою очередь, — это веб-платформа для хостинга Git-репозиториев, предоставляющая дополнительные инструменты для совместной работы, такие как pull requests, issues, code review и интеграция с CI/CD. Основное отличие заключается в том, что Git — это инструмент для контроля версий, а GitHub — облачный сервис, использующий Git как основу, но добавляющий социальные и коллаборативные функции для удалённого хранения и сотрудничества.

**Что такое репозиторий (repository)?**
Репозиторий в Git — это директория, где хранится весь проект с историей изменений, включая файлы кода, конфигурации и метаданные. Он представляет собой базу данных, содержащую все коммиты, ветки, теги и ссылки, позволяющую отслеживать эволюцию проекта во времени. Локальный репозиторий находится на компьютере разработчика и может быть синхронизирован с удалённым репозиторием на серверах вроде GitHub. Репозиторий инициализируется командой git init, и в нём хранится скрытая папка .git, содержащая всю внутреннюю информацию. В контексте Android-разработки репозиторий часто содержит код приложения, ресурсы, Gradle-файлы и историю версий для совместной работы команды.

**Что такое commit? Как его создать?**
Commit в Git — это фиксация изменений в репозитории, представляющая собой снимок состояния проекта на определённый момент времени, с уникальным идентификатором (SHA-1 хэш), сообщением и метаданными (автор, дата). Он позволяет сохранять историю изменений и возвращаться к предыдущим версиям. Чтобы создать commit, сначала добавьте файлы в staging area с помощью git add <файл>, затем выполните git commit -m "Сообщение о изменениях", где сообщение описывает суть коммита. В Android-разработке commit часто фиксирует добавление новой фичи или исправление бага, обеспечивая traceability в проекте.

**Что такое staging area (индекс)? Как добавить файлы в staging?**
Staging area, или индекс, в Git — это промежуточная область между рабочей директорией и репозиторием, где собираются изменения перед их фиксацией в commit. Она позволяет выбрать, какие модификации включить в следующий commit, исключая ненужные файлы. Чтобы добавить файлы в staging, используйте команду git add <файл> для конкретного файла или git add . для всех изменённых файлов. В Android это полезно для поэтапного добавления изменений в Gradle скрипты или XML-файлы, чтобы commit был атомарным и осмысленным.

**Чем отличается git commit -a от обычного commit?**
Опция -a в git commit -a -m "message" автоматически добавляет все отслеживаемые изменённые файлы в staging area перед коммитом, пропуская шаг git add. Обычный commit требует предварительного добавления файлов в staging. Это удобно для быстрых коммитов, но не добавляет новые файлы (только tracked). В Android это ускоряет фиксацию мелких правок в коде.

**Что такое HEAD в Git?**
HEAD — это указатель на текущий commit или ветку, в которой находится рабочая директория. Он представляет "голову" текущей ветки и обновляется при коммитах или checkout. HEAD может быть detached (не привязан к ветке), что полезно для экспериментов. В Android HEAD помогает в навигации по истории, например, при git checkout HEAD~1 для просмотра предыдущего состояния проекта.

**Что такое branch и зачем нужны ветки?**
Branch — это параллельная линия разработки, указатель на commit, позволяющий работать над функциями независимо от основной ветки. Ветки нужны для изоляции изменений (feature branches, bugfix), экспериментов и параллельной работы команды. В Android ветки используются для разработки новых фич (branch "feature/new-ui"), без влияния на master/main.

**Как создать новую ветку? Как переключиться на неё?**
Новая ветка создаётся командой git branch <имя-ветки>, где имя — описательное (например, "feature/login"). Чтобы переключиться, используйте git checkout <имя-ветки> или git switch <имя-ветки> (в новых версиях Git). Альтернатива: git checkout -b <имя> создаёт и сразу переключает. В Android это позволяет работать над новым экраном без риска сломать основной код.

**Как слить (merge) ветку в основную?**
Чтобы слить ветку, переключитесь на основную (например, git checkout main), затем выполните git merge <имя-ветки>. Git автоматически создаст merge-commit, если нет конфликтов. Если конфликты — разрешите вручную в файлах и закоммитьте. В Android merge используется для интеграции фичи в main после code review.

**Что такое merge conflict и как его разрешить?**
Merge conflict возникает, когда в сливаемых ветках изменены одни и те же строки файлов. Git помечает конфликты в файлах (<<<, ===, >>>). Чтобы разрешить, откройте файл, отредактируйте нужную версию, удалите маркеры, добавьте в staging (git add) и завершите merge коммитом. В Android конфликты часто в XML или Gradle, используйте инструменты вроде IntelliJ для разрешения.

**Что такое pull request (PR) и зачем он нужен?**
Pull request — запрос на слияние изменений из одной ветки в другую (обычно в main), с возможностью code review, обсуждения и тестов. Зачем нужен: обеспечивает качество кода, коллаборацию, интеграцию CI/CD. На GitHub PR создаётся через UI, с описанием, скриншотами и тегами.

**Как создать pull request на GitHub?**
На GitHub перейдите в репозиторий, выберите ветку с изменениями, кликните "New pull request". Выберите базовую ветку (main), добавьте заголовок, описание, назначьте ревьюеров. Добавьте labels, milestones. После создания PR проходит code review, тесты, и merge.

**Что такое fork репозитория?**
Fork — копия репозитория на вашем аккаунте GitHub, позволяющая вносить изменения без влияния на оригинал. Используется для вклада в open-source: форкните, измените, создайте PR в оригинал. В Android fork полезен для экспериментов с библиотеками.

**Как сделать fork и внести изменения через pull request?**
На GitHub кликните "Fork" в репозитории. Клонируйте свой fork локально (git clone <url>). Создайте ветку, внесите изменения, закоммитьте и запушьте. Затем создайте PR из своей ветки в оригинальный репозиторий. В Android это стандарт для вклада в библиотеки вроде Retrofit.

**Отличие merge и rebase**
Merge создаёт новый коммит (merge commit), который объединяет две ветки, сохраняя полную историю изменений обеих веток в виде графа с двумя родителями. История остаётся нелинейной, но полностью отражает, когда и как ветки развивались параллельно.
Команда: git merge feature-branch (находясь на main).
Rebase переписывает историю текущей ветки, перенося её коммиты поверх целевой ветки (обычно main). В результате получается линейная история, как будто все изменения делались последовательно в одной ветке.
Команда: git rebase main (находясь на feature-branch).

**Что такое remote repository?**
Remote repository — это версия вашего проекта, размещённая на удалённом сервере (GitHub, GitLab, Bitbucket и т.д.), которая служит центральным хранилищем кода для совместной работы команды. Он позволяет синхронизировать локальные изменения с коллегами через команды push и pull, а также выступает основой для code review, pull request и CI/CD. В отличие от локального репозитория, remote хранит историю в облаке и обеспечивает доступность проекта для всех участников. Обычно основной remote называется origin и добавляется автоматически при клонировании или явно через git remote add origin <url>.

**Чем отличается git pull от git fetch + git merge?**
Команда git pull — это комбинация двух операций: git fetch (загружает изменения из remote без их применения) и git merge (сливает загруженные изменения в текущую ветку). Таким образом, git pull сразу обновляет локальную ветку, что удобно, но менее контролируемо. Вариант git fetch + git merge позволяет сначала посмотреть, что именно будет влито (например, через git log origin/main..main), разрешить возможные конфликты осознанно или даже отменить merge, если изменения нежелательны. Это более безопасный и прозрачный подход, особенно в командах с частыми изменениями в основной ветке.

**Что такое git clone и что он делает?**
git clone <url> — команда, которая полностью копирует удалённый репозиторий на локальный компьютер, создавая локальную копию со всей историей коммитов, ветками и тегами. При этом автоматически создаётся remote с именем origin, указывающий на исходный URL, и выполняется checkout основной ветки (обычно main или master). Клонирование скачивает не только файлы текущего состояния, но и всю метаданную Git, что позволяет работать автономно и синхронизироваться позже через push/pull. В Android-проектах это стандартный первый шаг для начала работы над кодом.

**Что такое git push --force и почему он опасен?**
**Что такое git push --force-with-lease?**
git push --force принудительно переписывает историю на удалённом репозитории, заменяя её локальной версией ветки, даже если удалённая ветка содержит коммиты, которых нет локально. Это опасно, потому что может удалить чужие коммиты, нарушить работу коллег, которые уже основывались на старой версии ветки, и привести к потере истории.
git push --force-with-lease — более безопасная альтернатива: она проверяет, что удалённая ветка не изменилась с момента последнего fetch. Если кто-то успел запушить изменения, push отклоняется, предотвращая случайное удаление чужой работы. Это рекомендуемый способ в командах для безопасного переписывания истории.

**Что такое protected branch на GitHub?**
Protected branch — это ветка на GitHub (обычно main или master), для которой включены правила защиты: запрет прямого push'а, обязательный code review, проверка статуса CI/CD, ограничение на слияние только через pull request. Это предотвращает случайные или нерецензированные изменения в основной ветке, обеспечивает стабильность и качество кода. Настройки защиты находятся в разделе Settings → Branches репозитория.

**Что такое GitHub Actions?**
**Как создать простой workflow (CI/CD) в Git Actions?**
GitHub Actions — встроенная платформа CI/CD, позволяющая автоматизировать сборку, тестирование, деплой и другие задачи прямо в репозитории через YAML-файлы в директории .github/workflows. Простой workflow создаётся следующим образом: создайте файл .github/workflows/ci.yml, укажите on: [push, pull_request], затем добавьте job с runs-on: ubuntu-latest, шагами checkout, setup-java (для Android), gradlew build/test. Пример: запуск ./gradlew build и ./gradlew test на каждый push в main. Workflow запускается автоматически и отображается в разделе Actions репозитория.

**Что такое GitHub Pages?**
GitHub Pages — бесплатный сервис статического хостинга, позволяющий публиковать веб-сайты прямо из репозитория GitHub. Поддерживает HTML, CSS, JavaScript и генераторы сайтов (Jekyll, Hugo и т.д.). Активируется в настройках репозитория (Settings → Pages), можно выбрать ветку (обычно gh-pages или main) и папку (например, /docs). Используется для документации проектов, портфолио или демонстрации Android-приложений (например, статические скриншоты или демо-страницы).

**Как работает GitHub Issues и Projects?**
GitHub Issues — система для отслеживания задач, багов, улучшений и обсуждений в проекте. Каждая issue имеет заголовок, описание, labels, assignee, milestone и комментарии. GitHub Projects — Kanban-доска (или таблица), связанная с репозиторием, где issues и PR визуализируются как карточки (To Do → In Progress → Done). Поддерживает автоматизацию (например, перемещение issue в Done при merge PR). В Android-проектах это стандартный инструмент планирования фич и багфиксов.

**Что такое GitHub Releases?**
GitHub Releases — функциональность для публикации готовых версий проекта (APK, JAR, бинарники и т.д.) с тегами, changelog, описанием и прикреплёнными файлами. Releases привязаны к git-тегам и отображаются в разделе Releases репозитория. Поддерживают pre-releases и assets (например, APK для Android).

**Как создать release с тегом и changelog?**
Создайте тег локально (git tag v1.0.0 и git push origin v1.0.0) или через интерфейс GitHub. Перейдите в раздел Releases → Draft a new release → выберите тег (или создайте новый) → добавьте заголовок, описание (можно сгенерировать changelog из коммитов через «Generate release notes»), прикрепите файлы (например, app-release.apk) → отметьте «This is a pre-release» при необходимости → нажмите Publish release. В Android это стандартный способ публикации версий приложения.

**Как отменить последний commit (git reset, git revert)?**
Отмена последнего коммита может выполняться двумя основными способами в зависимости от того, был ли коммит уже запушен в удалённый репозиторий. Команда git reset --hard HEAD~1 полностью удаляет последний коммит и все изменения в рабочей директории, возвращая HEAD на предыдущий коммит (опасно для запушенных изменений, так как переписывает историю). Альтернатива — git revert HEAD, которая создаёт новый коммит, отменяющий изменения предыдущего, сохраняя историю неизменной и безопасно работая с уже опубликованными коммитами. В Android-проектах git revert предпочтительнее для main-ветки, чтобы не нарушить работу команды, а reset используется локально для быстрой очистки неудачных экспериментов.

**Что такое cherry-pick и когда он нужен?**
Cherry-pick — это команда git cherry-pick <commit-hash>, которая позволяет применить изменения из конкретного коммита другой ветки в текущую ветку, без слияния всей ветки. Это полезно, когда нужно перенести только одно исправление бага или небольшую фичу, не затрагивая остальную историю разработки. Например, если баг был исправлен в feature-ветке, но релиз уже вышел, cherry-pick позволяет быстро перенести фикс в release-ветку. Команда создаёт новый коммит с теми же изменениями, но новым хэшем, что требует осторожности при повторном применении (возможны конфликты).

**Что такое stash и как его использовать?**
Stash — это временное сохранение незакоммиченных изменений рабочей директории и индекса, чтобы быстро переключиться на другую ветку или задачу без создания коммита. Команда git stash сохраняет изменения в стек (последний сохранённый — сверху), git stash list показывает список, git stash apply или git stash pop применяет их обратно (pop удаляет из стека). В Android это удобно при срочном переключении на багфикс, когда в рабочей ветке остались недоделанные изменения — их можно «спрятать», пофиксить баг и вернуть stash.

**Как работать с submodule в Git?**
Submodule — это способ включить другой репозиторий как поддиректорию в основной проект, сохраняя независимую историю. Добавляется командой git submodule add <url> path, после чего в репозитории появляется файл .gitmodules и ссылка на коммит подмодуля. При клонировании проекта нужно выполнить git submodule update --init --recursive, чтобы загрузить содержимое. Обновление: git submodule update --remote или переход в подмодуль и pull. В Android submodules часто используются для библиотек (например, Room, Retrofit), чтобы фиксировать конкретную версию зависимости.

**Что такое .gitignore и как правильно его настроить?**
Файл .gitignore — это текстовый файл в корне репозитория, содержащий шаблоны файлов и директорий, которые Git должен игнорировать при коммитах (например, build/, *.apk, local.properties). Он предотвращает попадание временных файлов, кэшей Gradle, ключей API и локальных настроек в репозиторий. Правильная настройка включает использование готовых шаблонов (например, от github/gitignore для Android), игнорирование больших файлов (через Git LFS для бинарников) и проверку через git status --ignored. В Android типичный .gitignore исключает build/, .idea/, *.iml, local.properties и keystore-файлы.

**Как исправить последний commit (git commit --amend)?**
Команда git commit --amend позволяет изменить сообщение последнего коммита или добавить в него пропущенные файлы без создания нового коммита. Если нужно добавить изменения, сначала git add нужные файлы, затем git commit --amend (или --amend -m "новое сообщение"). Если коммит уже запушен, потребуется git push --force-with-lease (или --force), но это опасно в команде. В Android это удобно для исправления мелких опечаток в сообщении коммита или добавления забытого ресурса.

**Как удалить файл из истории Git (git filter-branch / BFG Repo-Cleaner)?**
Удаление файла из всей истории Git требуется, когда в репозиторий попали чувствительные данные (ключи, пароли). Классический способ — git filter-branch --force --index-filter 'git rm --cached --ignore-unmatch путь/к/файлу' --prune-empty --tag-name-filter cat -- --all, после чего нужно принудительно запушить изменения. Более современный и быстрый инструмент — BFG Repo-Cleaner (Java-утилита), который проще в использовании и быстрее работает с большими репозиториями. После очистки рекомендуется изменить все пароли/ключи.

**Что такое GitHub CLI и как его использовать?**
GitHub CLI (gh) — официальный командный интерфейс GitHub для работы с репозиториями прямо из терминала. Устанавливается через brew, scoop или пакетный менеджер. Основные команды: gh repo create, gh pr create, gh issue list, gh release create. Например, gh pr create --title "Fix bug" --body "Описание" создаёт pull request. В Android это ускоряет работу с PR и релизами без перехода в браузер.

**Как настроить SSH-ключи для GitHub?**
Для настройки SSH-ключей сгенерируйте пару ключей командой ssh-keygen -t ed25519 -C "your@email.com", сохраните в ~/.ssh/id_ed25519. Скопируйте публичный ключ (cat ~/.ssh/id_ed25519.pub), добавьте его в GitHub → Settings → SSH and GPG keys → New SSH key. Проверьте подключение командой ssh -T git@github.com. После этого git clone/push будет работать без ввода логина/пароля.

**Что такое GitHub token (PAT) и как его использовать?**
Personal Access Token (PAT) — это токен аутентификации вместо пароля для операций через HTTPS. Создаётся в GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (выберите scopes: repo, workflow и т.д.). Токен используется вместо пароля: git clone https://<username>:<token>@github.com/user/repo.git или в credential helper. PAT необходим для GitHub Actions, скриптов и CI.

**Как настроить GitHub Actions для Android-проекта (build, tests, lint)?**
Для Android-проекта создайте файл .github/workflows/android.yml. Пример: триггер on: [push, pull_request], job на runs-on: ubuntu-latest с шагами checkout, установка JDK, Gradle cache, ./gradlew build test lint. Добавьте secrets для подписывания APK. Workflow может включать upload-artifact для APK и уведомления в Slack/Telegram

**Что такое Dependabot и как он работает?**
Dependabot — встроенный инструмент GitHub для автоматического обновления зависимостей (Gradle, npm, Maven и т.д.). Он сканирует репозиторий, находит устаревшие версии библиотек, создаёт pull request с обновлением и changelog. Настраивается в .github/dependabot.yml (указание экосистемы, интервал проверки). В Android Dependabot обновляет версии в build.gradle и создаёт PR с комментарием о безопасности и изменениях.

# Multithreading

**Что такое процесс и поток? В чём разница?**
Процесс — это независимая единица выполнения операционной системы, которая имеет собственное адресное пространство, ресурсы (файлы, память, дескрипторы) и изолирована от других процессов. Каждый процесс запускается операционной системой и может содержать один или несколько потоков. Поток (thread) — это наименьшая единица выполнения внутри процесса, которая делит с другими потоками того же процесса адресное пространство, открытые файлы и ресурсы, но имеет собственный стек вызовов, регистры процессора и счётчик команд. Основное различие заключается в изоляции: процессы полностью изолированы друг от друга (переключение между ними дорогое), тогда как потоки внутри одного процесса работают в общей памяти и переключаются гораздо быстрее. В Android каждый процесс имеет свой собственный Dalvik/ART VM, а приложение обычно запускается в одном процессе, внутри которого и выполняются все потоки.

**Что такое Main Thread (UI Thread) в Android и почему он критически важен?**
Main Thread (он же UI Thread) — это основной поток Android-приложения, созданный системой при запуске процесса приложения. Именно в нём обрабатываются все события пользовательского интерфейса: отрисовка экрана, обработка касаний, нажатий, анимации, обновление View и вызовы методов жизненного цикла Activity/Fragment. Этот поток критически важен, потому что Android строго запрещает выполнять длительные операции в нём: любая блокировка дольше нескольких секунд приводит к появлению диалога ANR (Application Not Responding), приложение считается «зависшим», и пользователь может его принудительно закрыть. Кроме того, все изменения UI-элементов (вызовы setText, setVisibility и т.д.) должны происходить именно в Main Thread — иначе возникает исключение CalledFromWrongThreadException. Именно поэтому все тяжёлые операции (сеть, база данных, обработка файлов) выносятся в фоновые потоки или корутины.

**Что такое асинхронность и синхронность в контексте Android-приложения?**
Синхронность означает, что операция выполняется последовательно: текущий поток блокируется до полного завершения задачи (например, сетевой запрос на Main Thread — приложение «замерзает» до получения ответа). Асинхронность позволяет запускать длительную операцию в фоновом потоке или корутине, не блокируя основной поток: задача выполняется параллельно, а результат обрабатывается позже через callback, LiveData/StateFlow или suspend-функцию. В Android асинхронность критически важна для поддержания отзывчивости интерфейса: пользователь может продолжать взаимодействовать с приложением, пока идёт загрузка данных, парсинг JSON или запись в базу. Примеры асинхронных механизмов: Coroutines, WorkManager, Retrofit с suspend, Room с Flow/StateFlow.

**Можно ли выполнять сетевые запросы и работу с БД на Main Thread?**
Нет, выполнять сетевые запросы и операции с базой данных на Main Thread категорически запрещено в Android начиная с API 11 (Android 3.0). Система намеренно выбрасывает NetworkOnMainThreadException при попытке сетевого запроса и блокирует UI при длительных операциях с БД, потому что это приводит к зависанию интерфейса и ANR. Сетевые запросы могут занимать от сотен миллисекунд до десятков секунд (в зависимости от сети), а операции с БД — от миллисекунд до секунд при сложных запросах. Это нарушает принцип отзывчивости: пользователь видит «замерзший» экран и не может взаимодействовать с приложением. Современные библиотеки (Retrofit с suspend, Room с Flow, WorkManager) изначально проектируются для асинхронной работы.

**Как создать и запустить поток в Kotlin/Java?**
В Java/Kotlin поток создаётся несколькими способами: через наследование от класса Thread (class MyThread : Thread() { override fun run() { ... } }), через реализацию интерфейса Runnable (val runnable = Runnable { ... }; Thread(runnable).start()), или через лямбду (Thread { ... }.start()). В Kotlin чаще используют корутины, но классический способ остаётся актуальным для низкоуровневых задач. Запуск потока выполняется исключительно методом start(), который создаёт новый поток ОС и вызывает метод run(). Прямой вызов run() просто выполнит код в текущем потоке, без создания нового. В Android создание потоков вручную используется редко — предпочтительнее Coroutines или WorkManager.

**Разница между Thread.start() и Thread.run()?**
Метод start() запускает новый поток операционной системы, создаёт стек вызовов и вызывает метод run() в этом новом потоке. После вызова start() поток переходит в состояние runnable и начинает независимое выполнение. Прямой вызов run() просто выполняет код метода в текущем потоке, без создания нового — это обычный вызов функции. Ошибка новичков — вызов run() вместо start(), из-за чего код выполняется синхронно в вызывающем потоке, а не асинхронно. В Android это критично: если вызвать run() на потоке, который должен обновлять UI, изменения могут произойти не в Main Thread.

**Что такое Daemon Thread?**
Daemon Thread (демон-поток) — это фоновый поток, который автоматически завершается, когда все не-демон потоки (user threads) в приложении заканчивают работу. JVM не ждёт завершения daemon-потоков при выходе из программы. Примеры: потоки сборщика мусора, потоки таймеров, потоки обработки событий в Swing. В Android daemon-потоки используются редко, так как приложение может быть убито системой в любой момент; вместо этого применяются Coroutines с правильной отменой или WorkManager для фоновых задач.

**Что делают методы sleep(), yield(), join() и interrupt()?**
Thread.sleep(millis) — приостанавливает текущий поток на указанное время, не освобождая монитор (если поток внутри synchronized). yield() — предлагает планировщику отдать процессорное время другому потоку того же приоритета (не гарантировано). join() — заставляет текущий поток ждать завершения другого потока (например, thread.join() ждёт окончания thread). interrupt() — устанавливает флаг прерывания потока; поток может проверить флаг через Thread.interrupted() или поймать InterruptedException (например, из sleep). В Android эти методы используются редко — предпочтительнее Coroutines с delay() и cancel().

**Что такое ThreadLocal и где его используют в Android?**
ThreadLocal — класс, который предоставляет переменную, уникальную для каждого потока: каждый поток видит своё собственное значение. При вызове set() значение сохраняется только для текущего потока, get() возвращает значение именно этого потока. Используется для хранения контекста, который должен быть разным в каждом потоке (например, локальный кэш, транзакция БД, пользовательская локаль). В Android классический пример — Looper и Handler, где каждый поток имеет свой Looper через ThreadLocal. Также может использоваться для хранения временных данных в пуле потоков (например, в ExecutorService).

**Что такое Race Condition? Приведи реальный пример из Android.**
Race Condition — ситуация, когда результат программы зависит от порядка выполнения потоков, а не от логики кода. Это происходит при одновременном доступе нескольких потоков к общей изменяемой переменной без синхронизации. Реальный пример в Android: счётчик нажатий кнопки var clickCount = 0. Два быстрых нажатия в разных потоках (например, через debounce и параллельный запрос) могут прочитать одно и то же значение, увеличить его и записать одинаковый результат — вместо +2 получится +1. Решается через synchronized, AtomicInteger или Mutex в Coroutines.

**Что такое Deadlock? Как его предотвратить?**
Deadlock — ситуация, когда два или более потока заблокировали друг друга, ожидая освобождения ресурсов, которые держит противоположный поток, и ни один не может продолжить выполнение. Классический пример — два потока, каждый из которых захватил один монитор и ждёт второй. Предотвращение: всегда захватывать ресурсы в одном и том же порядке, использовать таймауты на lock (tryLock()), применять ReentrantLock с fairness, избегать вложенных блокировок, использовать высокоуровневые механизмы (Coroutines, Channels). В Android deadlock часто возникает при неправильной синхронизации с UI (например, synchronized блок на Main Thread + ожидание фонового результата).

**Что такое Livelock и Starvation?**
Livelock — ситуация, когда потоки активно выполняют действия, но не могут продвинуться к цели из-за постоянной реакции друг на друга (например, два потока постоянно уступают друг другу дорогу, переходя в состояние ожидания). Starvation — когда поток или группа потоков постоянно откладываются и не получают процессорное время из-за более высокоприоритетных потоков или несправедливого планировщика. В Android livelock может возникнуть в сложных системах уведомлений, starvation — при высокой конкуренции за Main Thread или в пуле потоков с фиксированным размером.

**Что такое Memory Visibility Problem и как с ним бороться?**
Memory Visibility Problem — ситуация, когда один поток записал значение в переменную, но другой поток не видит это изменение из-за кэширования процессора или реордеринга инструкций. В Java/Kotlin без правильной синхронизации изменения могут быть невидимы для других потоков. Борьба: использование volatile (гарантирует видимость и запрещает реордеринг), synchronized, Atomic классы, Lock с volatile-полями, или высокоуровневые механизмы Coroutines с правильным диспетчером.

**Что такое Happens-Before relationship?**
Happens-Before — отношение в модели памяти Java, которое гарантирует, что операции одного потока видны другому потоку в определённом порядке. Основные правила: запись в volatile-поле happens-before чтение этого поля; выход из synchronized-блока happens-before вход в тот же блок; завершение потока happens-before join(); действия в конструкторе happens-before присвоение ссылки на объект. Это формальная гарантия видимости изменений без лишних блокировок. В Android это критично при работе с LiveData/StateFlow и многопоточными обновлениями UI.

**Как работает ключевое слово synchronized?**
Ключевое слово synchronized в Java (и Kotlin через @Synchronized или synchronized блок) обеспечивает взаимоисключающий доступ к критической секции кода, предотвращая одновременное выполнение этой секции несколькими потоками. При использовании на методе (synchronized fun method()) блокировка происходит на объекте this (для нестатических методов) или на классе (для статических). При использовании в блоке (synchronized(lock) { ... }) блокировка захватывается на указанном объекте-мониторе. Когда поток входит в synchronized-секцию, он захватывает монитор объекта; другие потоки, пытающиеся войти в эту же секцию, блокируются до освобождения монитора. После выхода из блока монитор автоматически освобождается, даже при исключении. Это гарантирует атомарность операций и видимость изменений памяти (happens-before).

**Разница между synchronized методом и блоком?**
Synchronized-метод автоматически оборачивает весь код метода в блокировку на объекте this (или классе для статического метода), что удобно, но может привести к избыточной длительности блокировки, если часть метода не требует синхронизации. Synchronized-блок позволяет точно указать объект-монитор (synchronized(lock) { ... }) и ограничить синхронизируемую область только критической секцией, что повышает производительность при высокой конкуренции. Блок также даёт гибкость: можно использовать разные объекты-мониторы для разных операций в одном классе. В Android рекомендуется использовать блок с явным лок-объектом (например, private val lock = Any()), чтобы избежать случайной блокировки на самом объекте класса, что может привести к неожиданным дедлокам.

**Что такое volatile и когда его обязательно использовать?**
volatile — модификатор поля, который гарантирует видимость изменений значения этого поля для всех потоков и запрещает компилятору и процессору выполнять реордеринг инструкций вокруг доступа к полю. Без volatile один поток может не увидеть изменения, сделанные другим потоком, из-за кэширования в локальном кэше процессора. volatile обеспечивает happens-before: запись в volatile-поле видна всем последующим чтениям этого поля. Обязательно использовать volatile для флагов завершения (volatile var running = true), простых статусов или ссылок, которые обновляются одним потоком и читаются другими, без дополнительных операций (например, volatile var latestData: Data?). Не заменяет полную синхронизацию при составных операциях (например, i++).

**Что такое ReentrantLock, ReadWriteLock и когда их использовать вместо synchronized?**
ReentrantLock — явная блокировка с возможностью повторного захвата тем же потоком (reentrant), поддержкой tryLock с таймаутом, fairness и interruptible lock. В отличие от synchronized, позволяет проверять состояние блокировки, прерывать ожидание и выбирать справедливый порядок. ReadWriteLock (обычно ReentrantReadWriteLock) разделяет блокировку на чтение (множественные читатели одновременно) и запись (эксклюзивно). Используют вместо synchronized, когда нужна тонкая блокировка: ReentrantLock — для сложной логики синхронизации (таймауты, fairness), ReadWriteLock — для сценариев «читать много, писать редко» (например, кэш данных в Android-приложении). В большинстве случаев synchronized проще и быстрее, но явные locks предпочтительнее при необходимости продвинутого контроля.

**Что такое Atomic-классы (AtomicInteger, AtomicReference и т.д.)?**
Atomic-классы из пакета java.util.concurrent.atomic предоставляют потокобезопасные операции над примитивными типами и ссылками без явной блокировки. Они используют Compare-And-Swap (CAS) на уровне процессора, что делает операции атомарными и неблокирующими. Примеры: AtomicInteger.incrementAndGet(), AtomicReference.compareAndSet(). Позволяют выполнять простые операции (get, set, increment, compareAndSet) без synchronized, что снижает overhead и повышает производительность при высокой конкуренции.

**Чем Atomic лучше synchronized в некоторых случаях?**
Atomic-классы обеспечивают атомарность без блокировки, используя lock-free алгоритмы на основе CAS, что исключает переключение контекста и ожидание в очереди блокировок. synchronized всегда использует монитор и может привести к приостановке потоков, особенно при высокой конкуренции. Atomic предпочтительнее для простых операций (счётчики, флаги, одиночные ссылки), где не требуется сложная логика внутри критической секции. В Android это критично для счётчиков нажатий, состояния загрузки или обновления UI-данных из нескольких корутин без лишних блокировок.

**Что такое Coroutines и чем они принципиально отличаются от Threads?**
Coroutines — это лёгковесный механизм асинхронного программирования в Kotlin, позволяющий писать асинхронный код в последовательном стиле с помощью suspend-функций. В отличие от потоков, корутины не привязаны к конкретному потоку ОС: они могут приостанавливаться и возобновляться на разных потоках, не блокируя их. Потоки — дорогостоящие (1 МБ стека, переключение контекста), их количество ограничено (тысячи). Корутины — дешёвые (несколько килобайт), их можно создавать десятками тысяч. В Android корутины позволяют писать асинхронный код без callback hell, с автоматической отменой при уничтожении экрана и поддержкой Structured Concurrency.

**Что такое suspend функция и как она работает под капотом?**
suspend — модификатор функции, указывающий, что она может приостанавливаться (suspend) без блокировки потока. Под капотом компилятор преобразует suspend-функцию в Continuation-passing style: добавляется параметр Continuation, который хранит состояние приостановки. При вызове suspend функция возвращает результат через Continuation.resume(). Это позволяет библиотекам (например, Retrofit, Room) приостанавливать выполнение на время ожидания I/O, освобождая поток. В отличие от обычных функций, suspend-функции можно вызывать только из других suspend-функций или корутин-билдеров (launch, async).

**Что такое Coroutine Context и Coroutine Scope?**
Coroutine Context — это набор элементов (Job, Dispatcher, CoroutineName, ExceptionHandler и т.д.), определяющих поведение корутины: на каком потоке выполнять, как обрабатывать исключения, есть ли родительская Job. Coroutine Scope — это область видимости корутин, которая управляет жизненным циклом всех дочерних корутин через Job. Scope гарантирует, что все дочерние корутины будут отменены при отмене scope (Structured Concurrency). В Android viewModelScope и lifecycleScope — это готовые scope, автоматически отменяющиеся при уничтожении ViewModel или Fragment.

**Какие встроенные Coroutine Scopes существуют в Android?**
В Android существуют готовые scope: lifecycleScope (привязан к жизненному циклу Activity/Fragment), viewModelScope (привязан к ViewModel, живёт дольше конфигурационных изменений), repeatOnLifecycle (для Flow внутри lifecycle-aware компонентов). Также часто создают свои scope на основе CoroutineScope(SupervisorJob() + Dispatchers.Default) для фоновых задач. GlobalScope существует, но не рекомендуется в продакшене, так как не привязан к жизненному циклу и может привести к утечкам.

**Разница между viewModelScope, lifecycleScope и GlobalScope?**
viewModelScope привязан к жизненному циклу ViewModel: отменяется при onCleared(), идеален для операций, связанных с данными экрана. lifecycleScope привязан к жизненному циклу Activity/Fragment, отменяется при ON_DESTROY, подходит для UI-задач. GlobalScope — глобальный scope без автоматической отмены, может привести к утечкам корутин после уничтожения экрана. Правило: используйте viewModelScope для логики ViewModel, lifecycleScope для задач внутри Activity/Fragment, избегайте GlobalScope.

**Что такое Structured Concurrency и почему это важно?**
Structured Concurrency — принцип, при котором все дочерние корутины должны завершиться до завершения родительской корутины или scope. Это достигается через Coroutine Scope и Job: при отмене scope отменяются все дочерние корутины, при исключении в дочерней корутине оно передаётся вверх. Важно, потому что предотвращает утечки корутин, упрощает управление жизненным циклом и гарантирует, что ресурсы освобождаются. В Android это критично: без Structured Concurrency корутины могут продолжать работать после уничтожения экрана, вызывая утечки памяти и краши.

**Разница между coroutineScope и supervisorScope?**
coroutineScope — обычный scope, который отменяет все дочерние корутины при любом исключении и передаёт исключение вверх (fail-fast). supervisorScope — scope, который не отменяет остальные дочерние корутины при исключении в одной из них и не передаёт исключение вверх (fail-safe). Используется, когда сбой одной задачи не должен останавливать остальные (например, параллельная загрузка нескольких изображений). В Android supervisorScope полезен в ViewModel для независимых фоновых задач.

**Что такое Job и Deferred?**
Job — объект, представляющий жизненный цикл корутины: может быть активным, завершённым, отменённым. Позволяет отменять корутину (job.cancel()) и отслеживать состояние. Deferred — это Job с результатом: возвращается из async, позволяет получить результат через await(). async возвращает Deferred<T>, launch — просто Job. В Android Job используется для отмены задач при уничтожении экрана, Deferred — для получения результата асинхронных вычислений.

**Как правильно отменять корутины?**
Отмена корутин выполняется через Job.cancel() или scope.cancel(). Корутина должна быть кооперативной: проверять isActive или использовать ensureActive(), yield(), delay(). При использовании withContext, withTimeout отмена передаётся автоматически. Лучшая практика — привязывать корутины к viewModelScope или lifecycleScope, которые автоматически отменяются при уничтожении компонента. Вручную отменять нужно редко, но если корутина не кооперативна — отмена не сработает.

**Что такое withContext, withTimeout, withTimeoutOrNull?**
withContext(Dispatcher) переключает контекст выполнения на указанный диспетчер (например, с Main на IO) и ждёт завершения блока. withTimeout(timeMillis) ограничивает время выполнения блока и бросает TimeoutCancellationException при превышении. withTimeoutOrNull возвращает null вместо исключения при таймауте. Все три функции отменяемы и поддерживают Structured Concurrency. В Android withContext(Dispatchers.IO) используется для тяжёлой работы, withTimeout — для ограничения долгих операций.

**Какие Dispatchers существуют? Когда какой использовать?**
Основные диспетчеры: Dispatchers.Main — для работы с UI (обновление View, LiveData.postValue). Dispatchers.IO — для дисковых операций, сетевых запросов, Room-запросов (ограниченный пул потоков). Dispatchers.Default — для CPU-интенсивных задач (парсинг, вычисления), использует пул потоков по числу ядер. Dispatchers.Unconfined — выполняется в текущем потоке без переключения (редко, опасно). Dispatchers.Main.immediate — выполняется немедленно, если уже на Main Thread. В Android Main — для UI, IO — для I/O, Default — для вычислений.

**Почему нельзя использовать Dispatchers.Main для тяжёлой работы?**
Dispatchers.Main привязан к Main Thread (UI Thread), и выполнение длительных операций в нём блокирует отрисовку интерфейса, обработку событий и анимации, что приводит к зависанию UI и ANR. Android строго запрещает тяжёлую работу (сеть, БД, парсинг больших JSON) на Main Thread, выбрасывая исключения (NetworkOnMainThreadException). Все задачи, требующие ожидания или вычислений, должны переключаться на IO или Default через withContext.

**Что такое Dispatchers.IO vs Dispatchers.Default?**
Dispatchers.IO предназначен для операций ввода-вывода (сеть, диск, БД), использует ограниченный пул потоков (по умолчанию до 64 или число ядер × 64), чтобы не перегружать систему. Dispatchers.Default предназначен для CPU-интенсивных задач (парсинг, шифрование, вычисления), использует пул потоков по числу ядер процессора. В Android IO — для Retrofit, Room, файлов; Default — для сложных вычислений в фоне.

**Можно ли запускать корутины на Dispatchers.Main.immediate?**
Да, Dispatchers.Main.immediate позволяет запускать корутину немедленно, если текущий поток уже Main Thread, без постановки в очередь. Это полезно для микрозадач, которые должны выполниться синхронно на UI-потоке (например, немедленное обновление View после события). Обычный Dispatchers.Main всегда ставит задачу в очередь, даже если уже на Main. Однако Main.immediate следует использовать осторожно: если задача тяжёлая, она всё равно заблокирует UI.

**Что такое Kotlin Flow?**
Kotlin Flow — это асинхронный поток данных в Kotlin Coroutines, представляющий собой последовательность значений, которые могут быть вычислены асинхронно и доставлены подписчикам по мере их появления. Flow является холодным (cold) по умолчанию: он начинает производить значения только при наличии активного коллектора (collect) и завершается вместе с ним. Flow поддерживает операторы для трансформации (map, filter), комбинирования (combine, zip) и обработки ошибок, что делает его мощным инструментом для реактивного программирования. В Android Flow широко используется для работы с Room (как Flow-результат запроса), Retrofit (через flow-адаптер), StateFlow для UI-состояния и SharedFlow для событий.

**Разница между Cold Flow и Hot Flow?**
Cold Flow запускает производство данных заново для каждого нового коллектора: каждый collect вызывает выполнение блока flow { … } с нуля. Это означает, что если два коллектора подписываются на один и тот же cold Flow, каждый получит независимый набор значений. Hot Flow (StateFlow, SharedFlow) производит данные независимо от количества коллекторов: эмиссия начинается один раз (обычно при создании или при первом подписчике), и все коллекторы получают одни и те же значения, включая уже существующие (в случае replay). Cold Flow подходит для одноразовых запросов (например, сетевой вызов), hot Flow — для общего состояния или событий (UI-данные, уведомления).

**Что такое StateFlow и SharedFlow?**
StateFlow — это горячий поток, который всегда хранит последнее значение и сразу выдаёт его новому коллектору (аналогично LiveData). Он имеет начальное значение, поддерживает distinctUntilChanged по умолчанию и предназначен для представления состояния (например, UI state: loading/data/error). SharedFlow — более общий горячий поток без обязательного начального значения, может иметь replay (количество последних значений для новых подписчиков) и используется для одноразовых событий (например, показ Snackbar, навигация, toast). StateFlow всегда имеет ровно одно актуальное значение, SharedFlow может не иметь значения или иметь несколько.

**Когда использовать StateFlow, а когда SharedFlow?**
StateFlow используется для представления состояния экрана или данных, которые всегда должны иметь актуальное значение (например, список пользователей, статус загрузки, выбранный элемент). Новый наблюдатель сразу получает текущее состояние без дополнительных запросов. SharedFlow применяется для событий и уведомлений, которые происходят один раз и не требуют хранения значения (показ сообщения, навигация, аналитика). Если нужно, чтобы новые подписчики получили предыдущие события — задаём replay > 0, иначе SharedFlow подходит для fire-and-forget событий.

**Что такое shareIn и stateIn?**
shareIn и stateIn — операторы, превращающие cold Flow в hot (shared) поток с управляемым жизненным циклом. shareIn(scope, replay) преобразует Flow в SharedFlow с заданным replay и автоматической остановкой при отмене scope. stateIn(scope, initialValue) превращает Flow в StateFlow с начальным значением и автоматической подпиской в scope. Оба оператора обеспечивают, что эмиссия начинается только при наличии коллекторов и останавливается при отмене scope, предотвращая утечки. В Android их используют для преобразования Room Flow или сетевых запросов в StateFlow для UI.

**Как правильно использовать repeatOnLifecycle с Flow?**
repeatOnLifecycle — расширение из lifecycle-runtime-ktx, позволяющее собирать Flow только в определённых состояниях жизненного цикла (например, STARTED или RESUMED). Синтаксис: lifecycleScope.launch { repeatOnLifecycle(Lifecycle.State.STARTED) { flow.collect { … } } }. Это автоматически отменяет сбор при уходе в STOPPED и возобновляет при возвращении, предотвращая лишнюю работу и утечки. В Android это стандартный способ подписки на Flow в Fragment/Activity, чтобы не собирать данные, когда экран невидим.

**Что такое combine, zip, flatMapConcat, flatMapLatest, debounce, distinctUntilChanged?**
combine объединяет несколько Flow, выдавая значение при изменении любого из них (например, combine(userFlow, settingsFlow)). zip ждёт значение от каждого Flow и выдаёт результат только при наличии всех. flatMapConcat последовательно обрабатывает внутренние Flow (ждёт завершения предыдущего). flatMapLatest отменяет предыдущий внутренний Flow при новом значении внешнего (актуально для поиска). debounce подавляет частые эмиссии, выдавая последнее значение через заданное время. distinctUntilChanged фильтрует последовательные одинаковые значения.

**Как тестировать Flow и StateFlow?**
Flow и StateFlow тестируются с помощью kotlinx-coroutines-test: runTest создаёт тестовый scope с виртуальным временем. Для Flow используют collect в тесте или библиотеку Turbine (flow.test { expectItem(), expectNoEvents() }). Для StateFlow проверяют начальное значение и последующие через value или collect. advanceUntilIdle() продвигает время до завершения всех задач, advanceTimeBy() симулирует задержку. Тесты пишутся в runTest с TestScope и StandardTestDispatcher.

**Как тестировать код с корутинами?**
Тестирование корутин проводится с помощью kotlinx-coroutines-test. Основной инструмент — runTest, который создаёт контролируемый TestScope и TestDispatcher. Внутри runTest можно использовать advanceUntilIdle() для завершения всех задач, advanceTimeBy() для симуляции времени, runCurrent() для выполнения отложенных задач. Для suspend-функций тестируют через runTest { mySuspendFun() }. Для отмены проверяют ensureActive() или исключения при отмене scope.

**Что такое runTest, TestScope, StandardTestDispatcher?**
runTest — тестовая функция-обёртка, создающая TestScope и StandardTestDispatcher для управления временем и корутинами. TestScope — тестовый scope, автоматически отменяющийся после теста. StandardTestDispatcher — виртуальный диспетчер, позволяющий контролировать выполнение задач через advanceUntilIdle(), advanceTimeBy(), runCurrent(). Это позволяет детерминированно тестировать задержки, таймауты и последовательность корутин.

**Как использовать Turbine для тестирования Flow?**
Turbine — библиотека для удобного тестирования Flow. Синтаксис: flow.test { awaitItem(), expectItem(expected), expectNoEvents(), awaitComplete() }. Позволяет проверять последовательность эмиссий, ошибки, завершение и отсутствие событий. Работает с runTest и TestScope, поддерживает expectMostRecentItem(), expectNoMoreEvents(). Идеально для тестирования StateFlow, SharedFlow и сложных цепочек операторов.

**Как тестировать StateFlow и SharedFlow?**
StateFlow тестируют через прямой доступ к value или collect в runTest: проверяют начальное значение и последующие эмиссии. SharedFlow тестируют через collect с Turbine или прямой collect с expectItem(). Для обоих используют advanceUntilIdle() после эмиссий, чтобы дождаться обработки. В ViewModel-тестах проверяют, что StateFlow обновляется при вызове методов и сохраняет последнее значение.

**Что такое advanceUntilIdle() и advanceTimeBy()?**
advanceUntilIdle() продвигает виртуальное время до выполнения всех отложенных задач в TestDispatcher, имитируя завершение всех текущих корутин. advanceTimeBy(millis) продвигает время на указанное количество миллисекунд, позволяя протестировать delay(), withTimeout и debounce. В тестах это ключевые методы для управления временем и проверки поведения корутин с задержками.

**Coroutines vs RxJava — что лучше в 2025 году и почему?**
В 2025 году Kotlin Coroutines являются предпочтительным выбором для большинства Android-приложений по сравнению с RxJava. Coroutines предлагают более естественный, последовательный стиль написания асинхронного кода благодаря suspend-функциям, что значительно снижает сложность по сравнению с цепочками операторов RxJava. Они имеют встроенную поддержку Structured Concurrency, автоматическую отмену при уничтожении экрана (viewModelScope, lifecycleScope), меньший overhead и лучшую интеграцию с Jetpack-библиотеками (Room, Retrofit, Paging, DataStore). RxJava остаётся актуальной только в legacy-проектах или в случаях, когда требуется сложная обработка потоков с backpressure (например, в высоконагруженных системах с потоками данных). Coroutines проще тестировать (runTest + Turbine), имеют меньший размер APK и лучше поддерживаются сообществом Google.

**Coroutines vs Threads — когда всё-таки лучше использовать обычные Thread?**
Обычные Thread следует использовать крайне редко в современных Android-приложениях, только в случаях, когда требуется низкоуровневый контроль над потоками ОС или интеграция с legacy-кодом, использующим Thread напрямую (например, старые библиотеки с собственным пулами потоков). Coroutines значительно легче, дешевле по памяти, поддерживают Structured Concurrency и автоматическую отмену, а также позволяют писать асинхронный код в синхронном стиле. Thread не имеют встроенной отмены, сложнее отлаживать и могут привести к утечкам, если забыть остановить. В 2025 году единственные обоснованные сценарии для Thread — это высокопроизводительные нативные вызовы (JNI) или специальные задачи, где Coroutines по каким-то причинам не подходят.

**AsyncTask vs Coroutines vs WorkManager — когда что использовать?**
AsyncTask полностью устарел и не рекомендуется с 2018 года (deprecated в Android 11+): он не поддерживает отмену, легко приводит к утечкам и не учитывает жизненный цикл. Coroutines — основной инструмент для асинхронных задач внутри приложения (сетевые запросы, работа с БД, вычисления), особенно с viewModelScope и lifecycleScope — они автоматически отменяются при уничтожении экрана и хорошо интегрируются с Jetpack. WorkManager предназначен для отложенных, гарантированных фоновых задач, которые должны выполниться даже после перезапуска приложения (например, синхронизация данных, загрузка файлов, отправка аналитики), поддерживает ограничения (сеть, заряд батареи) и перезапуск при сбоях. Простое правило: Coroutines — для задач внутри экрана, WorkManager — для задач вне экрана.

**Как реализовать параллельную загрузку нескольких изображений?**
Параллельная загрузка изображений в Kotlin Coroutines реализуется через async внутри coroutineScope или supervisorScope с последующим awaitAll. Пример: supervisorScope { val deferreds = urls.map { url -> async(Dispatchers.IO) { loadImage(url) } }; val images = deferreds.awaitAll() }. supervisorScope обеспечивает, что сбой загрузки одного изображения не отменяет остальные. Для обновления UI можно использовать withContext(Dispatchers.Main) или StateFlow. Это эффективнее последовательной загрузки, использует пул потоков IO и легко отменяется при уничтожении экрана.

**Как правильно использовать Mutex и Semaphore в Coroutines?**
Mutex — это корутинный аналог ReentrantLock, используется для защиты критической секции в корутинах: mutex.withLock { критический код }. Поддерживает отмену и не блокирует поток. Semaphore ограничивает количество одновременно выполняемых задач (например, 3 параллельных запроса): semaphore.withPermit { ... }. В Android Mutex часто применяют для защиты общей переменной или очереди в ViewModel, Semaphore — для ограничения параллельных сетевых запросов. Важно: использовать их только внутри suspend-функций, избегать вложенных блокировок и всегда учитывать Structured Concurrency.

**Что такое Channel и когда его использовать вместо Flow?**
Channel — это потокобезопасная очередь сообщений между корутинами (аналог BlockingQueue, но для suspend). Поддерживает send/receive, capacity, onBufferOverflow (SUSPEND, DROP_OLDEST, DROP_LATEST). Channel используется, когда нужно передавать события между корутинами в стиле producer-consumer без хранения состояния (например, очередь задач, события от сервиса). Flow подходит для потоков данных с операторами и реактивной обработкой. Channel предпочтительнее, когда требуется точный контроль над очередью и backpressure (например, send не должен блокировать, если буфер полон).

**Разница между produce, actor и обычными корутинами?**
produce создаёт потокобезопасный producer-канал, который отправляет элементы в Channel и автоматически закрывает его при завершении. actor — это корутина, которая обрабатывает сообщения из Channel последовательно, гарантируя thread-safety без дополнительных блокировок (аналог actor-модели). Обычные корутины (launch, async) не имеют встроенного канала и не гарантируют последовательной обработки сообщений. Actor и produce полезны для высоконагруженных сценариев с большим количеством событий, где нужна строгая последовательность.

**Как сделать thread-safe singleton в Kotlin?**
Самый простой и рекомендуемый способ — использовать object (Kotlin singleton): object MySingleton { ... }. Он инициализируется лениво, потокобезопасен и гарантирует единственный экземпляр. Альтернатива — val instance by lazy(LazyThreadSafetyMode.SYNCHRONIZED) { MyClass() } или double-checked locking с volatile в Java-стиле. В Android object — стандарт для App-level синглтонов (например, аналитика, логгер). Избегайте статических полей с mutable состоянием.

**Как работать с SharedPreferences в многопоточной среде?**
SharedPreferences в Android потокобезопасны для чтения, но запись через apply() асинхронна, а commit() синхронна и блокирует поток. При параллельной записи из нескольких потоков возможны race condition и потеря данных. Рекомендуется использовать edit().put…().apply() в одном потоке (например, через Dispatchers.IO в корутине) или Mutex для защиты секции редактирования. Для частых операций лучше перейти на DataStore (Preferences DataStore), который изначально потокобезопасен и поддерживает корутины.

**Как безопасно обновлять UI из нескольких корутин?**
Обновление UI должно происходить только на Dispatchers.Main. Используйте withContext(Dispatchers.Main) или launch(Dispatchers.Main) внутри ViewModel. Лучшая практика — собирать данные в StateFlow или LiveData в ViewModel, а UI (Activity/Fragment) наблюдает за ним через collectAsState() или observe(). Это гарантирует, что все обновления происходят в правильном потоке и учитывают жизненный цикл. Избегайте прямых вызовов runOnUiThread — Coroutines + StateFlow делают это автоматически и безопасно.

**Как отлаживать зависшие корутины и deadlocks?**
Для отладки зависших корутин используют Android Studio Profiler (CPU → Threads) для просмотра стека потоков, Thread Dump (через ADB или Profiler) для анализа блокировок и kotlinx-coroutines-debug для вывода состояния всех корутин (Coroutine Debugger). Deadlock обнаруживают через Thread Dump (ищут потоки в состоянии WAITING/BLOCKED на одном мониторе) или Profiler. Для корутин — логирование Job состояния, использование CoroutineExceptionHandler и проверка isActive/ensureActive(). В продакшене добавляют Crashlytics/ Sentry с Thread Dump.

# SQL

**Что такое реляционная база данных?**
Реляционная база данных — это система управления базами данных (СУБД), которая организует данные в виде таблиц (отношений), состоящих из строк (записей) и столбцов (атрибутов). Каждая таблица имеет строго определённую структуру, а связь между таблицами устанавливается через ключи (обычно первичный и внешний). Основная идея реляционной модели, предложенной Эдгаром Коддом в 1970 году, заключается в том, что данные хранятся без дублирования и избыточности благодаря нормализации, а операции над данными выполняются с помощью декларативного языка SQL. В Android реляционные базы данных представлены SQLite (встроенная СУБД) и Room (абстракция над SQLite), которые позволяют эффективно хранить структурированные данные приложения: списки пользователей, заметки, кэш сетевых ответов и т.д.

**Что такое SQL и чем он отличается от NoSQL?**
SQL (Structured Query Language) — это декларативный язык запросов, предназначенный для работы с реляционными базами данных. Он позволяет описывать, какие данные нужны и как их обработать (SELECT, INSERT, UPDATE, DELETE, JOIN и т.д.), а СУБД сама определяет оптимальный способ выполнения. SQL-системы (SQLite, PostgreSQL, MySQL) строго придерживаются реляционной модели, схемы данных фиксированы заранее, обеспечивают ACID-свойства транзакций и идеально подходят для структурированных данных с предсказуемыми отношениями. NoSQL (Not Only SQL) — класс баз данных, которые отказываются от строгой реляционной модели в пользу гибкости: документоориентированные (MongoDB, Firestore), ключ-значение (Redis), колоночные (Cassandra), графовые (Neo4j). NoSQL лучше справляется с большими объёмами неструктурированных данных, горизонтальным масштабированием и высокой скоростью записи, но часто жертвуют строгой консистентностью (BASE вместо ACID). В Android SQL представлен Room/SQLite для локальных данных, NoSQL — Firebase Firestore/Realtime Database для облачных сценариев.

**Отличие в системах БД**
Хотя SQLite, PostgreSQL, MySQL, MariaDB, SQL Server, Oracle и другие реляционные базы данных действительно используют один и тот же стандартный язык SQL (с небольшими вариациями), они отличаются очень существенно по архитектуре, возможностям, производительности, сценариям применения и ограничениям.
–
SQLite — встраиваемая (embedded) база данных. Работает как обычная библиотека внутри приложения, не требует отдельного серверного процесса. Файл базы данных — это один файл на диске (например, app.db).
PostgreSQL, MySQL, MariaDB, SQL Server, Oracle — клиент-серверные СУБД. Требуют отдельного запущенного сервера (демона/службы), к которому подключаются клиенты по сети (TCP/IP) или локально.
–
SQLite — очень быстрая на чтение и запись при низкой конкурентности. Идеальна для мобильных приложений (Android, iOS), десктопных программ, встроенных устройств, тестов, локального кэша. Размер базы ограничен ~281 терабайтами (практически не достигается).
PostgreSQL — считается одной из самых быстрых и надёжных open-source СУБД для сложных запросов, аналитики, GIS, JSONB, полнотекстового поиска.
MySQL / MariaDB — очень быстрая на простых операциях чтения/записи, особенно при высокой нагрузке на чтение (репликация master-slave). Часто выбирается для веб-приложений (WordPress, Laravel и т.д.).
Oracle / SQL Server — ориентированы на корпоративные системы с огромными объёмами данных, сложной аналитикой и высокой доступностью.

**Что такое ACID и BASE?**
**ACID**-свойства транзакций — это классическая модель, описывающая требования к надёжным транзакциям в реляционных базах данных. Аббревиатура расшифровывается следующим образом:
*Atomicity (атомарность*) — транзакция выполняется полностью или не выполняется вовсе. Если хотя бы одна операция внутри транзакции завершилась неудачно, все изменения откатываются (rollback), и база данных возвращается в состояние до начала транзакции.
*Consistency (согласованность)* — после завершения транзакции база данных должна находиться в корректном, согласованном состоянии. Это означает, что все ограничения целостности (PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE и т.д.) должны быть соблюдены.

*Isolation (изоляция)* — транзакции, выполняющиеся параллельно, не должны видеть промежуточных или непоследовательных изменений друг друга. Существуют разные уровни изоляции (Read Uncommitted, Read Committed, Repeatable Read, Serializable), причём более строгие уровни снижают производительность, но повышают надёжность.
*Durability (долговечность)* — после успешного завершения транзакции (commit) все изменения сохраняются даже в случае сбоя системы (например, отключения питания). Это достигается за счёт записи в журнал транзакций (WAL — Write-Ahead Logging) перед подтверждением.
ACID — это основа надёжности реляционных баз данных (SQLite, PostgreSQL, MySQL/InnoDB и т.д.). В Android Room и SQLite полностью поддерживают ACID, что делает их подходящими для приложений, где важна целостность данных (например, финансы, заметки, корзина покупок).
–
**BASE** — это альтернативная модель, характерная для многих NoSQL-систем, особенно распределённых и высоконагруженных. Аббревиатура расшифровывается как:
*Basically Available (базовая доступность)* — система всегда отвечает, даже если не все данные актуальны.
*Soft state (мягкое состояние)* — состояние системы может изменяться со временем даже без внешних действий (из-за репликации и eventual consistency).
*Eventual consistency (конечная согласованность)* — со временем все реплики данных придут в согласованное состояние, но в любой момент они могут отличаться.
–
BASE — это компромисс ради высокой доступности и масштабируемости в распределённых системах (например, Cassandra, DynamoDB, MongoDB, Firebase Realtime Database/Firestore). В Android BASE-подход чаще встречается в облачных решениях (Firestore), где данные синхронизируются между устройствами, но могут временно расходиться (например, один пользователь видит старую версию заметки, пока синхронизация не завершится).
–
Основное отличие ACID vs BASE:
ACID фокусируется на строгой согласованности и надёжности (жертвуя масштабируемостью и доступностью), BASE — на высокой доступности и масштабируемости (жертвуя моментальной согласованностью). В Android-приложениях ACID (Room/SQLite) используется для локальных критически важных данных, а BASE (Firestore) — для сценариев синхронизации и оффлайн-поддержки с eventual consistency.

**Что такое PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL?**
PRIMARY KEY — это столбец (или комбинация столбцов), который однозначно идентифицирует каждую запись в таблице; он не может содержать NULL и всегда уникален. FOREIGN KEY — столбец (или набор столбцов), который ссылается на PRIMARY KEY другой таблицы, обеспечивая ссылочную целостность (referential integrity). UNIQUE — ограничение, гарантирующее уникальность значений в столбце или группе столбцов (может быть несколько UNIQUE в таблице, допускает NULL). NOT NULL — ограничение, запрещающее вставку NULL-значений в столбец. В Android (Room/SQLite) эти ограничения задаются аннотациями @PrimaryKey, @ForeignKey, @ColumnInfo(unique = true), nullable = false и обеспечивают целостность данных на уровне базы.

**Что такое нормализация? Какие формы нормализации вы знаете?**
Нормализация — процесс организации данных в таблицах, направленный на устранение избыточности и аномалий при вставке/обновлении/удалении. Основная цель — минимизировать дублирование данных и обеспечить логическую целостность. Первая нормальная форма (1NF) требует атомарности значений и отсутствия повторяющихся групп. Вторая (2NF) — отсутствие частичной зависимости неключевых атрибутов от составного ключа. Третья (3NF) — отсутствие транзитивных зависимостей (неключевые атрибуты зависят только от первичного ключа). Форма Бойса-Кодда (BCNF) — более строгая версия 3NF. В Android-разработке обычно достаточно 3NF: избыточная нормализация может ухудшить производительность запросов, поэтому иногда допускают контролируемую денормализацию (например, хранение имени пользователя вместе с ID в таблице сообщений).

**Разница между INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN?**
INNER JOIN возвращает только те строки, для которых есть совпадение в обеих таблицах (пересечение). LEFT JOIN (LEFT OUTER JOIN) возвращает все строки из левой таблицы и соответствующие строки из правой; если совпадений нет — в правой части NULL. RIGHT JOIN — наоборот: все строки из правой таблицы и соответствующие из левой. FULL JOIN (FULL OUTER JOIN) возвращает все строки из обеих таблиц, дополняя NULL там, где нет совпадений. В Android (Room/SQLite) INNER JOIN — самый частый (например, пользователь + его посты), LEFT JOIN — когда нужно показать пользователей даже без постов.

**Что такое GROUP BY и HAVING?**
GROUP BY группирует строки по указанным столбцам, позволяя применять агрегатные функции (COUNT, SUM, AVG, MAX, MIN) к каждой группе. Например, SELECT user_id, COUNT(*) FROM posts GROUP BY user_id покажет количество постов по каждому пользователю. HAVING применяется после группировки и фильтрует группы по условию (аналог WHERE, но для агрегатов). Например, HAVING COUNT(*) > 5 оставит только пользователей с более чем 5 постами. В Android GROUP BY часто используется для подсчёта статистики (количество сообщений в чате, средний рейтинг).

**Что такое индекс? Какие виды индексов бывают?**
Индекс — структура данных (обычно B-дерево), ускоряющая поиск, сортировку и объединение таблиц за счёт быстрого доступа к строкам по значению столбца. Виды: уникальный (UNIQUE) — запрещает дубликаты, составной (composite) — по нескольким столбцам, кластерный (clustered) — определяет физический порядок строк (обычно PRIMARY KEY), некластерный (non-clustered) — отдельная структура. В SQLite индексы создаются автоматически для PRIMARY KEY и FOREIGN KEY. В Room — через @Index в @Entity.

**Когда нужно создавать индекс, а когда он может навредить?**
Индекс нужен на столбцах, по которым часто выполняются WHERE, JOIN, ORDER BY, GROUP BY или поиск (например, user_id, timestamp, поиск по названию). Он ускоряет чтение, но замедляет INSERT, UPDATE, DELETE (требуется обновление индекса) и увеличивает размер базы. В Android вредно создавать индексы на столбцах с низкой селективностью (например, столбец boolean) или на таблицах с частыми вставками/обновлениями (логи). Правило: индексировать только то, что реально используется в запросах и где выигрыш в скорости чтения превышает overhead записи.

**Что такое транзакция? Что такое ACID?**
Транзакция — неделимая последовательность операций с базой данных, которая либо полностью выполняется, либо полностью отменяется. ACID — свойства транзакции: Atomicity (атомарность — всё или ничего), Consistency (согласованность — база остаётся в корректном состоянии), Isolation (изоляция — транзакции не видят промежуточных изменений друг друга), Durability (долговечность — изменения сохраняются после коммита даже при сбое). В SQLite транзакции поддерживаются через BEGIN TRANSACTION / COMMIT / ROLLBACK. В Room — через @Transaction.

**Что такое COMMIT, ROLLBACK, SAVEPOINT?**
COMMIT фиксирует все изменения текущей транзакции, делая их постоянными и видимыми для других транзакций. ROLLBACK отменяет все изменения с начала транзакции (или с SAVEPOINT). SAVEPOINT создаёт промежуточную точку сохранения внутри транзакции, к которой можно откатиться частично (ROLLBACK TO SAVEPOINT). В Android это используется в Room для сложных операций (например, вставка пользователя + его постов: если пост не вставился — откат к savepoint, пользователь остаётся).

**Что такое SQLite и почему Android использует именно её?**
SQLite — это лёгкая, встраиваемая (embedded) реляционная база данных, которая не требует отдельного серверного процесса и хранится в виде одного файла на устройстве. Она полностью самодостаточна, поддерживает большинство возможностей SQL (транзакции, индексы, триггеры, представления), но имеет ограниченную конкурентность (одна запись одновременно). Android использует SQLite по нескольким причинам: минимальный размер (около 2 МБ), отсутствие необходимости в серверной инфраструктуре, высокая надёжность (ACID-транзакции), хорошая производительность на мобильных устройствах и лицензия public domain. Это позволяет каждому приложению иметь собственную локальную базу данных без дополнительных зависимостей, что идеально для хранения пользовательских данных (списки, настройки, кэш) в условиях ограниченных ресурсов и отсутствия постоянного сетевого соединения.

**Как создать SQLite базу данных в Android (классический способ)?**
Классический способ создания базы данных в Android — использование класса SQLiteOpenHelper. Нужно создать класс, наследующий SQLiteOpenHelper, переопределить конструктор (с именем базы, версией и фабрикой курсоров) и два метода: onCreate() для создания таблиц при первом запуске и onUpgrade() для миграций при изменении версии. В onCreate() выполняется SQL-запрос CREATE TABLE с нужными столбцами и ограничениями. При первом обращении к базе через getWritableDatabase() или getReadableDatabase() система автоматически вызывает onCreate(), если базы ещё нет. Этот подход обеспечивает контроль над схемой и версиями, а также автоматическое управление открытием и закрытием соединения.

**Что такое SQLiteOpenHelper и какие методы в нём важны?**
SQLiteOpenHelper — абстрактный класс-помощник для работы с SQLite в Android, который управляет созданием, открытием, обновлением и версионированием базы данных. Основные методы: конструктор (принимает контекст, имя базы, фабрику курсоров и версию), onCreate() — вызывается при первом создании базы для инициализации таблиц, onUpgrade() — вызывается при увеличении версии для миграций (DROP TABLE, ALTER TABLE или перенос данных), onDowngrade() — для понижения версии (редко используется), getWritableDatabase() и getReadableDatabase() — возвращают объект SQLiteDatabase для операций записи/чтения. Дополнительно есть onOpen() для дополнительных настроек после открытия базы.

**Что такое Cursor и как правильно с ним работать?**
Cursor — это указатель на результат SQL-запроса в SQLite, представляющий набор строк с возможностью итеративного чтения данных. После выполнения query() или rawQuery() возвращается объект Cursor, который позволяет перемещаться по строкам (moveToFirst(), moveToNext(), moveToPosition()) и извлекать значения по имени столбца или индексу (getString(), getInt(), getLong() и т.д.). Правильная работа включает: проверку cursor != null и cursor.moveToFirst(), использование в try-with-resources (или явное закрытие), избегание утечек памяти (всегда закрывать cursor.close() после использования). В Android Cursor часто используется с Loader или напрямую в Repository слое.

**Как правильно закрывать Cursor и Database?**
Cursor и SQLiteDatabase необходимо закрывать вручную, чтобы избежать утечек памяти и файловых дескрипторов. Лучшая практика — использовать try-with-resources (с Java 7+ и Kotlin): try (Cursor cursor = db.query(...)) { ... }. Если try-with-resources недоступен, закрывать в finally-блоке: cursor.close(); db.close(). SQLiteOpenHelper автоматически управляет открытием/закрытием базы при вызове getReadable/WritableDatabase(), но сам Cursor всегда нужно закрывать явно. В Room закрытие происходит автоматически благодаря абстракции, но при ручной работе с SQLiteOpenHelper это обязательный шаг.

**Что такое rawQuery и execSQL?**
rawQuery(String sql, String[] selectionArgs) — метод для выполнения произвольного SELECT-запроса и получения результата в виде Cursor. Поддерживает параметризованные запросы для защиты от SQL-инъекций (знаки ? заменяются значениями из selectionArgs). execSQL(String sql, Object[] bindArgs) — метод для выполнения запросов, не возвращающих данные (INSERT, UPDATE, DELETE, CREATE TABLE и т.д.), также с параметризацией. rawQuery используется для чтения, execSQL — для модификации схемы или данных. В Room эти методы почти не используются — вместо них аннотации @Query, @Insert и т.д.

**Чем отличается insert(), update(), delete(), query() от rawQuery?**
Методы insert(), update(), delete(), query() — это высокоуровневые API SQLiteDatabase, которые принимают структурированные параметры (таблица, ContentValues, WHERE-условие) и автоматически экранируют данные, предотвращая SQL-инъекции. Они возвращают количество затронутых строк или Cursor (для query). rawQuery и execSQL позволяют писать произвольный SQL, что даёт максимальную гибкость, но требует ручной экранировки и повышает риск ошибок. В современном Android рекомендуется использовать высокоуровневые методы или Room, оставляя raw-запросы только для сложных случаев, не поддерживаемых API.

**Как правильно использовать ContentValues?**
ContentValues — это класс-обёртка над Map<String, Object>, используемый для передачи значений столбцов в методы insert(), update() и т.д. Правильное использование: создать объект ContentValues, вызвать put(String key, value) для каждого столбца (поддерживает разные типы: String, Integer, Long, Double, byte[] и null). Важно: не использовать null для отсутствующих значений — это означает "установить NULL", для пропуска столбца просто не вызывать put(). В Android ContentValues часто используется в ContentProvider и SQLiteOpenHelper для безопасной вставки/обновления данных без ручного SQL.

**Что такое SQLiteDatabase.CONFLICT_REPLACE и другие конфликты?**
SQLiteDatabase предоставляет константы для стратегий разрешения конфликтов при вставке/обновлении: CONFLICT_ABORT (по умолчанию — откат транзакции), CONFLICT_FAIL (откат только текущей операции), CONFLICT_IGNORE (игнорировать вставку), CONFLICT_REPLACE (заменить существующую строку), CONFLICT_ROLLBACK (полный откат). Эти значения передаются в методы insertWithOnConflict(), updateWithOnConflict() и replace(). CONFLICT_REPLACE чаще всего используется при upsert-сценариях, когда нужно обновить запись или вставить новую при конфликте первичного ключа.

**Как оптимизировать запросы в SQLite для мобильных устройств?**
Оптимизация запросов в SQLite для Android включает: создание индексов на часто используемых в WHERE/JOIN/ORDER BY столбцах, использование LIMIT и OFFSET для пагинации, минимизация возвращаемых столбцов (SELECT name вместо SELECT *), применение подготовленных запросов (query() вместо rawQuery со строковой конкатенацией), использование транзакций для пакетных операций, настройка PRAGMA (journal_mode=WAL, synchronous=NORMAL, cache_size), избегание сложных подзапросов. В Room это достигается правильными @Index, @Query с LIMIT, использованием Paging 3 и профилированием через Database Inspector.

**Работа индексов на примере**

Индекс в базе данных — это специальная структура данных, которая создаётся поверх таблицы и значительно ускоряет поиск, сортировку и соединение таблиц (JOIN). Без индекса при выполнении запроса WHERE id = 5 база данных вынуждена просматривать все строки таблицы по очереди (полное сканирование таблицы — full table scan). С индексом база данных может найти нужную строку очень быстро, обычно за логарифмическое время.

–

Большинство современных баз данных (включая SQLite, PostgreSQL, MySQL) используют для индексов структуру B-дерево (или B+ дерево).

B-дерево — это сбалансированное дерево, где каждый узел может содержать множество ключей и указателей на дочерние узлы.

Ключи в индексе отсортированы.

Это позволяет находить нужную запись не перебирая все строки, а "спускаясь" по дереву (как в телефонном справочнике).

Когда вы создаёте индекс на столбце user_id, база данных строит отдельную структуру, где хранятся значения user_id + ссылка (rowid или указатель) на полную строку в таблице.

–

Предположим таблицу пользователей:

```
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    username TEXT,
    email TEXT,
    created_at INTEGER
);
```

Создаём индекс:

```
CREATE INDEX idx_username ON users(username);
```

Теперь тот же запрос (*SELECT * FROM users WHERE username = 'alex';*) использует индекс:

База данных смотрит в индекс idx_username (который отсортирован).

Быстро находит нужное значение alex.

По указателю сразу переходит к полной строке в таблице.

–

**Важные нюансы**

Плюсы индекса: ускоряет WHERE, ORDER BY, GROUP BY, JOIN.

Минусы индекса: замедляет операции INSERT, UPDATE, DELETE (нужно обновлять индекс), занимает дополнительную память на диске.

Составной индекс (CREATE INDEX idx_name_age ON users(name, age)) ускоряет запросы, где используются оба столбца одновременно.

**Что такое Room и почему Google рекомендует его вместо прямой работы с SQLite?**
Room — это официальная библиотека от Google, представляющая собой высокоуровневую абстракцию над SQLite, которая упрощает работу с базой данных в Android-приложениях. Room предоставляет декларативный подход через аннотации (@Entity, @Dao, @Query), автоматически генерирует boilerplate-код, обеспечивает compile-time проверку SQL-запросов, поддерживает корутины, Flow, LiveData и Paging из коробки. Google рекомендует Room вместо прямой работы с SQLite, потому что это снижает вероятность ошибок (например, SQL-инъекций или неправильных типов данных), упрощает миграции, повышает читаемость кода, улучшает тестируемость и интегрируется с современным стеком Jetpack. Прямой доступ к SQLite через SQLiteOpenHelper требует ручного написания SQL, управления курсорами и обработки ошибок, что увеличивает объём кода и вероятность утечек памяти.

**Из каких основных компонентов состоит Room (Entity, DAO, Database)?**
Room состоит из трёх основных компонентов. Entity — это класс, аннотированный @Entity, представляющий таблицу в базе данных; поля класса соответствуют столбцам таблицы. DAO (Data Access Object) — интерфейс или абстрактный класс с аннотациями @Dao, содержащий методы для операций с данными (@Query, @Insert, @Update, @Delete и т.д.). Database — абстрактный класс, аннотированный @Database, который определяет сущности (entities), версию базы и DAO-интерфейсы, а также предоставляет методы для получения экземпляров DAO. Эти компоненты вместе образуют чистую и типобезопасную абстракцию над SQLite.

**Что такое @Entity, @PrimaryKey, @ColumnInfo, @Ignore?**
@Entity — аннотация, помечающая класс как таблицу в базе данных; параметры позволяют указать имя таблицы, индексы и внешние ключи. @PrimaryKey — аннотация для поля, которое будет первичным ключом (автоинкремент или вручную). @ColumnInfo — задаёт имя столбца, тип, default-значение и другие свойства столбца (например, @ColumnInfo(name = "user_name")). @Ignore — исключает поле из маппинга в таблицу (полезно для вычисляемых или временных полей, которые не нужно сохранять в базе).

**Что такое @Embedded и @Relation?**
@Embedded позволяет вложить объект одного класса в другой как часть таблицы (flat mapping): поля вложенного объекта сохраняются как столбцы основной таблицы. @Relation используется для описания отношений один-ко-многим или многие-ко-многим при выборке связанных сущностей в одном запросе (например, пользователь и его посты). @Relation работает только с @Query-методами DAO и требует правильной настройки внешних ключей.

**Как работает @ForeignKey в Room?**
@ForeignKey задаётся в аннотации @Entity и указывает на связь с другой таблицей: entity — класс родительской сущности, parentColumns — столбцы родителя, childColumns — столбцы ребёнка, onDelete и onUpdate — действия при удалении/обновлении родителя (CASCADE, SET_NULL, RESTRICT и т.д.). Room проверяет целостность на уровне компиляции и генерирует соответствующие SQL-ограничения в базе данных.

**Что такое DAO и какие аннотации в нём используются?**
DAO (Data Access Object) — интерфейс или абстрактный класс, аннотированный @Dao, содержащий методы для операций с базой данных. Основные аннотации: @Query для произвольных SQL-запросов, @Insert, @Update, @Delete для стандартных операций, @Transaction для группировки операций в транзакцию, @RawQuery для низкоуровневых запросов. DAO — центральное место для всей бизнес-логики работы с данными.

**Разница между @Query, @Insert, @Update, @Delete, @Transaction?**
@Query — произвольный SQL-запрос (SELECT, сложные JOIN), может возвращать сущности, примитивы, Flow, LiveData. @Insert — вставка сущностей (одной или списка), поддерживает onConflict стратегии. @Update — обновление сущностей по первичному ключу. @Delete — удаление сущностей. @Transaction — группирует несколько операций в одну транзакцию, обеспечивая атомарность и откат при ошибке; обязательно использовать для операций, затрагивающих несколько таблиц.

**Как Room поддерживает возвращение LiveData, Flow и StateFlow?**
Room автоматически генерирует observable запросы: если метод DAO возвращает LiveData<T>, Flow<T> или StateFlow<T>, Room создаёт внутренний наблюдатель и обновляет значение при изменении данных в таблице. LiveData устаревает в пользу Flow/StateFlow. StateFlow обеспечивает единственное актуальное значение, SharedFlow — события. Поддержка встроена через аннотацию @Query и генерацию кода на этапе компиляции.

**Что такое TypeConverter и когда он нужен?**
TypeConverter — класс с методами @TypeConverter, который учит Room преобразовывать сложные типы (List, Date, Enum, кастомные классы) в примитивы SQLite (String, Long и т.д.) и обратно. Нужен, когда в Entity используется тип, который SQLite не понимает напрямую (например, List<String> → JSON-строка). Room вызывает конвертер автоматически при чтении/записи.

**Как сделать составной первичный ключ в Room?**
Составной первичный ключ задаётся через @Entity(primaryKeys = {"column1", "column2"}). Поля column1 и column2 вместе уникально идентифицируют запись. Это полезно для связующих таблиц в отношениях многие-ко-многим.

**Что такое Room Database Inspector и как им пользоваться?**
Room Database Inspector — инструмент в Android Studio, позволяющий просматривать и редактировать содержимое Room-базы данных в реальном времени на устройстве или эмуляторе. Запускается в разделе App Inspection → Database Inspector после подключения устройства. Позволяет выполнять запросы, просматривать таблицы, редактировать данные и проверять схему. Очень полезен для отладки миграций и проверки содержимого базы.

**Как правильно настраивать RoomDatabase?**
RoomDatabase.Builder позволяет настроить поведение: fallbackToDestructiveMigration() — при ошибке миграции удаляет и пересоздаёт базу (опасно для продакшена). allowMainThreadQueries() — разрешает запросы на Main Thread (только для тестов, в продакшене запрещено). Рекомендуется использовать addMigrations() для явных миграций и addCallback() для инициализации данных.

**Что такое @Database(entities = [...], version = ...)?**
@Database — аннотация на абстрактном классе, наследующем RoomDatabase. entities — массив классов @Entity, version — номер версии базы. При изменении схемы версия увеличивается, и Room вызывает onUpgrade или миграцию. Также можно указать exportSchema = true для генерации схемы.

**Как делать миграции в Room (Migration класс)?**
Миграция создаётся как объект Migration(startVersion, endVersion) с методом migrate(SupportSQLiteDatabase). В migrate() пишутся SQL-команды ALTER TABLE, CREATE TABLE, INSERT для переноса данных. Затем миграция добавляется в Room.databaseBuilder().addMigrations(migration1, migration2). Это позволяет безопасно обновлять схему без потери данных.

**Как сделать авто-миграцию в Room 2.6+?**
В Room 2.6+ появилась поддержка автоматических миграций через @AutoMigration(from = ..., to = ...). Нужно создать класс с аннотацией @AutoMigrationSpec и методами, описывающими изменения схемы. Room генерирует SQL автоматически на основе diff между версиями. Это упрощает миграции, но требует осторожности при сложных изменениях.

**Можно ли использовать Room в многопоточном режиме?**
Да, Room поддерживает многопоточность. По умолчанию запросы на Main Thread запрещены, но можно разрешить allowMainThreadQueries() (не рекомендуется). Лучше использовать Coroutines (suspend DAO-методы), Flow/StateFlow или LiveData для асинхронного доступа. Room использует один экземпляр SQLiteDatabase с внутренним пулом соединений, что безопасно для параллельного чтения и записи в WAL-режиме.

**Как тестировать Room (RoomDatabase.inMemoryDatabaseBuilder)?**
Для тестирования Room создают in-memory базу: Room.inMemoryDatabaseBuilder(context).build(). Это создаёт временную базу в памяти, которая уничтожается после теста. Используют @RunWith(AndroidJUnit4::class), @Before создают базу, @After закрывают. Тестируют DAO-методы, миграции и поведение Flow/LiveData.

**Как правильно использовать Room с Coroutines и Flow?**
DAO-методы помечаются suspend для асинхронных операций (@Insert, @Update, @Delete). Для чтения возвращают Flow<T> или StateFlow<T> — Room автоматически обновляет поток при изменении данных. Вызовы выполняются в viewModelScope или lifecycleScope с Dispatchers.IO. Это обеспечивает реактивность и автоматическую отмену.

**Что такое @Transaction в DAO и когда его обязательно использовать?**
@Transaction — аннотация, которая группирует несколько операций в одну транзакцию. Обязательна при чтении связанных сущностей (с @Relation) или при сложных операциях (вставка пользователя + его постов). Без @Transaction Room может вернуть неконсистентные данные при параллельных изменениях.

**Как Room работает под капотом (Room vs SQLite)?**
Room — это compile-time генератор кода над SQLite. На этапе компиляции Room проверяет SQL-запросы (@Query), генерирует реализацию DAO и преобразует аннотации в SQLite-вызовы. Под капотом Room использует SQLiteOpenHelper для создания и управления базой, но скрывает низкоуровневые детали (Cursor, ContentValues, транзакции). Room добавляет типобезопасность, поддержку Flow/LiveData и миграции, делая работу с SQLite проще и безопаснее.

**Что такое NoSQL и когда его стоит использовать в Android?**
NoSQL (Not Only SQL) — класс баз данных, которые отказываются от строгой реляционной модели и табличной структуры в пользу более гибких форматов: документоориентированные (JSON-подобные документы), ключ-значение, колоночные или графовые. NoSQL базы обычно жертвуют строгой согласованностью (ACID) ради высокой доступности, горизонтальной масштабируемости и скорости работы с большими объёмами неструктурированных или полуструктурированных данных. В Android NoSQL стоит использовать, когда важна синхронизация данных между устройствами в реальном времени, оффлайн-поддержка с автоматическим разрешением конфликтов или работа с динамическими структурами (например, чаты, пользовательские настройки с произвольными полями). Типичные сценарии — Firebase Firestore/Realtime Database для облачных приложений, Realm для локального хранения с высокой скоростью записи или MongoDB Realm (теперь Atlas Device Sync) для гибридных решений.

**Что такое Realm и чем он отличается от Room?**
Realm — это объектно-ориентированная мобильная база данных, которая хранит данные как живые объекты напрямую в памяти приложения, без необходимости маппинга в таблицы и строки, как в реляционных базах. В отличие от Room, который является абстракцией над SQLite и требует явного описания схемы через @Entity и SQL-подобные запросы, Realm использует собственный движок и позволяет работать с данными как с обычными Kotlin/Java-объектами (RealmObject или RealmModel). Realm автоматически синхронизирует изменения между объектами в памяти и файлом на диске, поддерживает живые запросы (live queries) и встроенную синхронизацию с облаком (Realm Sync). Room работает с Cursor и требует ручного управления транзакциями, тогда как Realm обеспечивает более декларативный и объектный подход.

**Преимущества и недостатки Realm по сравнению с Room?**
Преимущества Realm: очень высокая скорость чтения и записи (особенно для сложных объектов), объектно-ориентированный API без SQL, автоматические live-обновления (изменения отражаются в UI мгновенно), встроенная поддержка синхронизации через Realm Sync, простые миграции и меньший boilerplate-код. Недостатки: собственный формат файла базы (не SQLite), больший размер библиотеки (~5–10 МБ), отсутствие полной поддержки сложных SQL-запросов (JOIN, подзапросы), ограниченная поддержка транзакций по сравнению с ACID в Room, и потенциальные проблемы совместимости при смене платформы. Room остаётся предпочтительным для приложений, требующих строгой реляционной модели, сложных запросов, интеграции с Paging 3 и полной совместимости с SQLite-экосистемой.

**Что такое Firebase Firestore и Realtime Database?**
Firebase Realtime Database — это облачная NoSQL-база данных реального времени, которая хранит данные в виде JSON-дерева и синхронизирует изменения между клиентами мгновенно через WebSocket-подобный протокол. Она поддерживает оффлайн-режим с локальным кэшем и автоматической синхронизацией при восстановлении соединения. Firestore — более новая и современная версия, также NoSQL, но с документо-коллекционной моделью (коллекции → документы → подколлекции), мощными запросами (составные индексы, where/in/arrays-contains), масштабируемостью и лучшей поддержкой оффлайн. Firestore считается более гибким и безопасным (правила безопасности на уровне данных), в то время как Realtime Database проще для небольших приложений с частыми обновлениями.

**Когда лучше использовать Room, когда Realm, а когда Firestore?**
Room — лучший выбор для большинства локальных приложений, где нужна строгая реляционная модель, сложные запросы, интеграция с Paging 3, Coroutines/Flow и полная поддержка ACID-транзакций (например, заметки, финансы, локальный кэш). Realm подходит для приложений с высокой скоростью записи, сложными объектными графами (например, чаты с вложенными сообщениями) или когда нужна встроенная синхронизация без Firebase (Realm Sync). Firestore идеален для облачных приложений с синхронизацией между пользователями, оффлайн-поддержкой и реальным временем (чаты, коллаборативные списки, социальные функции). Выбор зависит от потребности в локальном хранении (Room/Realm) или облачной синхронизации (Firestore).

**Что такое DataStore (Preferences и Proto) и зачем он появился?**
DataStore — современная замена SharedPreferences в Android Jetpack, появившаяся для решения проблем SharedPreferences (отсутствие типобезопасности, отсутствие транзакций, блокировка при записи, сложность миграций). DataStore бывает двух видов: Preferences DataStore — для простых пар ключ-значение (аналог SharedPreferences, но асинхронный и типобезопасный), Proto DataStore — для структурированных данных с использованием Protocol Buffers (protobuf), что позволяет хранить сложные объекты с версионированием и миграциями. DataStore полностью асинхронный (на основе корутин и Flow), поддерживает транзакции и гарантирует согласованность данных.

**Чем DataStore лучше SharedPreferences?**
DataStore полностью асинхронный и неблокирующий (работает на корутинах), в отличие от SharedPreferences, где commit() блокирует поток, а apply() может потерять данные при краше. DataStore обеспечивает типобезопасность (нет строковых ключей), транзакционность (все изменения атомарны), поддержку Flow для реактивного чтения, встроенные миграции и лучшую обработку ошибок. Preferences DataStore проще SharedPreferences, Proto DataStore позволяет хранить сложные структуры с версионированием схемы. SharedPreferences остаётся только для очень простых случаев или legacy-кода.

**Как работает Proto DataStore?**
Proto DataStore использует Protocol Buffers (protobuf) для сериализации структурированных данных. Сначала создаётся .proto-файл с описанием сообщения (например, UserPreferences с полями theme, language). Затем генерируется Kotlin-класс через protoc. DataStore создаётся через DataStore<UserPreferences> с помощью createDataStore и produceProtobufDataStore. Чтение — через dataStore.data (Flow<UserPreferences>), запись — через dataStore.updateData { it.copy(...) } в транзакции. При изменении схемы добавляется миграция. Proto DataStore обеспечивает типобезопасность, версионирование и миграции, что делает его идеальным для хранения настроек пользователя.

**Как правильно проектировать базу данных для оффлайн-приложения?**
Проектирование базы данных для оффлайн-приложения начинается с определения доменной модели и сценариев использования: какие данные должны быть доступны без сети, как часто они обновляются и как разрешать конфликты при синхронизации. Необходимо нормализовать структуру до 3-й нормальной формы для минимизации дублирования, но допускать контролируемую денормализацию там, где скорость чтения критична (например, хранение имени пользователя в таблице сообщений). Важно заранее продумать индексы на часто фильтруемых полях (user_id, timestamp, status), составные ключи для связующих таблиц и стратегию миграций. В Android рекомендуется использовать Room с чётким разделением слоёв: Entity → DAO → Repository → UseCase, с поддержкой Flow/StateFlow для реактивного обновления UI. Также следует предусмотреть механизм кэширования сетевых данных и разрешения конфликтов (last-write-wins или пользовательское слияние).

**Что такое Single Source of Truth и как его реализовать?**
Single Source of Truth (SSoT) — это принцип архитектуры, при котором все данные в приложении имеют единственный источник истины, а остальные компоненты получают их только от этого источника. В Android это обычно локальная база данных (Room), которая становится единственным местом хранения и модификации данных, даже если они изначально приходят из сети. Реализация: Repository выступает фасадом, который всегда читает из Room и обновляет сеть только при необходимости (например, при pull-to-refresh); сетевые ответы сначала сохраняются в Room, а UI наблюдает за Room через Flow/StateFlow. Это обеспечивает согласованность данных, оффлайн-поддержку и упрощает тестирование.

**Как организовать Repository слой при работе с Room?**
Repository — это слой между источниками данных (Room, сеть, DataStore) и бизнес-логикой (UseCase/ViewModel), который абстрагирует способ получения данных и обеспечивает Single Source of Truth. В типичной реализации Repository содержит методы, возвращающие Flow или suspend-функции, внутри которых сначала проверяется локальный кэш (Room), при отсутствии или устаревании — выполняется сетевой запрос, результат сохраняется в Room и возвращается. Рекомендуется использовать Repository как интерфейс с реализацией (например, OfflineFirstRepository), чтобы легко подменять источники данных при тестировании. В Android Repository обычно инжектируется через Hilt/Dagger и работает с Coroutines/Flow.

**Как кэшировать данные из сети в Room?**
Кэширование сетевых данных в Room реализуется через паттерн «сначала сеть, потом кэш» или «оффлайн-first»: при запросе данных сначала проверяется локальная база (Room), если данные есть и актуальны — возвращаются сразу; если нет или устарели — выполняется сетевой запрос (Retrofit), ответ сохраняется в Room, после чего UI обновляется через Flow/StateFlow. Важно реализовать механизм инвалидации кэша (по времени жизни, по событию или по версии) и обработку конфликтов. В Android это часто делается в Repository с использованием Flow для реактивного обновления и Coroutines для асинхронной записи.

**Что такое Paging 3 и как оно работает с Room?**
Paging 3 — библиотека Jetpack для эффективной загрузки больших списков по частям (pagination), которая подгружает данные по мере прокрутки RecyclerView, минимизируя использование памяти и избегая загрузки всего списка сразу. В Room Paging 3 интегрируется через @Query с возвратом PagingSource: метод DAO возвращает PagingSource<Key, Value>, где Key — тип ключа пагинации (например, Int для offset), Value — тип сущности. Room генерирует запросы с LIMIT и OFFSET автоматически. PagingSource работает с Flow<PagingData<Value>>, что позволяет наблюдать за загрузкой и отображать placeholder'ы. Это идеально для списков сообщений, постов или товаров в Android-приложениях.

**Как делать сложные запросы в Room (много JOIN, подзапросы)?**
Сложные запросы в Room выполняются через @Query с полноценным SQL: можно использовать JOIN (INNER, LEFT, CROSS), подзапросы, агрегатные функции, CASE, UNION и т.д. Room проверяет синтаксис на этапе компиляции и генерирует безопасный код. Для связанных сущностей рекомендуется использовать @Relation или @Embedded с POJO-классом для результата. Пример: @Query("SELECT u.*, COUNT(p.id) as postCount FROM users u LEFT JOIN posts p ON u.id = p.user_id GROUP BY u.id"). Для очень сложных запросов можно использовать @RawQuery, но это теряет типобезопасность.

**Как работать с полиморфными сущностями в Room?**
Полиморфные сущности в Room реализуются через несколько подходов: 1) одна таблица с дискриминатором (@ColumnInfo(name = "type") и @Embedded для разных подтипов); 2) отдельные таблицы для каждого подтипа с общей базовой таблицей и @ForeignKey; 3) использование @Embedded с @Relation для связей. Room не поддерживает наследование сущностей напрямую, поэтому чаще всего создают отдельные @Entity для каждого конкретного типа и используют общий DAO с UNION или JOIN. Это позволяет хранить разные типы объектов (например, разные виды сообщений в чате) с общим интерфейсом.

**Что такое upsert в Room и как его реализовать?**
Upsert (update or insert) — операция, которая обновляет запись, если она существует (по первичному ключу), или вставляет новую, если записи нет. В Room нет встроенной аннотации @Upsert (появилась только в более поздних версиях), поэтому реализуют через комбинацию @Insert(onConflict = OnConflictStrategy.REPLACE) или вручную в @Transaction: сначала попытка update, если rowsAffected == 0 — insert. В новых версиях Room (2.6+) есть @Upsert. Это удобно для синхронизации данных из сети.

**Как правильно обрабатывать ошибки в Room?**
Обработка ошибок в Room включает: использование try-catch вокруг suspend-методов DAO, проверку возвращаемого значения (например, rowsAffected после update/delete), использование Flow для наблюдения за состоянием, и обработку SQLiteException (например, SQLiteConstraintException при нарушении уникальности). Рекомендуется возвращать sealed-класс Result<Success, Error> из Repository, где Error содержит тип ошибки (constraint, disk full и т.д.). Для миграций — fallbackToDestructiveMigration() только в debug-сборке, в продакшене — явные миграции с обработкой ошибок.

**Как оптимизировать производительность Room (индексы, запросы, prefetch)?**
Оптимизация Room включает создание индексов (@Index в @Entity) на столбцах в WHERE/JOIN/ORDER BY, использование Paging 3 для больших списков (prefetchDistance, initialLoadSize), ограничение возвращаемых столбцов в @Query, запуск тяжёлых операций на Dispatchers.IO, использование @Transaction для пакетных вставок, настройка PRAGMA (journal_mode=WAL), кэширование часто запрашиваемых данных в памяти (LruCache или StateFlow). Важно профилировать запросы через Database Inspector и избегать N+1 проблемы с помощью @Relation и JOIN.

**Что такое Database Inspector и как им пользоваться на практике?**
Database Inspector — инструмент в Android Studio (App Inspection → Database Inspector), позволяющий в реальном времени просматривать и редактировать содержимое Room/SQLite-базы на подключённом устройстве или эмуляторе. После запуска приложения и выбора процесса в Inspector отображаются все базы данных и таблицы; можно выполнять произвольные SQL-запросы, редактировать строки, просматривать схему и экспортировать данные. На практике используется для отладки миграций, проверки содержимого после вставки, поиска проблем с данными и верификации индексов.

**Как делать бэкап и восстановление базы данных в Android?**
Бэкап базы данных в Android выполняется копированием файла базы (по умолчанию в /data/data/<package>/databases/) с помощью Context.getDatabasePath() или прямого доступа через File. Для Room можно использовать Room.databaseBuilder().createFromAsset() для предзаполненной базы или копировать файл вручную. Восстановление — обратный процесс: закрыть базу, скопировать файл в нужную директорию, открыть заново. Для пользовательского бэкапа рекомендуется использовать Backup API (Auto Backup) или ручной экспорт в /sdcard/. Для безопасности — шифрование базы (SQLCipher) и проверка целостности при восстановлении.

**Как защитить SQLite/Room базу данных (SQLCipher, encryption)?**
Защита SQLite/Room базы данных в Android достигается преимущественно через шифрование с помощью библиотеки SQLCipher, которая заменяет стандартный SQLite на зашифрованную версию. SQLCipher использует AES-256 в режиме CBC с 256-битным ключом, получаемым из пользовательского пароля (или ключа, сгенерированного с помощью PBKDF2). В Room интеграция осуществляется через net.zetetic:android-database-sqlcipher, где в RoomDatabase.Builder указывается SupportFactory с passphrase. Ключ может быть получен из Keystore, биометрии или пользовательского ввода. Без ключа база остаётся недоступной даже при физическом доступе к файлу. Это критично для хранения чувствительных данных (токены, персональные данные, медицинская информация). Альтернативы — Jetpack Security (EncryptedSharedPreferences и EncryptedFile), но они не шифруют саму базу Room/SQLite.

**Как правильно работать с чувствительными данными в БД?**
Работа с чувствительными данными в базе требует строгого подхода к безопасности и минимизации рисков утечек. Во-первых, данные должны шифроваться на уровне базы (SQLCipher) или отдельных полей (например, с помощью EncryptedString через Jetpack Security). Во-вторых, доступ к базе должен быть ограничен: использовать Room с suspend-функциями и Flow только в Repository, избегать прямых ссылок на DAO из UI. В-третьих, ключи шифрования хранятся в Android Keystore или получаются через биометрию/UserAuthenticationRequired. В-четвёртых, чувствительные данные (пароли, токены, медицинские записи) лучше не хранить в базе вовсе, а использовать зашифрованные альтернативы (KeyStore, EncryptedSharedPreferences). В-пятых, регулярно проводить аудит и тестирование на утечки (LeakCanary, Mobile Security Framework). В Android это особенно важно из-за возможности root-доступа и физического извлечения данных с устройства.

**Как тестировать DAO и Repository слой?**
Тестирование DAO и Repository слоя в Android выполняется с помощью AndroidJUnit4 и Room in-memory базы. Для DAO создаётся временная база через Room.inMemoryDatabaseBuilder(context).allowMainThreadQueries().build(), затем тестируются методы insert/update/delete/query с проверкой результатов через assertEquals или Turbine для Flow. Repository тестируется через инъекцию моков (Mockito или MockK): мок сети (Retrofit), мок DAO, проверка, что при успешном запросе данные сохраняются в Room и возвращаются через Flow/StateFlow. Используются runTest из kotlinx-coroutines-test для корутин и Turbine для проверки последовательности эмиссий. Тесты покрывают happy path, ошибки сети, пустые ответы и миграции.

**Что такое Integration Test для Room?**
Integration Test для Room — это тест, проверяющий взаимодействие DAO с реальной базой данных (обычно in-memory), включая вставку, обновление, удаление и выборку данных, а также работу с Flow/LiveData. В отличие от unit-тестов DAO, integration-тесты проверяют всю цепочку: от вызова метода DAO до получения результата, включая транзакции (@Transaction), индексы и поведение при параллельном доступе. Используется AndroidJUnit4, Room.inMemoryDatabaseBuilder и runTest для корутин. Такие тесты особенно важны для проверки миграций, сложных запросов с JOIN и @Relation.

**Как измерить производительность запросов в Room?**
Производительность запросов в Room измеряется несколькими способами: через Android Studio Database Inspector (показывает план запроса EXPLAIN QUERY PLAN), Android Profiler (CPU → Database → Query Execution Time), логирование времени выполнения с помощью @Query с Loggable, или через Benchmark библиотеку (androidx.benchmark). Для реальных сценариев используют Espresso/Compose UI Test с измерением времени загрузки списка. Важно измерять не только время, но и количество сканируемых строк (rows scanned) и использование индексов. В продакшене добавляют Crashlytics Custom Keys с временем выполнения критических запросов.

**Почему большие запросы в Room могут вызвать ANR?**
Большие запросы в Room могут вызвать ANR, если они выполняются на Main Thread (например, синхронный вызов DAO-метода без suspend/Flow). SQLite блокирует базу при записи, а сложные SELECT с JOIN или без индексов выполняют полный скан таблицы, что занимает сотни миллисекунд или секунды. Это блокирует Main Thread, приводит к зависанию UI и ANR (если задержка >5 секунд). Даже асинхронные запросы могут косвенно вызывать ANR, если результат обрабатывается на Main Thread без правильного использования Dispatchers.Main.immediate или Flow.collectAsState().

**Как правильно использовать @Transaction для сложных операций?**
@Transaction в DAO гарантирует, что все операции внутри метода выполняются в одной транзакции: если хотя бы одна завершится неудачно, все изменения откатываются. Это обязательно использовать при операциях, затрагивающих несколько таблиц (например, вставка пользователя + его постов + обновление счётчика). Также @Transaction нужен при чтении связанных сущностей через @Relation, чтобы избежать неконсистентного состояния при параллельных изменениях. В корутинах метод помечается suspend, а вызов выполняется в IO-диспетчере.

**Что такое Query Callback и когда его использовать?**
Query Callback — механизм в Room, позволяющий получать уведомления о выполнении каждого запроса (onQuery) или изменении данных (onTableChanged). Настраивается через RoomDatabase.Callback() .addOnQueryCallback() или addOnTableChangedCallback(). Используется для отладки (логирование всех запросов), мониторинга производительности или реализации кастомной логики (например, логирование медленных запросов). В продакшене применяется редко, чаще в debug-сборке или для аналитики.

**Как правильно использовать Foreign Key с каскадным удалением в Room?**
@ForeignKey в @Entity задаёт связь с родительской таблицей: entity — родительский класс, parentColumns/childColumns — ключи, onDelete = CASCADE/SET_NULL/RESTRICT/NO_ACTION. CASCADE автоматически удаляет дочерние записи при удалении родительской (например, удаление пользователя удаляет его посты). SET_NULL устанавливает NULL в дочернем ключе. Важно: каскадные действия выполняются только при использовании @Delete или DELETE-запроса через Room, а не при ручном SQL. Для сложных случаев используют @Transaction.

## Как обрабатывать большие данные (миллионы записей) в Room?
Для работы с миллионами записей в Room используют Paging 3 с PagingSource из DAO (@Query с LIMIT/OFFSET), что позволяет загружать данные порциями. Обязательны индексы на часто фильтруемых столбцах, правильный выбор типов (Long вместо String для дат), WAL-режим (journal_mode=WAL), пакетные вставки через @Insert с @Transaction, использование Flow/PagingData для реактивного обновления UI. Избегать полного SELECT * — указывать только нужные столбцы. Для экстремальных случаев применяют FTS5 (полнотекстовый поиск) или внешние NoSQL-решения.

**Journal_mode=WAL**
Это один из режимов ведения журнала транзакций (journal mode) в SQLite, который значительно улучшает производительность и конкурентность операций чтения и записи по сравнению с режимом по умолчанию (DELETE или TRUNCATE). В обычном режиме (rollback journal) при любой модификации данных SQLite создаёт временный файл-журнал, в который записывает копии изменяемых страниц таблицы. Пока транзакция не завершена, все чтения блокируются, чтобы избежать чтения несогласованных данных. Это означает, что при записи вся база блокируется для чтения.
Недостатки: 1) появляется дополнительный файл .wal и .shm (shared memory), что увеличивает размер базы на диске. 2) При сбое питания или краше приложения возможна небольшая потеря последних изменений (но это редкость при правильной работе с транзакциями). 3) В очень редких случаях может потребоваться ручной checkpoint (PRAGMA wal_checkpoint).

# Android Core

**Что такое Android и на базе какой ОС он построен?**
Android — это мобильная операционная система с открытым исходным кодом, разработанная Google и предназначенная преимущественно для смартфонов, планшетов, телевизоров, часов и других устройств. Она построена на базе ядра Linux (Linux Kernel), которое обеспечивает взаимодействие с аппаратным обеспечением, управление процессами, памятью, файловой системой и драйверами устройств. Однако Android не является просто «Linux для мобильных устройств»: поверх ядра расположены многочисленные проприетарные и открытые слои Google (Android Runtime, HAL, System Services, Google Mobile Services), а также собственная реализация Java-виртуальной машины (ART), что делает Android существенно отличающимся от классических дистрибутивов Linux (Ubuntu, Fedora и т.д.).

**Какую роль играет Linux Kernel в Android?**
Linux Kernel в Android выполняет роль базового уровня операционной системы, обеспечивая взаимодействие между аппаратным обеспечением и остальными слоями Android. Он отвечает за управление процессами и потоками, выделение и освобождение памяти, работу с файловой системой, драйверами устройств (GPU, камера, сенсоры, Wi-Fi, Bluetooth), планирование задач, безопасность (через SELinux) и энергопотребление. Android использует модифицированное ядро Linux (обычно версии 4.x–5.x в зависимости от версии ОС), в котором добавлены специфические патчи для мобильных устройств (например, wakelocks, Binder-драйвер, низкоуровневая поддержка touch-экранов). Без ядра Linux Android не смог бы работать с железом устройства.

**Что такое Dalvik и ART? В чём разница?**
Dalvik и ART — это две последовательные реализации виртуальной машины для выполнения Android-приложений. Dalvik (использовалась до Android 4.4) — это регистровая виртуальная машина, которая компилировала APK в промежуточный байт-код DEX (Dalvik Executable) и интерпретировала его во время выполнения. ART (Android Runtime), введённый в Android 4.4 и полностью заменивший Dalvik в Android 7.0, использует ahead-of-time (AOT) компиляцию: во время установки приложения DEX-код преобразуется в нативный машинный код, что значительно ускоряет запуск и выполнение приложений. ART также поддерживает JIT-компиляцию для горячих участков кода и имеет улучшенный сборщик мусора. Основное отличие: Dalvik — интерпретатор + JIT, ART — преимущественно AOT + JIT, что дало прирост производительности до 2–3 раз.

**Что такое Zygote и как он работает?**
Zygote — это специальный системный процесс в Android, который запускается при старте системы и остаётся в памяти всё время работы устройства. Его главная задача — служить «родителем» для всех пользовательских приложений: при запуске нового приложения Zygote форкает (создаёт копию) себя с уже загруженными базовыми библиотеками (Android Framework, ART) и Dalvik/ART runtime. Это позволяет значительно ускорить запуск приложений, так как не нужно каждый раз заново загружать и инициализировать сотни мегабайт системных библиотек. После форка Zygote передаёт управление новому процессу приложения, который продолжает выполнение с точки fork. Такой подход экономит время запуска (особенно заметно на старых устройствах) и уменьшает потребление памяти.

**Что такое Android Runtime (ART) и почему он заменил Dalvik?**
Android Runtime (ART) — это современная среда выполнения приложений в Android, которая пришла на смену Dalvik начиная с Android 4.4 (KitKat) и полностью вытеснила его в Android 7.0 (Nougat). ART использует ahead-of-time (AOT) компиляцию: при установке приложения байт-код DEX преобразуется в нативный машинный код для текущего процессора, что значительно ускоряет запуск и выполнение приложения по сравнению с интерпретацией в Dalvik. ART также имеет улучшенный сборщик мусора (concurrent mark-sweep), поддержку 64-битной архитектуры, JIT-компиляцию для горячих участков кода и лучшую оптимизацию. Замена Dalvik на ART была обусловлена необходимостью повысить производительность, уменьшить энергопотребление и подготовить платформу к 64-битным устройствам.

**Что такое Binder и как он используется в Android?**
Binder — это межпроцессный механизм коммуникации (IPC), разработанный специально для Android и лежащий в основе всей архитектуры системы. Он позволяет процессам обмениваться данными и вызывать методы друг друга через прокси-объекты, обеспечивая безопасность и производительность. Binder работает поверх Linux Kernel (через специальный драйвер /dev/binder) и используется для общения между приложениями и системными сервисами (ActivityManager, WindowManager, PackageManager и т.д.). При вызове метода через Binder передаётся Parcel (структурированный пакет данных), а система автоматически управляет правами доступа, UID и безопасностью. В Android практически все системные сервисы используют Binder (AIDL-интерфейсы), что делает его ключевым компонентом платформы.

**Что такое System Server и какие сервисы он запускает?**
System Server — это системный процесс, который запускается Zygote после старта устройства и содержит большинство критических системных сервисов Android. Он отвечает за работу всей платформы и предоставляет API для приложений через Binder. В System Server запускаются: ActivityManagerService (управление активностями), WindowManagerService (управление окнами), PackageManagerService (управление приложениями), PowerManagerService, AlarmManagerService, NotificationManagerService, SensorService, LocationManagerService и многие другие. Благодаря этому все приложения могут общаться с этими сервисами через Binder без запуска отдельных процессов для каждого сервиса. System Server — один из самых важных и нагруженных процессов в системе.

**Как работает процесс создания приложения в Android?**
Процесс создания приложения начинается с того, что пользователь нажимает иконку или система вызывает Intent. ActivityManagerService получает Intent и проверяет, существует ли уже процесс для этого приложения (по UID). Если процесса нет — ActivityManagerService запрашивает у Zygote создать новый процесс через Binder. Zygote форкает себя, загружает ART, инициализирует Android Framework и передаёт управление новому процессу. Затем ActivityThread (главный класс приложения) запускается, вызывает Application.onCreate(), после чего создаётся и запускается целевая Activity (onCreate → onStart → onResume). Всё это происходит очень быстро благодаря предварительной загрузке Zygote.

**Что такое User ID (UID) и как Android использует его для изоляции приложений?**
User ID (UID) — это уникальный идентификатор пользователя в Linux, который Android присваивает каждому установленному приложению. По умолчанию каждое приложение получает собственный UID (обычно от 10000 и выше), что обеспечивает изоляцию на уровне операционной системы: приложения не могут читать файлы, процессы или память друг друга без явных разрешений. UID также используется для управления правами доступа к файлам, сетевым сокетам и Binder-вызовам. Приложения одного разработчика с одинаковым sharedUserId могут иметь общий UID и делить данные, но это редко используется из-за рисков безопасности.

**Что такое SELinux и как он влияет на безопасность Android?**
SELinux (Security-Enhanced Linux) — это модуль безопасности ядра Linux, который реализует обязательный контроль доступа (MAC — Mandatory Access Control). В Android SELinux работает в enforcing-режиме (с Android 4.3) и ограничивает даже системные процессы и приложения в том, какие действия они могут выполнять. Политики SELinux определяют, какие домены (контексты) могут читать/писать файлы, отправлять Binder-запросы или запускать процессы. Это существенно снижает ущерб от уязвимостей и root-эксплойтов: даже если приложение получило root-доступ, SELinux может запретить ему доступ к критическим файлам. В Android SELinux защищает системные сервисы, файловую систему и межпроцессное взаимодействие.

**Что такое Application класс и когда он создаётся?**
Application — это базовый класс, представляющий глобальное состояние приложения. Он создаётся системой автоматически при запуске процесса приложения (до создания любой Activity). Разработчик может создать собственный класс, наследующий Application, и указать его в AndroidManifest.xml в теге <application android:name=".MyApplication">. Метод onCreate() вызывается один раз при создании процесса и идеально подходит для глобальной инициализации (например, установка Timber, инициализация Crashlytics, создание DI-контейнера). Важно: Application создаётся только один раз на весь процесс, даже если в процессе несколько Activity или Service.

**Жизненный цикл Application (onCreate, onTerminate, onLowMemory)?**
Жизненный цикл Application довольно простой по сравнению с Activity. onCreate() вызывается один раз при создании процесса и используется для глобальной инициализации. onTerminate() вызывается только в эмуляторе при явном завершении процесса (в реальных устройствах почти никогда не вызывается). onLowMemory() и onTrimMemory() уведомляют о нехватке памяти и позволяют освободить ресурсы (например, очистить кэши). В Android Application живёт столько же, сколько живёт процесс, и его методы редко используются для сложной логики, так как большинство задач лучше делегировать ViewModel или Repository.

**Что такое процесс приложения в Android?**
Процесс приложения — это экземпляр Linux-процесса, в котором выполняется код вашего приложения. По умолчанию каждое приложение запускается в собственном процессе с уникальным UID. Внутри процесса могут работать несколько компонентов (Activity, Service, BroadcastReceiver, ContentProvider). Процесс имеет собственную виртуальную машину ART, кучу памяти и стек. Android может убить процесс при нехватке памяти, но старается сохранять foreground-процессы и процессы с видимыми Activity. Процесс может быть multi-process (android:process в манифесте), но это редко используется.

**Чем отличается процесс приложения от Linux-процесса?**
Процесс приложения в Android — это обычный Linux-процесс, но с дополнительными особенностями платформы. Каждый процесс имеет собственный UID для изоляции, собственную виртуальную машину ART, собственный Dalvik/ART heap и собственный Binder-контекст. Android добавляет управление жизненным циклом (OOM Killer с приоритетами), интеграцию с ActivityManager и ограничения на фоновую работу. Обычный Linux-процесс не имеет этих ограничений и не интегрирован с Android Framework (Activity, Service и т.д.).

**Что такое Application Not Responding (ANR) и какие причины его вызывают?**
ANR (Application Not Responding) — это состояние, когда приложение не отвечает на события пользователя более 5 секунд (или 5 секунд без ответа на BroadcastReceiver). Основные причины: длительные операции на Main Thread (сетевые запросы, работа с БД, тяжёлые вычисления), блокировка Main Thread из-за synchronized, слишком много BroadcastReceiver в 10 секунд, зависший Service в foreground. Android показывает диалог «Приложение не отвечает» с возможностью принудительного закрытия. Чтобы избежать ANR, все тяжёлые операции выносятся в Worker Thread или Coroutines с правильным диспетчером.

**Что такое Process Death и как с ним бороться?**
Process Death — ситуация, когда Android убивает процесс приложения из-за нехватки памяти, но при этом сохраняет состояние компонентов (Activity, Service) в Bundle. После смерти процесса при возврате к приложению Android пересоздаёт процесс и восстанавливает состояние через savedInstanceState. Борьба: сохранять критическое состояние в ViewModel (которая живёт дольше процесса), использовать SavedStateHandle, избегать хранения данных в статических полях, правильно обрабатывать onSaveInstanceState/ onRestoreInstanceState. ViewModel + StateFlow/StateHolder — лучший способ пережить Process Death без потери данных.

**Что такое Application.onTrimMemory() и какие уровни существуют?**
onTrimMemory() — метод Application, вызываемый системой при нехватке памяти для освобождения ресурсов. Уровни: TRIM_MEMORY_BACKGROUND (приложение в фоне), TRIM_MEMORY_MODERATE (умеренная нехватка), TRIM_MEMORY_COMPLETE (критическая нехватка, процесс скоро убьют), TRIM_MEMORY_RUNNING_MODERATE/CRITICAL и т.д. В методе рекомендуется очищать кэши, отменять загрузки изображений, освобождать Bitmap. В Android это один из способов предотвратить OOM и сохранить приложение в памяти.

**Как Android убивает процессы (OOM Killer)?**
OOM Killer (Out Of Memory Killer) — механизм ядра Linux, который убивает процессы при критической нехватке памяти. Android модифицировал OOM Killer, добавив приоритеты (ADJ — Adj Score): foreground-процессы имеют самый высокий приоритет, затем visible, service, background и т.д. Чем ниже приоритет — тем раньше процесс будет убит. Также Android использует LMKD (Low Memory Killer Daemon), который мониторит давление памяти и убивает процессы с низким приоритетом заранее. Разработчик не может напрямую влиять на OOM Killer, но может повысить приоритет процесса через Foreground Service или Notification.

**Что такое foreground process, visible process, service process?**
Foreground process — процесс, содержащий Activity на экране или Foreground Service (самый высокий приоритет, почти никогда не убивается). Visible process — процесс с видимой, но не активной Activity (например, диалог поверх другого приложения). Service process — процесс с запущенным Service (не foreground). Background process — процесс без видимых компонентов. Android старается сохранять foreground и visible процессы, ниже по приоритету service, и еще ниже background-процессы.

**Как работает multi-process в Android (android:process)?**
Multi-process позволяет запускать компоненты приложения в отдельном процессе через атрибут android:process в манифесте (например, <service android:process=":remote">). Каждый процесс имеет собственный UID, собственный Dalvik/ART heap и собственную память. Это полезно для изоляции (например, отдельный процесс для WebView), но увеличивает overhead и усложняет обмен данными (нужен Binder, AIDL). В современных приложениях multi-process почти не используется — Google рекомендует избегать его из-за сложности и энергопотребления.

**Что такое Isolated Process и когда его использовать?**
Isolated Process — процесс с собственным UID, полностью изолированный от основного процесса приложения (android:isolatedProcess="true"). Используется для запуска ненадёжного кода (например, WebView, плагины, сторонние библиотеки), чтобы ограничить ущерб от уязвимостей. В изолированном процессе нет доступа к данным основного приложения, и он может быть убит без влияния на основное приложение. В Android это применяется в WebView (с Android 8+ по умолчанию) и в некоторых плагинах.

**Как проверить, в каком процессе работает ваш код?**
Чтобы проверить текущий процесс, используйте ActivityManager или android.os.Process. Самый простой способ: android.os.Process.myPid() возвращает PID текущего процесса. Для получения имени процесса: ActivityManager.getRunningAppProcesses() и поиск по PID, или Context.getApplicationInfo().processName. В Android Studio можно увидеть процесс в Profiler → Sessions. В коде часто сравнивают Process.myUid() с ApplicationInfo.uid из манифеста.

**Полный жизненный цикл Activity.**
Жизненный цикл Activity в Android состоит из семи основных методов, которые система вызывает в определённой последовательности в зависимости от состояния компонента. Методы: onCreate() — инициализация при первом создании или после смерти процесса; onStart() — Activity становится видимым, но ещё не интерактивным; onResume() — Activity полностью активна и получает фокус ввода; onPause() — Activity теряет фокус, но остаётся видимой (например, при появлении диалога); onStop() — Activity перестаёт быть видимой; onDestroy() — Activity уничтожается (при закрытии, смене конфигурации или смерти процесса); onRestart() — вызывается перед onStart(), если Activity была остановлена, но не уничтожена. Полный цикл при первом запуске: onCreate → onStart → onResume. При повороте экрана: onPause → onStop → onDestroy → onCreate → onStart → onResume. При уходе в фон: onPause → onStop. При возвращении: onRestart → onStart → onResume (или onCreate, если процесс умер).

**В каком порядке вызываются методы при первом запуске Activity?**
При первом запуске Activity система вызывает методы в строго определённом порядке: сначала onCreate(Bundle savedInstanceState), где выполняется базовая инициализация (setContentView, findViewById, ViewModel инициализация). Затем onStart(), когда Activity становится видимой, но ещё не полностью интерактивной. Далее onResume(), когда Activity получает фокус ввода и становится полностью активной. Таким образом, полный порядок при первом запуске: onCreate → onStart → onResume. После этого Activity готова к взаимодействию с пользователем. Важно: onCreate вызывается только один раз за жизненный цикл экземпляра Activity (если процесс не умер).

**Что происходит при повороте экрана (configuration change)?**
При повороте экрана происходит изменение конфигурации (configuration change), из-за чего Activity уничтожается и пересоздаётся заново. Порядок вызовов: onPause → onStop → onDestroy (старый экземпляр уничтожается), затем создаётся новый экземпляр: onCreate → onStart → onResume. Система автоматически сохраняет состояние в Bundle через onSaveInstanceState() перед onStop и передаёт его в onCreate нового экземпляра. Это позволяет восстановить UI (например, текст в EditText, позицию в списке). Однако ViewModel и SavedStateHandle переживают этот процесс, поэтому современная практика — хранить состояние именно там, а не в Bundle.

**Разница между onCreate() и onCreate(Bundle savedInstanceState)?**
onCreate() — это метод жизненного цикла, вызываемый при создании экземпляра Activity. Без параметра он используется в старых версиях или когда состояние не нужно восстанавливать. onCreate(Bundle savedInstanceState) — переопределённый метод, который получает Bundle с сохранённым состоянием (если Activity пересоздаётся, например, при повороте экрана или Process Death). Внутри этого метода обычно вызывается super.onCreate(savedInstanceState), а затем setContentView() и восстановление UI (если savedInstanceState != null). Разница в том, что второй вариант позволяет корректно восстанавливать состояние, первый — нет.

**Когда вызывается onSaveInstanceState() и onRestoreInstanceState()?**
onSaveInstanceState(Bundle outState) вызывается системой перед тем, как Activity может быть уничтожена не по воле пользователя (поворот экрана, Process Death, смена конфигурации), но после onPause(). В этот момент нужно сохранить временное состояние UI (текст в полях, позицию скролла) в Bundle. onRestoreInstanceState(Bundle savedInstanceState) вызывается после onStart(), если Activity пересоздаётся и Bundle не null. Метод используется для восстановления состояния, которое не было обработано в onCreate(). В современной практике эти методы почти не используются — состояние хранится в ViewModel и SavedStateHandle.

**Разница между onPause() и onStop()?**
onPause() вызывается, когда Activity теряет фокус, но остаётся видимой (например, при появлении диалога, частичном перекрытии другим Activity или блокировке экрана). Это момент, когда нужно приостановить анимации, освободить ресурсы камеры/микрофона и остановить обновления UI. onStop() вызывается, когда Activity полностью перестаёт быть видимой (переход в фон, запуск другой Activity на весь экран). Здесь освобождаются более тяжёлые ресурсы (например, unregisterReceiver, остановка сервисов). onPause всегда предшествует onStop, но onStop может не вызываться при быстром уничтожении процесса.

**Когда вызывается onDestroy()?**
onDestroy() вызывается, когда Activity полностью уничтожается: при вызове finish(), при смене конфигурации (поворот экрана), при уничтожении процесса системой (OOM Killer, низкий приоритет) или при закрытии приложения. Метод предназначен для окончательной очистки ресурсов (закрытие соединений, отписка от наблюдателей, освобождение Bitmap). Важно: onDestroy не гарантирует вызов (процесс может быть убит без него), поэтому критические ресурсы лучше освобождать в onStop или через LifecycleObserver.

**Что такое Activity Lifecycle Owner?**
Activity (а также Fragment) реализует интерфейс LifecycleOwner, который предоставляет доступ к Lifecycle — объекту, отслеживающему текущее состояние жизненного цикла (CREATED, STARTED, RESUMED, DESTROYED и т.д.). LifecycleOwner позволяет подписываться на события жизненного цикла (addObserver) и использовать lifecycle-aware компоненты (LiveData.observe, Flow.collect с repeatOnLifecycle). Это ключевая часть Jetpack Lifecycle, которая упрощает управление ресурсами и предотвращает утечки памяти, автоматически отписывая наблюдателей при уничтожении компонента.

**Как правильно сохранять состояние в ViewModel, а не в onSaveInstanceState()?**
ViewModel предназначен для хранения и управления UI-состоянием, которое должно пережить изменения конфигурации (поворот экрана) и Process Death. В отличие от onSaveInstanceState(), который сохраняет только Bundle и ограничен размером (около 1 МБ), ViewModel живёт дольше Activity и не имеет жёстких ограничений по объёму. Правильный подход: хранить данные в ViewModel (в полях или StateFlow), использовать SavedStateHandle для сохранения сложных объектов при Process Death, а onSaveInstanceState применять только для простых UI-состояний (позиция скролла, открытый диалог). Это делает код чище и тестируемее.

**Что такое launchMode и какие значения существуют?**
launchMode — атрибут в AndroidManifest.xml, определяющий поведение Activity при запуске через Intent. Возможные значения: standard (по умолчанию — создаётся новый экземпляр), singleTop (если Activity уже на вершине стека — переиспользуется и вызывается onNewIntent), singleTask (создаёт Activity в новой задаче, если её нет, или переносит существующую на вершину с очисткой стека сверху), singleInstance (аналог singleTask, но всегда в отдельной задаче без других Activity). Эти режимы влияют на навигацию и создание экземпляров Activity.

**Как работает Task и Back Stack?**
Task — это стек Activity, с которым пользователь взаимодействует (одна задача — одна последовательность экранов). Back Stack — это стек Activity внутри задачи, где верхняя Activity видна пользователю. При нажатии Back стек очищается сверху вниз (finish текущей Activity). При запуске новой Activity она добавляется на вершину стека текущей задачи (если launchMode позволяет). Несколько задач могут существовать одновременно (например, браузер и ваше приложение). Android управляет задачами через ActivityManager.

**Что такое affinity и clearTaskOnLaunch?**
Task affinity — атрибут, определяющий, к какой задаче должна принадлежать Activity (по умолчанию — имя пакета приложения). Если affinity отличается — Activity запускается в другой задаче. clearTaskOnLaunch — если true, при запуске приложения из лаунчера стек задачи очищается, и запускается только root Activity. Это полезно для приложений, которые не должны возвращаться к предыдущему состоянию после закрытия (например, банковские приложения)

**Как правильно обрабатывать configuration changes в Activity?**
Configuration changes (поворот экрана, изменение языка, размера шрифта) приводят к уничтожению и пересозданию Activity. Правильный подход: хранить состояние в ViewModel (оно переживает конфигурационные изменения) или SavedStateHandle. Избегать сохранения в onSaveInstanceState() всего, кроме простых значений (позиция скролла). Использовать ViewModel + StateFlow для реактивного UI. В манифесте можно указать android:configChanges="orientation|screenSize" для обработки изменений вручную (не рекомендуется). Лучшая практика — делегировать состояние ViewModel и использовать Lifecycle-aware компоненты.

**Полный жизненный цикл Fragment.**
Жизненный цикл Fragment включает: onAttach() — привязка к Activity; onCreate() — инициализация; onCreateView() — создание View; onViewCreated() — работа с View после создания; onStart() — Fragment видим; onResume() — Fragment активен; onPause() — потеря фокуса; onStop() — невидим; onDestroyView() — уничтожение View; onDestroy() — уничтожение Fragment; onDetach() — отвязка от Activity. При повороте экрана: onPause → onStop → onDestroyView → onDestroy → onDetach → onAttach → onCreate → onCreateView → onViewCreated → onStart → onResume.

**Разница между Fragment lifecycle и Activity lifecycle.**
Fragment lifecycle содержит дополнительные методы: onAttach()/onDetach() (привязка/отвязка к Activity), onCreateView()/onDestroyView() (создание/уничтожение View). Activity не имеет этих методов, так как View создаётся в onCreate через setContentView. Fragment может существовать без View (headless Fragment), Activity — нет. Fragment lifecycle более детализирован и зависит от хоста (Activity/FragmentManager).

**Когда вызывается onAttach(), onCreateView(), onViewCreated()?**
onAttach() вызывается первым, когда Fragment привязывается к Activity (получает Context). onCreateView() — создание View Fragment'а (возвращает View или null для headless). onViewCreated() — сразу после onCreateView(), когда View уже создан и готов к работе (findViewById, установка слушателей). Это рекомендуемое место для инициализации UI.

**Что такое FragmentManager и FragmentTransaction?**
FragmentManager — класс, управляющий Fragment'ами в Activity или другом Fragment'е (getSupportFragmentManager(), childFragmentManager). FragmentTransaction — объект для выполнения операций над Fragment'ами: add, replace, remove, hide, show, addToBackStack. Транзакции выполняются через beginTransaction().commit() или .commitAllowingStateLoss().

**Разница между add() и replace()?**
add() добавляет Fragment в контейнер, не удаляя существующие (можно наложить несколько). replace() удаляет все Fragment'ы в контейнере и добавляет новый. replace чаще используется для навигации по экранам, add — для наложения (диалоги, оверлеи).

**Что такое BackStack и как им управлять?**
BackStack — стек FragmentTransaction, добавленных с addToBackStack(). При нажатии Back система вызывает popBackStack(), отменяя последнюю транзакцию. Управление: addToBackStack(String name) для именованных стеков, popBackStack(), popBackStackImmediate(), popBackStack(String name, flags). В Android BackStack позволяет реализовать навигацию назад.

**Как правильно сохранять состояние Fragment при повороте экрана?**
Состояние Fragment сохраняется автоматически в Bundle, если оно реализует Parcelable/Serializable или хранится в ViewModel. Лучшая практика — использовать ViewModel для данных и SavedStateHandle для простых значений. Избегать setRetainInstance(true) — устарел. В onSaveInstanceState() сохранять только критические UI-состояния (позиция скролла), остальное — в ViewModel.

**Что такое setRetainInstance() и почему его не рекомендуют?**
setRetainInstance(true) указывает FragmentManager не уничтожать Fragment при смене конфигурации (поворот экрана), сохраняя его экземпляр. Однако метод устарел с AndroidX Fragment 1.3.0, так как нарушает принципы жизненного цикла и может привести к утечкам памяти. Рекомендуется использовать ViewModel для хранения состояния — он переживает конфигурационные изменения без хаков.

**Жизненный цикл DialogFragment.**
DialogFragment наследует Fragment, но добавляет диалоговое окно. Порядок: onCreateDialog() (создание Dialog), onCreateView() (если custom view), onViewCreated(), onStart() (диалог показан), onResume() (активен), onPause(), onStop(), onDestroyView(), onDestroy(). При повороте DialogFragment пересоздаётся, но состояние сохраняется через ViewModel или savedInstanceState.

**Как правильно использовать Fragment Result API?**
Fragment Result API — механизм передачи результатов от одного Fragment к другому без прямой зависимости. Используется FragmentManager .setFragmentResultListener() в получателе и setFragmentResult() в отправителе с уникальным requestKey. Работает через Bundle и поддерживает жизненный цикл. В Android это современная альтернатива startActivityForResult и interface-callback.

**Что такое Fragment Container View?**
FragmentContainerView — специальный ViewGroup, предназначенный для размещения Fragment'ов в XML-лейауте. Он заменяет <fragment> и обеспечивает правильную работу с FragmentManager, поддерживает animateLayoutChanges и navGraph. Рекомендуется использовать вместо FrameLayout для хоста Fragment'ов.

**Как работает ViewPager2 с FragmentStateAdapter?**
ViewPager2 использует RecyclerView внутри и FragmentStateAdapter для работы с Fragment'ами. FragmentStateAdapter создаёт и уничтожает Fragment'ы по мере прокрутки (сохраняет состояние через savedInstanceState), что экономит память по сравнению с устаревшим FragmentPagerAdapter. В конструкторе передаётся FragmentActivity или Fragment, реализуется getItemCount() и createFragment(position). ViewPager2 автоматически управляет жизненным циклом Fragment'ов и их сохранением при смене конфигурации.

**Что такое Service и зачем он нужен?**
Service — это компонент Android, предназначенный для выполнения длительных операций в фоновом режиме без видимого пользовательского интерфейса. В отличие от Activity, Service может работать даже когда приложение не на переднем плане, что делает его подходящим для задач, таких как воспроизведение музыки, загрузка файлов, синхронизация данных, обработка уведомлений или выполнение сетевых запросов. Service работает в том же процессе, что и приложение (по умолчанию), но может быть запущен в отдельном процессе. Основная цель Service — обеспечить продолжение работы даже после закрытия экрана, при этом система имеет право убить его при нехватке ресурсов, если он не является foreground-сервисом.

**Разница между Started Service и Bound Service?**
Started Service запускается через startService() и продолжает работать до тех пор, пока не будет остановлен вызовом stopSelf() или stopService(). Он не зависит от компонента, который его запустил, и может работать даже после уничтожения вызвавшего Activity. Bound Service работает через привязку (bindService()), создавая двустороннюю связь с клиентом (Activity, Fragment или другой Service) через Binder или Messenger. Bound Service уничтожается автоматически, когда все клиенты от него отсоединяются. Started Service подходит для независимых фоновых задач, Bound Service — для взаимодействия с клиентом (например, получение данных в реальном времени).

**Что такое Foreground Service и как его правильно запустить?**
Foreground Service — это тип Service, который должен быть видимым пользователю и имеет повышенный приоритет, поэтому система реже его убивает. Чтобы запустить Foreground Service, необходимо вызвать startForeground() в течение 5 секунд после создания, передав Notification с приоритетом FOREGROUND_SERVICE. С Android 8.0 (Oreo) для запуска требуется разрешение FOREGROUND_SERVICE и использование startForegroundService() вместо startService(). Правильный запуск включает создание уведомления с каналом, установку foreground-состояния и обработку остановки через stopForeground(true). Foreground Service используется для музыки, навигации, загрузки файлов и других длительных видимых операций.

**Какие ограничения на Foreground Service появились после Android 8.0?**
После Android 8.0 (Oreo) введены жёсткие ограничения на фоновую работу: обычные Background Service больше нельзя запускать из фона (только через JobScheduler или WorkManager). Foreground Service требует явного вызова startForegroundService() и обязательного перехода в foreground-состояние в течение 5 секунд. С Android 10+ добавлены ограничения на типы Foreground Service (camera, microphone, location и т.д.), а с Android 12+ — ещё более строгие правила (Foreground Service must have a notification and cannot be started from background in most cases). Также введены квоты на запуск и автоматическое удаление уведомления при остановке.

**Что такое IntentService и почему он устарел?**
IntentService — это устаревший подкласс Service, который автоматически обрабатывает входящие Intent в отдельном рабочем потоке (Worker Thread) и останавливается сам после завершения всех задач. Он был удобен для последовательной обработки запросов в фоне. IntentService устарел с Android 8.0, так как не поддерживает современные ограничения на фоновую работу, не имеет гибкой отмены и не интегрируется с Jetpack. Google рекомендует заменять его на WorkManager или CoroutineWorker для фоновых задач.

**Как работает JobIntentService и WorkManager?**
JobIntentService — это устаревшая обёртка над JobScheduler, которая позволяла запускать Intent в фоне даже после Android 8.0, используя JobService под капотом. Она автоматически выбирала между JobScheduler и обычным Service в зависимости от версии Android. WorkManager — современная замена, представляющая собой гибкую библиотеку для планирования отложенных и гарантированных задач. WorkManager поддерживает ограничения (сеть, заряд батареи), цепочки задач, повторяющиеся работы и работает даже после перезагрузки устройства. Он использует JobScheduler, AlarmManager или Firebase JobDispatcher в зависимости от версии.

**Что такое Bound Service и как с ним работать через Binder?**
Bound Service — это Service, к которому клиент (Activity, Fragment или другой Service) привязывается через bindService(). Связь осуществляется через Binder — интерфейс, позволяющий клиенту вызывать методы Service напрямую. Реализация: в Service переопределяется onBind(), возвращается IBinder (чаще всего Binder subclass или AIDL). Клиент получает IBinder в ServiceConnection.onServiceConnected() и может вызывать методы. Bound Service уничтожается, когда все клиенты отсоединяются (unbindService()). Это подходит для сценариев, где нужен двусторонний обмен данными.

**Разница между Local Service и Remote Service?**
Local Service работает в том же процессе, что и приложение, и Binder используется для прямого вызова методов (обычно через внутренний Binder-класс). Remote Service работает в отдельном процессе (android:process=":remote"), и общение происходит через AIDL (Android Interface Definition Language) или Messenger. Local Service проще и быстрее, но не изолирован. Remote Service обеспечивает изоляцию (процесс может быть убит отдельно), но требует сериализации данных и сложнее в реализации. В Android Remote Service используется редко из-за overhead.

**Как правильно останавливать Service (stopSelf, stopService)?**
Остановка Service зависит от его типа. Для Started Service используется stopSelf() внутри Service (когда работа завершена) или stopService(Intent) из клиента. stopSelf() останавливает Service только если нет pending Intent. Для Bound Service остановка происходит автоматически при отсоединении всех клиентов (unbindService()). Для Foreground Service обязательно вызвать stopForeground(true) перед stopSelf() или stopService(). Правильная практика — проверять isBound и isStarted, чтобы избежать утечек.

**Что такое onStartCommand() и какие возвращаемые значения существуют?**
onStartCommand(Intent intent, int flags, int startId) — основной метод Started Service, вызываемый при каждом startService(). Возвращаемые значения: START_STICKY — Service перезапускается с null Intent после kill; START_NOT_STICKY — не перезапускается; START_REDELIVER_INTENT — перезапускается с последним Intent; START_STICKY_COMPATIBILITY — совместимость со старыми версиями. Выбор зависит от того, нужно ли перезапустить Service после убийства системой (например, для музыки — START_STICKY).

**Как Service работает после kill приложения?**
После убийства приложения (swipe из недавних или OOM Killer) Started Service может быть перезапущен системой, если он возвращал START_STICKY или START_REDELIVER_INTENT. Foreground Service имеет высокий приоритет и перезапускается чаще. Bound Service уничтожается вместе с клиентом. WorkManager и JobScheduler продолжают работу даже после kill приложения благодаря системе. В Android 8+ фоновые Service сильно ограничены, поэтому для длительных задач рекомендуется Foreground Service или WorkManager.

**Что такое Sticky Service и Non-Sticky?**
Sticky Service — это Service, который система пытается перезапустить после убийства, передавая null Intent (START_STICKY). Non-Sticky Service (START_NOT_STICKY) не перезапускается автоматически. Sticky используется для сервисов, которые должны продолжать работу (музыка, загрузка), Non-Sticky — для разовых задач. В практике Sticky Service чаще всего используется с Foreground Service.

**Как использовать Foreground Service для длинных операций (музыка, загрузка)?**
Foreground Service используется для длительных видимых операций (воспроизведение музыки, скачивание файлов, навигация). Запуск: startForegroundService(Intent), затем в течение 5 секунд вызвать startForeground(notificationId, notification). Уведомление должно быть видимым и информативным. Для музыки — обновлять уведомление при смене трека. Для загрузки — показывать прогресс. С Android 10+ требуется явное разрешение FOREGROUND_SERVICE_TYPE_... в манифесте.

**Какие ограничения на Background Service в Android 8+ и 12+?**
С Android 8.0 (Oreo) Background Service нельзя запускать из фона (только через JobScheduler/WorkManager). С Android 12+ введены ещё более строгие ограничения: Foreground Service должен иметь явный тип (FOREGROUND_SERVICE_TYPE_MEDIA, LOCATION и т.д.) в манифесте и уведомлении. Запуск Background Service из невидимого приложения почти всегда блокируется. Это сделано для экономии батареи и повышения безопасности.

**Как правильно использовать WorkManager вместо Service?**
WorkManager — рекомендуемый способ для отложенных и гарантированных фоновых задач. Вместо Service создаётся Worker (CoroutineWorker для корутин), настраиваются Constraints (сеть, заряд, устройство не в Doze), BackoffPolicy и ExistingWorkPolicy. WorkManager автоматически выбирает оптимальный механизм (JobScheduler, AlarmManager). Используется для синхронизации данных, загрузки файлов, отправки аналитики. Преимущество — гарантированное выполнение даже после перезагрузки устройства.

**Что такое BroadcastReceiver и какие виды бывают?**
BroadcastReceiver — компонент Android, предназначенный для получения и обработки широковещательных сообщений (broadcast). Виды: статический (зарегистрирован в манифесте) и динамический (зарегистрирован через registerReceiver() в коде). Также различают системные broadcast (от системы) и пользовательские (от приложений). Статические receiver работают даже когда приложение не запущено, динамические — только когда приложение активно.

**Разница между статическим и динамическим BroadcastReceiver?**
Статический BroadcastReceiver объявляется в AndroidManifest.xml и может получать broadcast даже когда приложение не запущено (но с Android 8+ многие системные broadcast заблокированы). Динамический регистрируется в коде (registerReceiver) и работает только пока компонент активен (Activity/Fragment/Service). Динамический предпочтительнее для большинства случаев, так как позволяет гибко управлять подпиской и избегать утечек.

**Что такое Implicit и Explicit Broadcast?**
Implicit Broadcast — broadcast, отправленный без указания конкретного получателя (через Intent с action). Система доставляет его всем приложениям, зарегистрированным на этот action. Explicit Broadcast — отправляется с указанием конкретного компонента (package + class). С Android 8+ многие implicit broadcast запрещены для background приложений в целях безопасности и экономии батареи.

**Какие системные broadcast'ы существуют и какие из них больше не работают?**
Системные broadcast включают ACTION_BOOT_COMPLETED, ACTION_BATTERY_LOW, ACTION_PACKAGE_ADDED и многие другие. После Android 8.0 большинство implicit системных broadcast больше не доставляются приложениям в фоне (например, CONNECTIVITY_CHANGE, ACTION_NEW_PICTURE). Разрешены только несколько (например, ACTION_BOOT_COMPLETED с разрешениями). Это сделано для снижения энергопотребления и предотвращения злоупотреблений.

**Как правильно регистрировать и отменять регистрацию BroadcastReceiver?**
Динамический BroadcastReceiver регистрируется в onStart() или onResume() через registerReceiver() и обязательно отменяется в onStop() или onPause() через unregisterReceiver(). Статический — в манифесте, но с Android 8+ многие не работают. Лучшая практика — использовать LocalBroadcastManager для внутренних broadcast внутри приложения, чтобы избежать утечек и ненужных системных вызовов.

**Что такое LocalBroadcastManager и зачем он нужен?**
LocalBroadcastManager — класс из support library (сейчас androidx.localbroadcastmanager), который позволяет отправлять broadcast только внутри одного приложения. Он быстрее и безопаснее обычного BroadcastReceiver, так как не выходит за пределы процесса и не требует разрешений. Используется для внутренних событий (например, обновление UI после загрузки данных). Сейчас рекомендуется заменять на SharedFlow или Channel.

**Как работает Ordered Broadcast?**
Ordered Broadcast — broadcast, который доставляется получателям по приоритету (android:priority в манифесте). Каждый получатель может прервать дальнейшую доставку через abortBroadcast(). Это позволяет создать цепочку обработки (например, система → приложение → другое приложение). Ordered broadcast используется для системных событий, где важен порядок (например, SMS).

**Что такое Sticky Broadcast?**
Sticky Broadcast — broadcast, который остаётся в системе после доставки, и новый зарегистрированный receiver может его получить позже. Создаётся через sendStickyBroadcast(). С Android 5.0+ sticky broadcast сильно ограничен из-за безопасности, и большинство системных sticky broadcast устарели. Сейчас почти не используется.

**Как отправить broadcast с разрешениями?**
Broadcast отправляется с разрешением через sendBroadcast(Intent, String permission). Только приложения, имеющие указанное разрешение, смогут его получить. Это позволяет защитить broadcast от посторонних приложений. В Android 8+ для implicit broadcast требуется дополнительная настройка.

**Какие ограничения на BroadcastReceiver после Android 8.0?**
После Android 8.0 (Oreo) большинство implicit BroadcastReceiver больше не работают в фоне. Статические receiver для системных broadcast заблокированы, за исключением нескольких разрешённых (например, BOOT_COMPLETED с разрешениями). Динамические receiver работают только когда приложение активно. Это сделано для снижения энергопотребления и предотвращения злоупотреблений.

**Что такое ContentProvider и зачем он нужен?**
ContentProvider — компонент Android, предназначенный для безопасного обмена данными между приложениями. Он предоставляет унифицированный интерфейс (URI-based) для доступа к данным (файлы, базы данных, контакты). ContentProvider используется системой (Contacts, MediaStore) и позволяет другим приложениям читать/писать данные с контролем разрешений. В собственном приложении ContentProvider нужен для экспорта данных другим приложениям или для использования с FileProvider.

**Как работает URI в ContentProvider?**
URI в ContentProvider — это унифицированный идентификатор ресурса в формате content://authority/path/id. Authority — уникальное имя провайдера (обычно package name), path — путь к данным (например, /users), id — конкретная запись. ContentResolver использует URI для определения, какой провайдер должен обработать запрос. Это позволяет абстрагировать источник данных (база, файл, сеть) и обеспечивать безопасность через разрешения.

**Что такое FileProvider и как его использовать?**
FileProvider — специальный ContentProvider от Google, предназначенный для безопасного предоставления доступа к файлам приложения другим приложениям (например, для отправки фото через Intent). Он заменяет прямой доступ к file:// URI, который запрещён с Android 7.0 (FileUriExposedException). Настраивается в манифесте с путями (res/xml/file_paths.xml) и используется через FileProvider.getUriForFile(). Это позволяет делиться файлами без риска безопасности.

**Как правильно реализовать CRUD-операции в ContentProvider?**
CRUD-операции в ContentProvider реализуются через переопределение методов query(), insert(), update(), delete() и getType(). Каждый метод получает URI и параметры. Для query() возвращается Cursor, для insert() — URI новой записи. Важно реализовать уведомления через ContentResolver.notifyChange() для автоматического обновления наблюдателей. В Room это делается автоматически через @Dao, но при ручной реализации ContentProvider нужно вручную обрабатывать URI matching (UriMatcher).

**Что такое ContentResolver и как с ним работать?**
ContentResolver — класс, предоставляющий унифицированный интерфейс для взаимодействия с ContentProvider. Через ContentResolver вызываются query(), insert(), update(), delete() по URI. Он управляет разрешениями и уведомлениями. В Android ContentResolver используется для доступа к Contacts, MediaStore, Calendar и собственным провайдерам. Работа строится через getContentResolver().query(uri, projection, selection, selectionArgs, sortOrder).

**Как защитить ContentProvider (разрешения, exported)?**
ContentProvider защищается через атрибут android:exported="false" (по умолчанию false с Android 4.2+), что запрещает доступ извне. Для экспорта используются custom permissions (android:permission) и android:grantUriPermissions. Рекомендуется использовать FileProvider с временными URI (grantUriPermission) вместо постоянного экспорта. Для чувствительных данных — проверка вызывающего UID и ограничение операций.

**Что такое Stub ContentProvider?**
Stub ContentProvider — минимальная реализация ContentProvider, используемая для инициализации Application в multi-process приложениях или для dummy-провайдера. Он позволяет запустить процесс без реальной логики. Часто используется в библиотечных модулях или когда нужно зарегистрировать провайдер в манифесте, но без функционала.

**Как тестировать ContentProvider?**
ContentProvider тестируется через AndroidJUnit4 с ProviderTestRule или InstrumentationTestCase. Используется MockContentResolver для изоляции. Тесты проверяют CRUD-операции, уведомления и разрешения. Для Room-based ContentProvider тестируют через in-memory базу. Важно проверять edge-кейсы: пустые данные, конфликты, разрешения.

**Разница между Normal, Dangerous и Signature permissions?**
В Android разрешения делятся на несколько категорий по уровню риска и способу предоставления. Normal permissions — это низкорисковые разрешения (например, INTERNET, ACCESS_NETWORK_STATE), которые автоматически выдаются при установке приложения и не требуют явного согласия пользователя. Dangerous permissions — это высокорисковые разрешения (например, CAMERA, LOCATION, STORAGE, CONTACTS), которые могут дать доступ к приватным данным или аппаратным ресурсам; они требуют явного согласия пользователя во время работы приложения (runtime permission). Signature permissions — это специальные разрешения, которые выдаются автоматически только приложениям, подписанным тем же сертификатом, что и приложение, запрашивающее разрешение (используются для межприложного взаимодействия внутри одной команды разработчика). Signature permissions обеспечивают высокий уровень доверия между приложениями одного разработчика.

**Как работает Runtime Permission в Android 6.0+?**
Runtime Permission (введён в Android 6.0 Marshmallow) — это механизм, при котором опасные разрешения (Dangerous) запрашиваются у пользователя не при установке, а во время работы приложения, когда они действительно нужны. При вызове метода requestPermissions() система показывает диалог с объяснением, зачем требуется разрешение. Пользователь может разрешить или отклонить запрос. Приложение должно проверять наличие разрешения через checkSelfPermission() перед использованием соответствующей функции. Если разрешение было отклонено, можно показать rationale (объяснение) и предложить зайти в настройки. Разрешения можно отозвать в любой момент в настройках приложения, поэтому код должен gracefully обрабатывать отказ.

**Что такое Permission Group и как она работает?**
Permission Group — это логическая группировка связанных Dangerous permissions, введённая для упрощения пользовательского опыта. Например, группа CONTACTS включает READ_CONTACTS и WRITE_CONTACTS, группа LOCATION включает ACCESS_FINE_LOCATION и ACCESS_COARSE_LOCATION. Когда пользователь даёт разрешение на одну permission из группы, система автоматически выдаёт все разрешения этой группы. Это упрощает управление, но также означает, что отказ в одном разрешении группы влияет на всю группу. В коде всё равно нужно проверять каждое конкретное разрешение отдельно.

**Как правильно запрашивать несколько разрешений?**
Правильный способ запрашивать несколько разрешений — использовать метод requestPermissions(activity, arrayOfPermissions, requestCode), передавая массив строк с нужными разрешениями. Перед запросом рекомендуется проверить, какие разрешения уже выданы (checkSelfPermission), и показать пользователю rationale (shouldShowRequestPermissionRationale), если ранее было отказано. После получения результата в onRequestPermissionsResult() нужно обработать каждый результат отдельно, так как пользователь может разрешить одни и отказать в других. Лучшая практика — запрашивать разрешения постепенно, по мере необходимости, а не все сразу при запуске приложения.

**Что такое android:exported и как оно влияет на безопасность?**
android:exported — атрибут в AndroidManifest.xml, который определяет, может ли компонент (Activity, Service, BroadcastReceiver, ContentProvider) быть вызван из других приложений. Если exported="true" (по умолчанию для Activity с intent-filter), компонент доступен извне. Если exported="false", доступ возможен только внутри приложения. С Android 12+ этот атрибут стал обязательным для всех компонентов. Неправильная настройка exported="true" без необходимости может привести к уязвимостям (Intent Redirection, несанкционированный запуск Activity/Service), поэтому рекомендуется явно устанавливать exported="false" для внутренних компонентов и использовать разрешения для внешних.

**Что такое PendingIntent и почему он опасен?**
PendingIntent — это объект, который представляет собой «отложенное намерение» (Intent), который может быть запущен позже другим приложением или системой от имени вашего приложения. Он содержит Intent и контекст безопасности (UID приложения-создателя). PendingIntent опасен, потому что получатель (например, NotificationManager или AlarmManager) может запустить этот Intent с правами вашего приложения, даже если он находится в другом процессе. Это открывает вектор атак Intent Redirection и privilege escalation. Чтобы снизить риск, нужно использовать FLAG_IMMUTABLE (Android 12+) и явно указывать компонент.

**Как защитить приложение от Intent Redirection?**
Intent Redirection — это атака, при которой злоумышленник заставляет ваше приложение запустить вредоносный Intent от вашего имени. Защита включает: использование explicit Intent вместо implicit где возможно, установку android:exported="false" для внутренних компонентов, использование FLAG_IMMUTABLE для PendingIntent (Android 12+), проверку входящих Intent в onNewIntent() и onCreate(), использование PendingIntent.getActivity() с явным компонентом. Также рекомендуется использовать App Links вместо обычных deep links и проверять caller UID при получении Intent от других приложений.

**Что такое SafetyNet / Play Integrity API?**
SafetyNet Attestation API (теперь заменён на Play Integrity API) — это сервис Google, позволяющий проверить целостность устройства и приложения. Он определяет, является ли устройство настоящим (не эмулятор), не рутировано ли оно, не работает ли в unsafe окружении и не модифицировано ли приложение. Play Integrity API возвращает вердикт (MEETS_DEVICE_INTEGRITY, MEETS_BASIC_INTEGRITY, MEETS_STRONG_INTEGRITY) и токен, который можно проверить на сервере. В Android это используется для защиты от мошенничества, особенно в банковских и игровых приложениях.

**Как работает App Sandbox в Android?**
App Sandbox — это механизм изоляции приложений на уровне Linux Kernel. Каждому приложению присваивается уникальный User ID (UID), и оно запускается в отдельном процессе с собственным адресным пространством. Приложение имеет доступ только к своим файлам в /data/data/<package_name> и не может читать данные других приложений без явных разрешений. Sandbox также ограничивает доступ к системным ресурсам через permissions. Даже если приложение скомпрометировано, оно не может легко повлиять на другие приложения или систему благодаря этой изоляции. Это основа безопасности Android.

**Что такое SELinux и как он защищает приложение?**
SELinux (Security-Enhanced Linux) — это модуль обязательного контроля доступа (MAC), встроенный в Linux Kernel и активно используемый в Android. Он работает по принципу «по умолчанию запрещено»: даже если процесс имеет права (через UID и permissions), SELinux-политики могут запретить конкретные действия (чтение определённых файлов, Binder-вызовы, запуск процессов). В Android SELinux работает в enforcing-режиме и защищает системные сервисы, файловую систему и межпроцессное взаимодействие. Это существенно снижает ущерб от уязвимостей и эксплойтов, даже если приложение получило root-доступ.

**Что такое Task Affinity?**
Task Affinity — это механизм, определяющий, к какой задаче (Task) должна принадлежать Activity. По умолчанию affinity равен имени пакета приложения. Если у двух Activity разный affinity, они могут быть запущены в разных задачах. Affinity используется совместно с launchMode для контроля поведения стека. Например, можно задать android:taskAffinity="com.example.another_task", чтобы Activity открывалась в отдельной задаче. Это влияет на то, как работает кнопка «Recent Apps» и поведение при запуске из лаунчера.

**Как работает Deep Link и App Link?**
Deep Link — это механизм, позволяющий открывать конкретный экран приложения по специальной ссылке (http/https или custom scheme). App Link — это улучшенная версия Deep Link, использующая только HTTPS-ссылки, верифицированные через Digital Asset Links (файл assetlinks.json на сайте). При клике по ссылке Android проверяет, поддерживает ли приложение этот домен, и открывает соответствующий экран без выбора приложения. Deep Link настраивается через intent-filter в манифесте с android:autoVerify="true" для App Links.

**Что такое SplashScreen API (Android 12+)?**
SplashScreen API — это официальный API, введённый в Android 12, для создания красивого и унифицированного экрана загрузки приложения. Он автоматически показывает системный splash screen с иконкой приложения и анимацией, пока приложение не готово к отображению первого экрана. Разработчик может кастомизировать цвет фона, анимацию иконки и длительность через themes.xml (windowSplashScreenBackground, windowSplashScreenAnimatedIcon). API обеспечивает плавный переход от splash screen к первому Activity и решает проблемы с кастомными splash-экранами в старых версиях.

**Как работает App Startup Library?**
App Startup Library (androidx.startup) — это Jetpack-библиотека, позволяющая выполнять инициализацию компонентов (например, Timber, Crashlytics, WorkManager) при старте приложения в контролируемом порядке. Вместо ручной инициализации в Application.onCreate() компоненты регистрируются через <meta-data> в манифесте и реализуют Initializer<T>. Библиотека гарантирует правильный порядок инициализации (через зависимости) и lazy-выполнение. Это упрощает код Application-класса, улучшает производительность запуска и делает инициализацию тестируемой.

**Что такое Baseline Profiles и как они ускоряют приложение?**
Baseline Profiles — это профили производительности, которые разработчик создаёт для ключевых путей выполнения приложения (например, запуск экрана, прокрутка списка). Эти профили включаются в APK/AAB и используются ART (Android Runtime) для ahead-of-time компиляции критических методов при установке приложения. Это значительно ускоряет первый запуск и холодный старт приложения, уменьшая время JIT-компиляции. Baseline Profiles особенно эффективны для сложных приложений с большим количеством кода и улучшают пользовательский опыт на низко- и среднебюджетных устройствах.

**Как работает App Bundle (AAB) и Dynamic Feature Modules?**
App Bundle (AAB) — это формат публикации приложений в Google Play, который содержит весь код и ресурсы, но не является готовым APK. Google Play генерирует оптимизированные APK для каждого устройства (разделяя по архитектуре, экрану, языку). Dynamic Feature Modules — это модули, которые можно загружать по требованию после установки основного приложения. Они позволяют уменьшить размер начальной установки и загружать дополнительные функции (например, модуль камеры или AR) только когда они нужны. AAB + Dynamic Feature Modules значительно снижают размер приложения и улучшают конверсию.

**Что такое StrictMode и как его использовать?**
StrictMode — это инструмент для обнаружения нарушений лучших практик в Android (например, сетевые запросы или дисковые операции на Main Thread, утечки объектов, неправильная работа с Cursor). Он включается в debug-сборке в Application.onCreate() через StrictMode.setThreadPolicy() и StrictMode.setVmPolicy(). При обнаружении нарушения StrictMode либо логирует ошибку, либо бросает исключение. Это помогает выявлять проблемы производительности и утечки памяти на этапе разработки. В продакшене StrictMode обычно отключается.

**Как работает Android Profiler (CPU, Memory, Network)?**
Android Profiler — это инструмент в Android Studio, позволяющий в реальном времени анализировать производительность приложения. CPU Profiler показывает использование процессора, потоки, вызовы методов и flame graphs. Memory Profiler отслеживает выделение памяти, garbage collection, утечки и heap dump. Network Profiler показывает все сетевые запросы, размер трафика и время выполнения. Инструмент подключается к устройству или эмулятору и позволяет записывать сессии, анализировать bottlenecks и находить проблемы (например, частые GC или долгие сетевые запросы). Это один из главных инструментов для оптимизации Android-приложений.

**Как работает View Binding и Data Binding?**
View Binding — это библиотека, которая генерирует классы для прямого доступа к View из XML без findViewById(). Для каждого layout-файла создаётся класс с полями, соответствующими ID элементов, что устраняет риски NullPointerException и повышает производительность. Data Binding идёт дальше: оно позволяет связывать данные напрямую с XML (через @{expression}), поддерживает two-way binding и наблюдаемые данные (LiveData, ObservableField). View Binding рекомендуется для большинства случаев, Data Binding — когда нужна сложная привязка данных и логика в XML. Обе технологии генерируются на этапе компиляции.

**Что такое Jetpack App Startup и зачем он нужен?**
Jetpack App Startup — это библиотека, которая позволяет выполнять инициализацию компонентов (например, Crashlytics, Timber, WorkManager, Firebase) при старте приложения в контролируемом и оптимизированном порядке. Вместо ручной инициализации в Application.onCreate() компоненты регистрируются через Initializer и манифест, а библиотека гарантирует правильную последовательность и lazy-инициализацию. Это уменьшает время запуска приложения, упрощает код Application-класса и делает инициализацию тестируемой. App Startup особенно полезен в больших приложениях с множеством библиотек.

# Android Jetpack

**Что такое Android Jetpack и зачем он был создан?**
Android Jetpack — это современный набор библиотек, инструментов и рекомендаций от Google, предназначенный для упрощения разработки качественных Android-приложений. Он был создан для решения нескольких ключевых проблем: фрагментированности платформы, сложностей с жизненным циклом компонентов, отсутствия единого подхода к архитектуре и частых изменений в API. Jetpack объединяет лучшие практики Google под одним брендом, предоставляя разработчикам готовые, lifecycle-aware и модульные решения (ViewModel, LiveData, Navigation, Room, WorkManager, Hilt и др.). Основная цель — позволить разработчикам сосредоточиться на бизнес-логике приложения, а не на борьбе с особенностями Android, при этом обеспечивая совместимость с разными версиями ОС и улучшение производительности.

**Какие основные компоненты входят в Jetpack?**
Jetpack разделён на четыре основные категории: Foundation (AppCompat, Android KTX, Multidex), Architecture (ViewModel, LiveData, Navigation, Room, WorkManager, Hilt, Paging), Behavior (CameraX, Media, Notifications, Permissions, DownloadManager) и UI (Compose, Material Design, Animation, Fragment, RecyclerView). Наиболее важными для middle-разработчика считаются компоненты Architecture (ViewModel + StateFlow, Navigation, Room, WorkManager) и UI (Jetpack Compose). Все компоненты Jetpack являются lifecycle-aware, модульными и тесно интегрированы между собой, что позволяет строить современную архитектуру приложения с минимальным boilerplate-кодом.

**Чем Jetpack отличается от Support Library?**
Support Library (сейчас androidx) была создана для обеспечения обратной совместимости новых API на старых версиях Android. Jetpack — это эволюция и значительное расширение этой идеи. В отличие от Support Library, которая в основном решала проблемы совместимости, Jetpack предлагает полноценные архитектурные решения, лучшие практики и готовые компоненты для построения современного приложения (ViewModel, Navigation Component, Room, WorkManager, Hilt). Jetpack более модульный, активно развивается, имеет встроенную поддержку корутин и Compose, а также следует принципам lifecycle-awareness и Single Source of Truth. Фактически Jetpack — это «новое поколение» Support Library с гораздо более широким охватом задач.

**Какие принципы лежат в основе Jetpack (Lifecycle-aware, Modular, etc.)?**
Jetpack построен на нескольких ключевых принципах: Lifecycle-aware (компоненты автоматически реагируют на жизненный цикл Activity/Fragment), Modular (каждый компонент можно подключать независимо), Consistent (единый стиль API и поведения), Backward compatible (работает на старых версиях Android), и Testable (компоненты легко тестировать). Ещё один важный принцип — Unbundled: библиотеки Jetpack обновляются независимо от версии Android, что позволяет быстро получать новые возможности без ожидания обновления ОС. Всё это направлено на то, чтобы разработчик мог сосредоточиться на логике приложения, а не на борьбе с особенностями платформы.

**Как Jetpack помогает решать проблемы с жизненным циклом?**
Jetpack решает классические проблемы жизненного цикла Android через lifecycle-aware компоненты. ViewModel переживает поворот экрана и process death, автоматически сохраняя состояние. LiveData/StateFlow автоматически управляют подпиской/отпиской в зависимости от состояния lifecycle (активен/неактивен). repeatOnLifecycle и lifecycleScope позволяют безопасно собирать Flow только в нужных состояниях. WorkManager и Hilt тоже учитывают жизненный цикл. Благодаря этому разработчик избавляется от ручного управления onSaveInstanceState, onDestroy, отписок от BroadcastReceiver и подписок, что существенно снижает количество утечек памяти и ошибок.

**Что такое Jetpack Compose и почему Google активно его продвигает?**
Jetpack Compose — это современный declarative UI toolkit от Google, который позволяет описывать интерфейс с помощью аннотированных функций (@Composable), а не через XML. Он полностью заменяет традиционную систему View и ViewGroup. Google активно продвигает Compose, потому что он значительно ускоряет разработку UI, упрощает state management, делает код более декларативным и предсказуемым, лучше интегрируется с Kotlin и Coroutines, а также позволяет создавать сложные анимации и кастомные компоненты с меньшим объёмом кода. С 2023–2024 годов Compose стал рекомендуемым способом создания интерфейсов, и большинство новых проектов Google (включая внутренние) строятся именно на нём.

**Какие компоненты Jetpack считаются устаревшими в 2025 году?**
В 2025 году устаревшими считаются: AsyncTask (полностью deprecated), Loader (заменён на ViewModel + Flow), Android Support Library (перешёл в AndroidX), старый Navigation Component с XML (хотя всё ещё используется в legacy), LiveData (рекомендуется переходить на StateFlow), RxJava (в новых проектах уступает Coroutines), старый View Binding (хотя всё ещё поддерживается). Также устаревает прямое использование Handler и Looper в пользу Coroutines. Google активно рекомендует переходить на Jetpack Compose вместо XML + Data Binding, и на WorkManager вместо JobScheduler/AlarmManager для большинства фоновых задач.

**Что такое ViewModel и зачем она нужна?**
ViewModel — это класс из Jetpack Architecture Components, предназначенный для хранения и управления UI-состоянием, которое должно пережить конфигурационные изменения (поворот экрана) и Process Death. В отличие от Activity или Fragment, ViewModel не уничтожается при пересоздании экрана, что позволяет сохранять данные без использования onSaveInstanceState. Она также отделяет бизнес-логику и данные от UI, улучшая тестируемость и поддерживаемость кода. ViewModel обычно работает в паре с StateFlow или LiveData, предоставляя наблюдаемое состояние для UI.

**Как ViewModel переживает configuration changes и Process Death?**
ViewModel переживает configuration changes (поворот экрана, изменение языка) благодаря ViewModelStoreOwner (Activity или Fragment). При пересоздании экрана система сохраняет ViewModel в ViewModelStore, и новый экземпляр Activity/Fragment получает ту же самую ViewModel. При Process Death ViewModel уничтожается, но если использовать SavedStateHandle, критические данные можно восстановить из Bundle. Таким образом, ViewModel обеспечивает непрерывность состояния без сложного ручного сохранения.

**Разница между ViewModel и AndroidViewModel?**
ViewModel — базовый класс, не имеющий доступа к Context или Application. AndroidViewModel наследует ViewModel и получает Application в конструкторе, что позволяет использовать Application Context внутри ViewModel (например, для ресурсов или Application-level сервисов). AndroidViewModel рекомендуется только когда действительно нужен Application Context; в остальных случаях лучше использовать обычную ViewModel, чтобы избежать утечек и сохранить чистоту архитектуры.

**Как правильно инициализировать ViewModel (ViewModelProvider, by viewModels())?**
В современном Kotlin рекомендуется использовать делегат by viewModels() (из fragment-ktx или activity-ktx), который автоматически создаёт ViewModel через ViewModelProvider. Для кастомной фабрики используется by viewModels { MyViewModelFactory(...) }. Старый способ через ViewModelProvider напрямую всё ещё работает, но делегат by viewModels() является более идиоматичным и лаконичным. При использовании Hilt применяется @HiltViewModel и делегат by viewModels() без дополнительной фабрики.

**Что такое SavedStateHandle и когда его использовать?**
SavedStateHandle — это объект, который позволяет ViewModel сохранять и восстанавливать состояние при Process Death. Он работает как Bundle, но с типобезопасным API (get(), set(), getLiveData(), getStateFlow()). SavedStateHandle особенно полезен для хранения простых данных (ID выбранного элемента, поисковый запрос, состояние формы), которые должны пережить смерть процесса. Он автоматически сохраняется системой и восстанавливается при пересоздании ViewModel.

**Как правильно использовать StateFlow и SharedFlow в ViewModel?**
В ViewModel рекомендуется использовать MutableStateFlow для хранения состояния экрана (UI State) и SharedFlow для событий (одноразовые действия: показ Snackbar, навигация). StateFlow всегда имеет значение и автоматически фильтрует одинаковые последовательные эмиссии. SharedFlow используется с replay = 0 для событий. Все эмиссии должны происходить на Main Thread или через viewModelScope.launch(Dispatchers.Main). Лучшая практика — выставлять состояние через _uiState.value = ... и наблюдать из UI через collectAsState().

**Что такое UI State и как его правильно проектировать?**
UI State — это sealed class или data class, который полностью описывает текущее состояние экрана в один момент времени (например, Loading, Success(data), Error(message)). Правильный дизайн предполагает immutable состояние, использование StateFlow для его хранения в ViewModel и одного источника правды. UI State должен содержать все данные, необходимые для отображения экрана, включая флаги загрузки, ошибки и навигационные события. Это позволяет легко тестировать и восстанавливать экран после поворота или Process Death.

**Как тестировать ViewModel?**
ViewModel тестируется с помощью JUnit + kotlinx-coroutines-test. Основной инструмент — runTest, внутри которого можно запускать корутины и продвигать время (advanceUntilIdle()). Для StateFlow проверяется начальное значение и последующие эмиссии. Для SharedFlow удобно использовать Turbine. ViewModelFactory и зависимости (Repository) мокаются через MockK или Mockito. Тесты проверяют, что при вызове методов ViewModel правильно обновляет состояние и обрабатывает ошибки.

**Можно ли использовать один ViewModel на несколько Fragment'ов?**
Да, можно и часто рекомендуется использовать один ViewModel на несколько Fragment'ов, если они являются частью одного экрана или тесно связаны (например, ViewPager с табами или Master-Detail). Для этого ViewModel создаётся на уровне Activity (by activityViewModels()) или через Navigation Graph с scoped ViewModel. Это упрощает обмен данными между фрагментами и сохраняет состояние при навигации. Однако важно следить, чтобы ViewModel не становился God-object — он должен отвечать за состояние конкретного экрана.

**Что такое ViewModelFactory и когда он нужен?**
ViewModelFactory — это фабрика, которая создаёт экземпляры ViewModel с параметрами в конструкторе. Она нужна, когда ViewModel требует зависимостей (Repository, UseCase, SavedStateHandle), которые нельзя передать через стандартный конструктор без DI. С Hilt необходимость в ручной фабрике почти исчезает, но иногда она всё ещё используется для сложных случаев или legacy-кода. Factory реализует ViewModelProvider.Factory и переопределяет create().

**Как правильно обрабатывать ошибки в ViewModel?**
Ошибки в ViewModel рекомендуется обрабатывать через sealed class UI State (например, Sealed class UiState<out T> { object Loading : UiState<Nothing>(), data class Success<T>(val data: T) : UiState<T>(), data class Error(val message: String) : UiState<Nothing>() }). При возникновении исключения в корутине состояние переводится в Error. Для повторяющихся ошибок можно использовать SharedFlow<UiEvent> с событием Error. Важно не бросать исключения наружу, а обрабатывать их внутри ViewModel и уведомлять UI через observable состояние. Это делает UI реактивным и предсказуемым.

**Что такое LiveData и чем оно отличается от обычного Observable?**
LiveData — это lifecycle-aware observable класс из Jetpack Architecture Components, предназначенный для хранения и наблюдения за данными, которые автоматически учитывают жизненный цикл компонентов (Activity/Fragment). В отличие от обычного Observable (например, из RxJava или простого LiveData-подобного класса), LiveData автоматически управляет подпиской и отпиской: оно начинает доставлять обновления только когда наблюдатель находится в активном состоянии (STARTED или RESUMED) и прекращает доставку при переходе в DESTROYED. Это предотвращает утечки памяти и ненужные обновления UI. LiveData также поддерживает sticky-поведение — новый наблюдатель сразу получает последнее значение.

**Как LiveData автоматически учитывает жизненный цикл?**
LiveData автоматически учитывает жизненный цикл благодаря тому, что наблюдатель (observer) передаётся вместе с LifecycleOwner (Activity или Fragment). Внутри LiveData используется LifecycleObserver, который отслеживает переходы между состояниями (ON_START, ON_STOP, ON_DESTROY). Когда LifecycleOwner переходит в активное состояние, LiveData начинает доставлять данные; при переходе в неактивное — приостанавливает доставку. При уничтожении LifecycleOwner (onDestroy) LiveData автоматически удаляет наблюдателя, предотвращая утечки. Это делает LiveData безопасным для использования в Fragment и Activity без ручного управления подпиской.

**Разница между LiveData, StateFlow и SharedFlow?**
LiveData — lifecycle-aware observable, который автоматически управляет подпиской в зависимости от состояния LifecycleOwner и всегда доставляет последнее значение новому наблюдателю. StateFlow — это горячий поток из Kotlin Coroutines, который также всегда имеет значение, автоматически фильтрует одинаковые последовательные эмиссии (distinctUntilChanged) и предназначен для представления состояния. SharedFlow — более гибкий горячий поток, который может не иметь начального значения, поддерживает replay (количество последних значений для новых подписчиков) и используется преимущественно для событий (одноразовых уведомлений). LiveData тесно связан с Android Lifecycle, в то время как StateFlow и SharedFlow являются частью Coroutines и лучше интегрируются с современным Kotlin-кодом.

**Когда лучше использовать StateFlow вместо LiveData?**
StateFlow рекомендуется использовать вместо LiveData в новых проектах, особенно когда приложение уже активно использует Kotlin Coroutines. StateFlow предлагает лучшую производительность, встроенную фильтрацию одинаковых значений, более удобную работу с операторами Flow и лучшую интеграцию с Jetpack Compose (collectAsState()). LiveData всё ещё полезен в legacy-проектах или когда требуется строгая lifecycle-awareness без дополнительных зависимостей. StateFlow предпочтительнее для представления UI State в ViewModel, так как он более современный, типобезопасный и легче тестируется с помощью kotlinx-coroutines-test.

**Что такое distinctUntilChanged и как оно работает в Flow?**
distinctUntilChanged — это оператор Flow, который пропускает эмиссию нового значения, если оно равно предыдущему (сравнение через equals()). Он работает на уровне потока и значительно снижает количество ненужных обновлений UI, особенно когда StateFlow или LiveData часто эмитируют одинаковые данные. В контексте ViewModel это особенно полезно для избежания лишних recomposition в Compose или обновлений в RecyclerView. Оператор можно комбинировать с distinctUntilChangedBy() для сравнения по конкретному полю.

**Как правильно использовать repeatOnLifecycle с Flow?**
repeatOnLifecycle — это расширение из lifecycle-runtime-ktx, которое позволяет безопасно собирать Flow только пока LifecycleOwner находится в указанном состоянии (обычно STARTED или RESUMED). Правильное использование: внутри lifecycleScope.launch { repeatOnLifecycle(Lifecycle.State.STARTED) { myFlow.collect { ... } } }. Это автоматически отменяет сбор Flow при переходе в STOPPED и возобновляет при возвращении в STARTED, предотвращая утечки и ненужную работу в фоне. Это стандартный и рекомендуемый способ подписки на Flow в Fragment или Activity.

**Что такое combine, zip, flatMapLatest в контексте ViewModel?**
В контексте ViewModel эти операторы Flow используются для комбинирования нескольких потоков данных. combine() объединяет несколько Flow и эмитирует новое значение каждый раз, когда изменяется любой из них (идеально для объединения userFlow и settingsFlow). zip() ждёт значение от каждого Flow и выдаёт результат только когда все потоки эмитировали по одному значению. flatMapLatest() отменяет предыдущий внутренний Flow при новом значении внешнего (очень полезно для поиска с debounce). Эти операторы позволяют создавать сложное реактивное состояние в ViewModel без большого количества boilerplate-кода.

**Как тестировать LiveData и StateFlow?**
LiveData тестируется через InstantTaskExecutorRule (для запуска на тестовом потоке) и наблюдение за изменениями значения. StateFlow тестируется с помощью runTest из kotlinx-coroutines-test: внутри теста проверяется начальное значение и последующие эмиссии. Для обоих удобно использовать Turbine (для Flow) или просто collect в тестовом scope. Важно использовать TestScope и StandardTestDispatcher для контроля времени и порядка выполнения. Тесты должны проверять как начальное состояние, так и реакцию на вызовы методов ViewModel.

**Почему Google рекомендует переходить с LiveData на StateFlow?**
Google рекомендует переходить с LiveData на StateFlow, потому что StateFlow является частью Kotlin Coroutines, имеет лучшую производительность, встроенную фильтрацию одинаковых значений (distinctUntilChanged), лучшую интеграцию с Jetpack Compose (collectAsState()) и более удобную работу с операторами Flow. LiveData тесно связан с Android Lifecycle, но StateFlow более современный, типобезопасный и легче тестируется. StateFlow также лучше работает с Structured Concurrency и позволяет использовать все возможности Coroutines (combine, debounce, flatMapLatest и т.д.), что делает код чище и более реактивным.

**Что такое Jetpack Navigation Component?**
Jetpack Navigation Component — это библиотека, которая упрощает навигацию между экранами в Android-приложении, предоставляя единый граф навигации, безопасную передачу аргументов и глубокие ссылки. Она состоит из NavGraph (визуальное описание экранов и переходов), NavHost (контейнер для отображения фрагментов) и NavController (класс, управляющий навигацией). Компонент решает проблемы с FragmentTransaction, Back Stack и передачей данных, делая навигацию декларативной и типобезопасной (через Safe Args). Он поддерживает как XML, так и Compose.

**Из каких основных частей состоит Navigation Graph?**
Navigation Graph состоит из трёх основных частей: destinations (экраны — Activity или Fragment), actions (переходы между экранами) и deep links (ссылки для внешнего запуска). Граф может быть вложенным (nested graphs) и содержит информацию о стартовом destination, анимациях переходов и аргументах. В XML-варианте граф редактируется визуально в Navigation Editor, в Compose — декларативно через NavHost.

**Что такое NavHostFragment и NavController?**
NavHostFragment — это специальный Fragment, который выступает контейнером для отображения других Fragment'ов согласно Navigation Graph. NavController — это основной класс, который управляет навигацией: выполняет переходы (navigate()), управляет Back Stack, обрабатывает deep links и аргументы. NavController получается через NavHostFragment.findNavController() или напрямую из Activity/Fragment.

**Как работает Safe Args?**
Safe Args — это Gradle-плагин, который генерирует типобезопасные классы для передачи аргументов между экранами в Navigation Component. Вместо ручной работы с Bundle и ключей-строк генерируются классы с методами типа actionToDetail(id: Int). Это устраняет ошибки с неправильными ключами и типами данных. Safe Args работает как на этапе компиляции, так и предоставляет удобные расширения для Fragment и Activity.

**Как настроить BottomNavigationView с Navigation Component?**
Настройка BottomNavigationView с Navigation Component выполняется через NavController и setupWithNavController(). В layout создаётся BottomNavigationView и NavHostFragment. В коде Activity или Fragment вызывается bottomNavigationView.setupWithNavController(navController). Навигация между вкладками происходит автоматически, а граф должен содержать несколько вложенных графов или отдельные destinations с одинаковым ID для меню. Это обеспечивает правильную работу Back Stack и синхронизацию выбранной вкладки.

**Что такое Nested Navigation Graph?**
Nested Navigation Graph — это вложенный граф навигации внутри основного графа. Он позволяет разбить сложную навигацию на логические модули (например, отдельный граф для профиля, отдельный для настроек). Nested graphs упрощают поддержку больших приложений, позволяют иметь собственный Back Stack внутри вложенного графа и улучшают читаемость. В XML они создаются как отдельные navigation-файлы и включаются в основной граф через <navigation> тег.

**Как правильно передавать данные между Fragment'ами через Navigation?**
Передача данных между Fragment'ами через Navigation Component осуществляется через Safe Args или Bundle. Safe Args — предпочтительный способ: генерирует типобезопасные классы с методами navigate() и аргументами. Для простых случаев можно использовать navController.navigate(direction, Bundle). С AndroidX Navigation 2.4+ поддерживается передача Parcelable и Serializable объектов. Важно избегать прямого доступа к Fragment'ам — данные должны передаваться только через Navigation.

**Что такое NavOptions и как использовать popUpTo?**
NavOptions — это класс, позволяющий настроить поведение навигации: анимации, флаги сохранения состояния, очистку Back Stack. popUpTo() указывает, до какой destination нужно очистить стек при навигации (например, popUpTo(R.id.home) { inclusive = true } очистит всё до home включительно). Это полезно для реализации «возврата на главный экран» или singleTop поведения. NavOptions можно создать через NavOptions.Builder() или использовать в XML.

**Как работать с Navigation в Multi-Activity приложении?**
В Multi-Activity приложении каждый Activity имеет свой NavHostFragment и свой Navigation Graph. Навигация между Activity осуществляется через explicit Intent или deep links. Для единого Back Stack между Activity используется NavController с custom back handling. Рекомендуется минимизировать количество Activity и использовать Single Activity + Navigation Component, но если проект уже multi-activity, то каждый Activity управляет своим графом, а переходы между ними — через startActivity с Intent.

**Как тестировать Navigation Component?**
Тестирование Navigation Component осуществляется через Navigation Testing Library (androidx.navigation.testing). Используется TestNavHostController, который позволяет проверять текущий destination, аргументы и Back Stack. Тесты пишутся с помощью ActivityScenario или FragmentScenario, где проверяется, что navigate() приводит к нужному экрану и передаёт правильные аргументы. Для Compose Navigation используются отдельные тесты с createComposeRule.

**Что такое WorkManager и зачем он нужен?**
WorkManager — это Jetpack-библиотека для планирования и выполнения отложенных, гарантированных фоновых задач, которые должны быть выполнены даже после перезагрузки устройства или смерти приложения. Она нужна для задач, которые не требуют немедленного выполнения (синхронизация данных, загрузка файлов, отправка аналитики), но должны быть надёжно завершены. WorkManager автоматически выбирает оптимальный механизм выполнения (JobScheduler, AlarmManager, Firebase JobDispatcher) в зависимости от версии Android и условий.

**Когда лучше использовать WorkManager вместо Coroutines или Foreground Service?**
WorkManager лучше использовать для отложенных, гарантированных задач, которые должны выполняться даже после перезагрузки устройства или смерти приложения (например, синхронизация данных, отправка отчётов, скачивание обновлений). Coroutines подходят для задач внутри экрана (сетевые запросы, обработка данных), где важна быстрая отмена и интеграция с UI. Foreground Service нужен для длительных видимых операций (музыка, навигация, активная загрузка), где требуется уведомление и высокий приоритет. WorkManager — для «fire and forget» задач с гарантией выполнения.

**Что такое Worker, ListenableWorker, CoroutineWorker?**
Worker — базовый класс для выполнения работы в WorkManager, выполняется синхронно в фоне. ListenableWorker — более гибкая версия, позволяющая возвращать ListenableFuture для асинхронной работы. CoroutineWorker — рекомендуемый класс для Kotlin, который использует suspend-функции и автоматически работает с Coroutines. CoroutineWorker упрощает написание кода, позволяя использовать suspend и withContext внутри doWork().

**Разница между OneTimeWorkRequest и PeriodicWorkRequest?**
OneTimeWorkRequest — это разовая задача, которая выполняется один раз (с возможностью отложенного запуска и backoff). PeriodicWorkRequest — периодическая задача, которая повторяется через заданный интервал (минимум 15 минут). PeriodicWorkRequest не гарантирует точное время выполнения из-за Doze Mode и Battery Optimization. OneTimeWorkRequest подходит для одноразовых действий, Periodic — для регулярных задач (синхронизация раз в день).

**Что такое Constraints в WorkManager?**
Constraints — это условия, при которых WorkManager может запустить задачу (наличие сети, заряд батареи не ниже определённого уровня, устройство на зарядке, не в режиме Doze и т.д.). Constraints позволяют экономить батарею и данные, выполняя работу только в подходящих условиях. Они задаются через Constraints.Builder() и добавляются в WorkRequest.

**Как работает Backoff Policy?**
Backoff Policy определяет стратегию задержки при повторных попытках выполнения работы после неудачи. WorkManager поддерживает два типа: Linear (линейное увеличение задержки) и Exponential (экспоненциальное увеличение). Задержка задаётся через setBackoffCriteria() с начальной задержкой и политикой. Это позволяет постепенно увеличивать интервал между попытками, снижая нагрузку на сервер и устройство при временных ошибках.

**Что такое ExistingWorkPolicy (REPLACE, KEEP, APPEND)?**
ExistingWorkPolicy определяет поведение при попытке поставить новую работу с тем же уникальным именем. REPLACE — заменяет существующую работу на новую. KEEP — оставляет существующую работу и игнорирует новую. APPEND — добавляет новую работу в очередь после существующей. Эти политики важны для избежания дублирования задач (например, синхронизация данных).

**Как создать цепочку работ (chaining)?**
Цепочка работ создаётся через WorkManager.beginWith(oneTimeWorkRequest) .then(nextWorkRequest).enqueue(). Каждая следующая работа запускается только после успешного завершения предыдущей. Можно комбинировать несколько работ параллельно с помощью beginWith(listOfWorks).then(). Работы в цепочке могут передавать данные через InputMerger. Это позволяет создавать сложные последовательности фоновых задач.

**Как передать данные между работами (InputMerger)?**
Передача данных между работами осуществляется через Data (ключ-значение). Каждая работа может возвращать outputData. Для объединения данных из нескольких предыдущих работ используется InputMerger (например, ArrayCreatingInputMerger или OverwritingInputMerger). InputMerger указывается в WorkRequestBuilder.setInputMerger(). Это позволяет собирать результаты предыдущих работ в одной последующей.

**Как тестировать WorkManager?**
Тестирование WorkManager осуществляется через WorkManagerTestInitHelper и SynchronousExecutor для синхронного выполнения. Используется @RunWith(AndroidJUnit4::class) и WorkManager.getInstance(context) с тестовым инициализатором. Для проверки состояния работы используются getWorkInfoByIdLiveData() или await(). Тесты проверяют Constraints, Backoff Policy, входные/выходные данные и порядок выполнения в цепочках.

**Как WorkManager работает после перезагрузки устройства?**
WorkManager сохраняет все запланированные работы в своей внутренней базе данных (Room). После перезагрузки устройства WorkManager автоматически перезапускается через Boot Completed Receiver и восстанавливает все работы. PeriodicWorkRequest и OneTimeWorkRequest с отложенным запуском продолжают выполняться согласно расписанию. Это гарантирует надёжность даже после перезагрузки.

**Что такое Expedited Work (Android 12+)?**
Expedited Work — это тип работы в WorkManager (Android 12+), который позволяет выполнить задачу с повышенным приоритетом, аналогично Foreground Service, но без необходимости показывать уведомление. Expedited Work использует специальный механизм системы для быстрого запуска даже при ограничениях на фоновую работу. Используется для критически важных задач, которые должны быть выполнены как можно скорее (например, отправка важного отчёта). Для Expedited Work требуется setExpedited() в WorkRequest.

**Что такое Hilt и чем он отличается от Dagger?**
Hilt — это официальная библиотека Dependency Injection от Google, построенная поверх Dagger, которая значительно упрощает настройку и использование DI в Android-приложениях. В отличие от чистого Dagger, который требует большого количества boilerplate-кода (компоненты, модули, графы), Hilt использует стандартные аннотации и автоматически генерирует большую часть инфраструктуры. Hilt глубоко интегрирован с Android Lifecycle (Activity, Fragment, ViewModel, Service), предоставляет готовые scopes и значительно снижает сложность конфигурации. По сути, Hilt — это «Dagger для Android», который делает внедрение зависимостей проще, безопаснее и более подходящим для мобильной разработки.

**Как настроить Hilt в приложении (@HiltAndroidApp)?**
Настройка Hilt начинается с аннотации @HiltAndroidApp на классе Application. Эта аннотация генерирует HiltApplication и подключает базовый компонент. Затем в Activity и Fragment добавляется @AndroidEntryPoint, а для ViewModel — @HiltViewModel. Модули с зависимостями помечаются @Module и @InstallIn (с указанием scope, например @InstallIn(SingletonComponent::class)). После этого Hilt автоматически создаёт и предоставляет зависимости через @Inject. В build.gradle необходимо добавить hilt-android и hilt-compiler плагины. Это позволяет полностью избавиться от ручного создания компонентов Dagger.

**Что такое @AndroidEntryPoint и @HiltViewModel?**
@AndroidEntryPoint — аннотация, которая помечает Activity, Fragment, Service или BroadcastReceiver как точку входа для Hilt. Она автоматически создаёт и инжектит зависимости в этот компонент. @HiltViewModel используется на классе ViewModel и позволяет Hilt автоматически создавать ViewModel с зависимостями через @Inject constructor. Без этой аннотации пришлось бы писать ViewModelFactory вручную. Обе аннотации значительно упрощают внедрение зависимостей в Android-компоненты и обеспечивают правильную работу с жизненным циклом.

**Разница между @Inject, @Provides и @Binds?**
@Inject используется на конструкторе класса или поле, указывая Hilt, что этот класс можно создать автоматически. @Provides применяется в модуле для предоставления экземпляров, которые нельзя создать через конструктор (например, интерфейсы, объекты из внешних библиотек или сложные зависимости). @Binds используется в модуле для связывания интерфейса с его реализацией (например, @Binds abstract fun bindRepository(impl: RealRepository): Repository). @Binds предпочтительнее @Provides, когда есть готовая реализация, так как генерирует меньше кода и работает быстрее.

**Какие scope существуют в Hilt (@Singleton, @ActivityRetainedScoped, @ViewModelScoped)?**
В Hilt существуют следующие основные scopes: @Singleton (живёт в течение всего процесса приложения), @ActivityRetainedScoped (живёт пока Activity не уничтожена полностью, переживает configuration changes), @ViewModelScoped (живёт вместе с ViewModel), @ActivityScoped, @FragmentScoped и @ServiceScoped. Scope определяет, сколько времени будет жить объект и сколько раз он будет создаваться. Правильный выбор scope критически важен для избежания утечек памяти и правильного управления жизненным циклом.

**Как работать с Assisted Injection в Hilt?**
Assisted Injection в Hilt позволяет передавать параметры в конструктор ViewModel или другого класса, которые не могут быть предоставлены через DI. Для этого используется аннотация @AssistedFactory на фабрике и @Assisted на параметрах. Hilt генерирует фабрику, которую можно инжектировать и использовать для создания экземпляра с дополнительными аргументами. Это особенно полезно для ViewModel, которым нужны параметры из Intent или Fragment arguments. С Hilt 2.31+ Assisted Injection работает очень удобно и интегрируется с @HiltViewModel.

**Как тестировать код с Hilt (@UninstallModules)?**
Тестирование кода с Hilt выполняется через @HiltAndroidTest и тестовый Application. Для замены модулей используется аннотация @UninstallModules (удаляет реальные модули) и @Module с тестовыми @Provides или @Binds. Hilt создаёт отдельный тестовый компонент. Для ViewModel и Fragment используются специальные правила (HiltAndroidRule). Это позволяет легко подменять реальные зависимости на моки (Repository, API и т.д.) и тестировать изолированно.

**Что такое @EntryPoint и когда он нужен?**
@EntryPoint — аннотация, которая позволяет получить доступ к Hilt-компоненту из кода, который не аннотирован @AndroidEntryPoint (например, из ContentProvider, BroadcastReceiver или старого кода). С помощью @EntryPoint создаётся интерфейс, через который можно получить нужные зависимости. Это необходимо в случаях, когда стандартный способ инъекции недоступен (например, в Application или в библиотечном коде). @EntryPoint используется редко, но критически важен в таких пограничных сценариях.

**Как правильно организовать модули в Hilt?**
Модули в Hilt рекомендуется организовывать по функциональным областям: отдельный модуль для Network (Retrofit, OkHttp), Repository, Database (Room), ViewModel и т.д. Каждый модуль помечается @Module и @InstallIn с соответствующим scope (SingletonComponent, ViewModelComponent и т.д.). Используйте @Binds для интерфейсов и @Provides только когда нужна сложная логика создания. Избегайте большого количества модулей в одном файле — лучше разбивать по пакетам. Это делает код модульным, легко тестируемым и понятным.

**Что такое Paging 3 и зачем оно нужно?**
Paging 3 — это библиотека Jetpack, предназначенная для эффективной загрузки больших списков данных по частям (pagination) с поддержкой оффлайн-режима, плейсхолдеров и автоматической обработки состояний загрузки. Она нужна для того, чтобы не загружать весь список в память сразу, особенно когда данных миллионы (например, лента новостей, чат, каталог товаров). Paging 3 интегрируется с RecyclerView, Room и RemoteMediator, обеспечивая плавную прокрутку, низкое потребление памяти и удобную обработку состояний (loading, error, empty).

**Из каких основных компонентов состоит Paging 3?**
Paging 3 состоит из трёх основных компонентов: PagingSource (источник данных, который загружает страницы), PagingData (контейнер для загруженных данных) и Pager (конфигурация и создание потока PagingData). Для UI используется PagingDataAdapter, который автоматически обрабатывает обновления списка. RemoteMediator используется для комбинирования локального кэша (Room) и сетевого источника. Всё это работает на базе Flow<PagingData<T>>.

**Как интегрировать Paging 3 с Room?**
Интеграция Paging 3 с Room осуществляется через метод DAO, который возвращает PagingSource<Key, Value> с помощью @Query с поддержкой LIMIT и OFFSET. Room автоматически генерирует PagingSource, который загружает данные из базы. Для синхронизации с сетью используется RemoteMediator, который решает, когда загружать данные из сети и сохранять их в Room. В итоге получается Flow<PagingData<T>>, который подаётся в PagingDataAdapter.

**Что такое PagingSource, PagingState, PagingData?**
PagingSource — это источник данных, который знает, как загружать следующую страницу по ключу (refreshKey). PagingState — состояние текущей загрузки (какая страница загружена, позиция и т.д.). PagingData — контейнер, содержащий загруженные страницы данных и инструкции по обновлению списка. PagingDataAdapter автоматически применяет эти инструкции к RecyclerView, обеспечивая эффективное обновление только изменившихся элементов.

**Как работает RemoteMediator?**
RemoteMediator — это класс, который решает, когда и как загружать данные из сети при работе с Paging 3 и Room. Он имеет метод load(), который вызывается при необходимости новой страницы. RemoteMediator проверяет локальный кэш, решает, нужно ли делать сетевой запрос, и сохраняет результат в Room. Он поддерживает три типа загрузки: REFRESH, PREPEND, APPEND. Это позволяет реализовать оффлайн-first подход с автоматической синхронизацией.

**Разница между PagingDataAdapter и обычным RecyclerView.Adapter?**
PagingDataAdapter — это специализированный адаптер для работы с Paging 3. Он автоматически обрабатывает загрузку страниц, состояния (loading, error), и использует DiffUtil для эффективного обновления только изменившихся элементов. Обычный RecyclerView.Adapter требует ручного управления загрузкой, пагинацией и обновлением списка. PagingDataAdapter значительно упрощает код и повышает производительность при работе с большими списками.

**Как добавить load state (header/footer) в Paging 3?**
Load state (header/footer) добавляется через метод addLoadStateHeader и addLoadStateFooter у PagingDataAdapter. Можно создать отдельный ViewHolder для состояний загрузки, ошибки или пустого списка. Paging 3 автоматически передаёт LoadState (Loading, Error, NotLoading) в эти компоненты. Это позволяет показывать progress bar, сообщение об ошибке или кнопку «Повторить» в начале или конце списка без сложной ручной логики.

**Как тестировать Paging 3?**
Тестирование Paging 3 осуществляется через PagingSourceTest и TestPager. Для PagingSource создаётся тестовый PagingSource и проверяются методы load() с разными LoadParams. Для всего потока используется runTest и PagingData с asPagingSourceFactory. Для UI-тестов используется Compose или Espresso с асинхронным ожиданием загрузки страниц. Библиотека предоставляет удобные методы для симуляции разных состояний (loading, error, success).

**Что такое Jetpack Compose и чем он отличается от XML?**
Jetpack Compose — это современный декларативный UI-фреймворк от Google, который позволяет описывать интерфейс с помощью аннотированных функций (@Composable), а не через XML-файлы. В отличие от XML, где UI описывается статично и обновляется императивно (findViewById, setText), Compose строит UI как функцию от состояния: при изменении состояния автоматически происходит recomposition только необходимых частей. Compose полностью написан на Kotlin, лучше интегрируется с Coroutines и StateFlow, и позволяет создавать сложные анимации и кастомные компоненты с меньшим объёмом кода.

**Что такое @Composable функция?**
@Composable — это аннотация, которая помечает функцию как composable-функцию, способную описывать часть UI и реагировать на изменения состояния. Composable-функции могут вызывать другие composable-функции, но не могут быть вызваны из обычного кода. Они являются основой Jetpack Compose и работают по принципу declarative UI: функция описывает, как должен выглядеть UI в зависимости от текущего состояния. При изменении состояния Compose автоматически перестраивает только изменившиеся части дерева UI.

**Как работает recomposition?**
Recomposition — это процесс повторного вызова composable-функций при изменении состояния. Compose отслеживает, какие composable зависят от какого состояния, и при изменении только этих состояний запускает recomposition только для затронутых частей дерева. Это делает обновление UI очень эффективным. Recomposition может происходить часто, поэтому важно писать composable-функции идемпотентно и избегать тяжёлых вычислений внутри них (для этого используется remember и derivedStateOf).

**Что такое State и mutableStateOf?**
mutableStateOf() — функция, которая создаёт наблюдаемое состояние (MutableState<T>). Когда значение состояния изменяется, Compose автоматически запускает recomposition всех composable, которые читают это состояние. State<T> — неизменяемая версия, которую обычно получают через by mutableStateOf(). В Compose состояние является источником правды: UI всегда отражает текущее значение State. Это заменяет старый подход с LiveData и setState в View.

**Разница между remember и rememberSaveable?**
remember — создаёт и сохраняет объект в памяти только на время текущей композиции (при recomposition объект не пересоздаётся). rememberSaveable сохраняет значение даже при уничтожении и пересоздании composable (например, при повороте экрана или Process Death), используя SavedStateHandle под капотом. rememberSaveable используется для состояния, которое должно пережить конфигурационные изменения (позиция скролла, текст в поле ввода).

**Что такое Side Effects (LaunchedEffect, DisposableEffect, SideEffect)?**
Side Effects в Compose — это механизмы для выполнения действий, которые выходят за пределы composable-функции (запуск корутин, подписка на Flow, работа с системными API). LaunchedEffect запускает suspend-блок при входе в композицию и отменяет его при выходе. DisposableEffect позволяет выполнять cleanup при выходе из композиции. SideEffect выполняется после каждой успешной recomposition. Эти эффекты позволяют безопасно работать с корутинами и внешними ресурсами внутри Compose.

**Как правильно использовать rememberCoroutineScope?**
rememberCoroutineScope() создаёт CoroutineScope, который автоматически отменяется при выходе composable из композиции. Он используется для запуска корутин внутри composable-функций (например, при нажатии кнопки). Scope привязан к композиции, поэтому при recomposition корутины не теряются, но отменяются при удалении composable. Это безопасный способ запуска асинхронных операций в Compose без утечек.

**Что такое CompositionLocal?**
CompositionLocal — это механизм для неявной передачи данных вниз по дереву composable без явного пробрасывания через параметры. Это аналог Context в Compose. Данные предоставляются через CompositionLocalProvider и читаются через Local...current. Пример: LocalContext, LocalDensity, LocalTheme. CompositionLocal используется для передачи тем, контекста, ViewModel и других глобальных значений, но его следует использовать осторожно, чтобы не нарушать принцип явной зависимости.

**Как работает Navigation в Compose?**
Navigation в Compose построена на NavHost и NavController. Навигация описывается декларативно внутри NavHost: каждый экран — это composable с route. Переход осуществляется через navController.navigate(route). Аргументы передаются через route с плейсхолдерами или SavedStateHandle. Compose Navigation полностью интегрируется с ViewModel и Lifecycle, автоматически обрабатывает Back Stack и сохраняет состояние экранов. Это значительно проще и безопаснее, чем Navigation Component с XML.

**Что такое Material 3 Theme в Compose?**
Material 3 Theme — это современная дизайн-система Google, реализованная в Compose. Она включает ColorScheme (светлая/тёмная тема с динамическими цветами), Typography, Shapes и другие компоненты Material. Тема настраивается через MaterialTheme и поддерживает dynamic color (Material You) на Android 12+. Material 3 обеспечивает единообразие дизайна, доступность и автоматическую адаптацию под системную тему устройства.

**Как интегрировать Compose с ViewModel?**
Интеграция Compose с ViewModel осуществляется через viewModel() делегат внутри composable и наблюдение за состоянием через collectAsState(). ViewModel предоставляет StateFlow или LiveData, которые composable-функция наблюдает. Для side effects используется LaunchedEffect и rememberCoroutineScope. Hilt автоматически инжектит ViewModel через @HiltViewModel. Это позволяет сохранять состояние при повороте экрана и Process Death.

**Как тестировать Compose UI?**
Тестирование Compose UI выполняется через createComposeRule() из androidx.compose.ui.test. Тесты используют семантические селекторы (onNodeWithText, onNodeWithTag, onNodeWithContentDescription). Проверяются действия (performClick, performTextInput) и утверждения (assertIsDisplayed, assertTextEquals). Compose тестирование намного проще и надёжнее, чем Espresso, так как работает напрямую с composable-деревом и не зависит от ID.

**LazyColumn vs RecyclerView — когда что использовать?**
LazyColumn (из Compose) рекомендуется использовать в новых проектах на Jetpack Compose — он проще, декларативен и автоматически обрабатывает recycling. RecyclerView остаётся предпочтительным в XML-based UI или когда нужна максимальная производительность и сложная кастомизация (например, сложные анимации, mixed layouts). LazyColumn лучше интегрируется с Paging 3 и StateFlow, но RecyclerView всё ещё может быть быстрее в очень сложных списках с большим количеством типов элементов.

**Что такое DataStore и чем он лучше SharedPreferences?**
DataStore — это современная, асинхронная и типобезопасная альтернатива SharedPreferences. В отличие от SharedPreferences, DataStore работает на корутинах и Flow, поддерживает транзакции, не блокирует Main Thread и имеет встроенные миграции. Preferences DataStore — для простых ключ-значение, Proto DataStore — для структурированных данных через Protocol Buffers. DataStore решает проблемы SharedPreferences (потеря данных при краше, отсутствие типобезопасности, блокировка при записи).

**Разница между Preferences DataStore и Proto DataStore?**
Preferences DataStore — это простой ключ-значение хранилище, аналогичное SharedPreferences, но асинхронное и типобезопасное. Proto DataStore использует Protocol Buffers для хранения структурированных данных с версионированием схемы и автоматическими миграциями. Preferences подходит для простых настроек, Proto — для сложных объектов (пользовательские настройки, фильтры, конфигурации). Proto DataStore обеспечивает лучшую типобезопасность и эволюцию схемы.

**Что такое Macrobenchmark и как его использовать?**
Macrobenchmark — это инструмент для измерения реальной производительности приложения (время запуска, прокрутка списка, переходы между экранами). В отличие от Microbenchmark, который тестирует отдельные функции, Macrobenchmark запускает реальное приложение и измеряет метрики на устройстве. Используется для отслеживания регрессий производительности и оптимизации. Тесты пишутся с помощью MacrobenchmarkRule и запускаются на физическом устройстве или эмуляторе.

**Что такое Security Jetpack (EncryptedSharedPreferences, EncryptedFile)?**
Security Jetpack — это набор библиотек для безопасного хранения данных. EncryptedSharedPreferences — зашифрованная версия SharedPreferences с использованием Master Key из Android Keystore. EncryptedFile — зашифрованный файл на диске. Обе библиотеки защищают данные от чтения при root-доступе или физическом извлечении файла. Они используют AES256 и автоматически управляют ключами, значительно упрощая безопасное хранение токенов, паролей и чувствительных настроек.

**Что такое CameraX и какие преимущества оно даёт?**
CameraX — это Jetpack-библиотека, которая значительно упрощает работу с камерой на Android. Она предоставляет унифицированный API поверх Camera2, автоматически обрабатывает различия между устройствами, поддерживает use cases (Preview, ImageCapture, ImageAnalysis, VideoCapture). Преимущества: простота использования, lifecycle-aware, автоматическая обработка поворота экрана, поддержка extensions (HDR, ночной режим), и высокая стабильность. CameraX позволяет сосредоточиться на бизнес-логике, а не на особенностях разных производителей устройств.

**Как правильно организовать многослойную архитектуру с Jetpack?**
Многослойная архитектура с Jetpack обычно строится по принципу Clean Architecture или MVVM: Presentation слой (Compose или XML + ViewModel), Domain слой (UseCase/Interactor), Data слой (Repository, Room, Retrofit). Jetpack помогает через ViewModel (управление состоянием), Hilt (DI), Room (локальное хранение), WorkManager (фоновая работа) и Navigation. Repository выступает Single Source of Truth. Всё строится на Coroutines/Flow и lifecycle-aware компонентах. Такая архитектура обеспечивает тестируемость, масштабируемость и разделение ответственности.

**Что такое Single Source of Truth в контексте Jetpack?**
Single Source of Truth — принцип, при котором все данные в приложении имеют единственный источник истины, обычно локальную базу данных (Room). Даже данные из сети сначала сохраняются в Room, а UI наблюдает за Room через Flow/StateFlow. Jetpack помогает реализовать этот принцип через Repository, который всегда читает из Room и обновляет сеть при необходимости. Это обеспечивает согласованность данных, оффлайн-поддержку и упрощает отладку.

**Как правильно использовать Repository с Room + Retrofit?**
Repository выступает посредником между источниками данных. Правильный подход: метод Repository возвращает Flow или suspend-функцию. Сначала проверяется локальный кэш (Room), если данных нет или они устарели — выполняется сетевой запрос (Retrofit), результат сохраняется в Room, после чего данные возвращаются из Room. Это реализует оффлайн-first подход. Repository инжектируется через Hilt и использует Coroutines для асинхронности.

**Какие антипаттерны существуют при использовании Jetpack?**
Частые антипаттерны: использование GlobalScope вместо viewModelScope или lifecycleScope, хранение состояния в Activity/Fragment вместо ViewModel, прямое использование Coroutines в UI-слое без диспетчеров, игнорирование Structured Concurrency, чрезмерное использование LiveData вместо StateFlow, создание огромных ViewModel (God ViewModel), отсутствие обработки отмены корутин, неправильное использование @Transaction в Room и игнорирование lifecycle-aware компонентов.

**Как правильно обрабатывать ошибки в Jetpack-компонентах?**
Ошибки в Jetpack-компонентах рекомендуется обрабатывать через sealed class Result или UiState (Loading, Success, Error). В Repository исключения перехватываются и преобразуются в Error-состояние. В ViewModel ошибки обрабатываются в корутинах с try-catch и эмитируются в StateFlow/SharedFlow. Для Flow используется catch() оператор. В UI ошибки отображаются через collectAsState(). Важно не бросать исключения вверх, а обрабатывать их явно на каждом слое.

**Как тестировать весь слой Presentation (ViewModel + Compose)?**
Тестирование Presentation слоя включает unit-тесты ViewModel (runTest + Turbine) и UI-тесты Compose (createComposeRule). ViewModel тестируется изолированно с моками Repository. Compose тестируется через семантические селекторы и проверку состояний. Для end-to-end тестирования используется Hilt с тестовыми модулями (@UninstallModules). Важно проверять как состояние (StateFlow), так и события (SharedFlow), а также навигацию и side effects.

**Как Jetpack помогает избежать memory leaks?**
Jetpack помогает избежать memory leaks через lifecycle-aware компоненты: LiveData/StateFlow автоматически отписываются при уничтожении LifecycleOwner, ViewModel переживает configuration changes, Coroutines отменяются через scope (viewModelScope, lifecycleScope), repeatOnLifecycle ограничивает сбор Flow нужным состоянием. Hilt правильно управляет scoped зависимостями. В целом Jetpack поощряет использование immutable состояния и Structured Concurrency, что значительно снижает риск утечек по сравнению с ручным управлением.